

CSIT321 Cover Sheet

Subject Code: CSIT321
Subject Name: Capstone Project
Submission Type: Final Report
Project Title: Adaptiv Health - Cardiovascular System
Team Name: Team Pulse
Student's names Omar Massoud, Abdallah Heggy, Ghina Ishaq, Hamza Al Raga,
Bithin Radhakrishnan, Karylle Ochave
Student's Number 7858681, 8457785, 871179, 7322586, 8910881, 7669057
Lecturer Name: Prof. Farhad Oroumchian
Due Date: 14/04/2026
Date Submitted: 13/04/2026

PLAGIARISM: The penalty for deliberate plagiarism is FAILURE in the subject. Plagiarism is cheating by using the written ideas or submitted work of someone else. UOWD has a strong policy against plagiarism. The University of Wollongong in Dubai also endorses a policy of nondiscriminatory language practice and presentation. PLEASE NOTE: STUDENTS MUST RETAIN A COPY OF ANY WORK SUBMITTED Optional Marks:	DECLARATION: I/We certify that this is entirely my/our own work, except where I/we have given fully-documented references to the work of others, and that the material contained in this document has not previously been submitted for assessment in any formal course of study. I/we understand the definition and consequences of plagiarism. We/I declare that the project proposal has not been used in any UOWD courses before and that this project idea is a total new idea of the project team Signature of Student: Mohammed Atwani
---	---

Website Link: <https://fast-lion-17969821.figma.site>

Youtube Video: <https://youtu.be/94Xa0Vf3XII?si=9eM6FwWz0cVbUVHd>

Github Repo Link: <https://github.com/Massoud3/AdaptivHealth>

Implementation Report	6
1. Introduction	6
2. System Architecture Overview	6
3. AWS Cloud Infrastructure	7
3.1 Infrastructure Components	7
3.2 VPC Network Design	8
3.3 EC2 Server Configuration	8
3.4 NGINX Configuration	9
4. Backend Implementation (FastAPI)	9
4.1 Project Structure	10
4.2 Authentication System	10
4.3 Data Encryption	11
4.4 API Endpoints	11
5. Database Implementation	13
5.1 Database Schema	13
5.2 Data Retention and Compliance	14
6. Machine Learning Implementation	14
6.1 Model Training	14
6.2 Risk Classification	15
6.3 On-Device Inference (Pure Dart)	15
6.4 Gemini AI Recommendation Pipeline	16
7. Flutter Mobile Application	16
7.1 Application Architecture	16
7.2 Key Screens.	17
7.3 Offline Capability	17
8. React Clinician Dashboard	18
8.1 Dashboard Screens	18
8.2 Authentication and Security	19
9. Security Implementation Summary	19
10. How to Run the System	21
10.1 GitHub Repository	21
10.2 Backend Setup	21
10.3 Flutter App Setup	22
10.4 React Dashboard Setup	22
10.5 Production System	22
11. Conclusion	22
Change Report	23
1. Purpose of This Document	23
2. Summary of All Changes	23
3. Detailed Change Descriptions	26
3.1 Backend Framework — Flask to FastAPI	26
3.2 Machine Learning Model — TensorFlow/LSTM to Random Forest	26

3.3 Database Architecture — Firebase + AWS to AWS RDS Only	27
3.4 Security Implementation — Planned to Delivered	28
3.5 Scope Changes — Features Added	30
3.6 Wearable Integration Approach	31
4. Scope Change Summary	31
5. What Remained Unchanged	31
6. Conclusion	32
Test Results Report	33
1.1 Test Enviroment	33
1.2 Summary of Results	33
2. Functionality Testing	34
3. Database Testing	36
4. UX/UI Testing	37
5. Security Testing	39
6. Machine Learning Testing	40
Conclusion	41
Semester 1 Reports + Test Plan Report	42
Report 1: Planning and Feasibility Analysis	42
1. Background Info about topic	42
2. Project Scope	42
Out of Scope	43
Target Audience	44
Secondary Users:	44
Competitor Analysis	44
Key Differentiator:	45
Limitations	45
Prerequisites	46
Expected Impact	47
Gantt Chart	47
3. Project Description and Objectives	47
Project Overview	47
System Experience:	48
Objectives	48
4. How We Do It	48
System Architecture (IoT → Cloud → AI → Dashboard)	48
Workflow and User Interaction	50
How the Design Ensures Feasibility	51
5. Software and Hardware requirements	52
Software Requirements:	52
Backend: Development & AI Technologies	52
Frontend Technologies	52
Hardware Requirements:	54
Challenges	55
Future Improvements	57

6. Project management tools to be used	58
Tools and Platforms:	58
Team Coordination	58
7. Diagrams:	59
8. Teamwork Distribution	63
Report 2: Requirements Analysis (SRS)	65
1. Introduction	65
1.1. Purpose	65
1.2. Notation	65
1.3. Scope	65
1.4. Context Diagram	66
1.5. Definitions and Acronyms	66
1.7. Overview	66
2. General Description	67
2.1. System Functions	67
2.2. User Characteristics	68
3. General Constraints	69
3.1. Software Constraints	69
3.2. Hardware Constraints	70
4. Assumptions and Dependencies	71
4.1 Assumptions	71
4.2 Key Dependencies and Regulatory Dependencies	72
5. Functional Requirements Master List	72
6. Functional Requirement REQ-1.1	73
6.1 REQ-1.1 – Real-Time Heart Rate Monitoring	73
6.2 REQ-1.2 – Adaptive AI Exercise Feedback	74
6.3 REQ-1.3– Emergency Alert System	74
6.4 REQ-1.4 – Data Visualization Dashboard	75
6.5 REQ-1.5 – User Authentication and Data Encryption	75
7. External Interface Requirement	76
7.1 Data Interfaces	76
7.2 User Interfaces	76
7.3 Other Interfaces	77
8. Non-Functional Requirements	77
8.1 System Performance	77
8.2 Information Security	78
8.3 Availability	79
8.4 Capacity	79
Report 3: System Design Document	81
High-Level Architecture	81
3.1 Main Architecture Diagram Description	82
3.2 Subsystem Descriptions	82
3.3 Reducing design costs through trade-offs and design rationale.	83
Section 4: Data and Logic Design	84

4.1 Data Dictionary	85
4.2 Data Flow Description	86
Section 5: System Component Algorithms and Pseudocode	87
HR_SignalEnhancement()	87
Segmentation Algorithm	87
Scoring Algorithm	88
Method Description	88
Wireframes:	92
User workflow:	92
Requirements Traceability Matrix:	93
Report 4: Test Plan	95
1. Functionality Testing	95
1.1 Test ID and Title	95
1.2 Introduction and Objective	95
1.3 Scope of Testing	96
1.4 Test Objectives and Approach	96
1.5 Test Schedule and Milestones	97
1.6 Test Environment Setup	98
1.7 Resources and Responsibilities	99
1.8 Test Deliverables	99
1.9 Entry and Exit Criteria	99
1.10 Risks and Mitigation Strategies	100
2. Database Testing	100
2.1 Test ID and Title	100
2.2 Introduction and Objective	100
2.3 Scope of Testing	101
2.4 Test Objectives and Approach	102
2.5 Test Schedule and Milestone	103
2.6 Test Environment Setup	103
2.7 Resources and Responsibilities	104
2.8 Test Deliverables	105
2.9 Entry and Exit Criteria	105
2.10 Risks and Mitigation Strategies	106
3. UX/UI Testing	106
3.1 Test ID and Title	106
3.2 Introduction and Objective	106
3.3 Scope of Testing	108
3.4 Test Objectives and Approach	108
3.5 Test Schedule and Milestones	109
3.6 Test Environment Setup	110
3.7 Resources and Responsibilities	111
3.8 Test Deliverables	111
3.9 Entry and Exit Criteria	112
3.10 Risks and Mitigation Strategies	112

4. Security Testing	112
4.1 Test ID and Title	112
4.2 Introduction and Objective	113
4.3 Scope of Testing	113
4.4 Test Objectives and Approach	114
4.5 Test Schedule and Milestones	115
4.6 Test Environment Setup	116
4.7 Resources and Responsibilities	117
4.8 Test Deliverables	117
4.9 Security Traceability Matrix	118
4.10 Risks and Mitigation Strategies	123
5. Machine Learning Testing	125
5.1 Test ID and Title	125
5.2. Introduction and Objective	125
5.3 Scope of Testing	126
5.4 Test Objectives and Approach	128
5.5 Test Schedule and Milestones	130
5.6 Test Environment Setup	132
5.7 Resources and Responsibilities	134
5.8 Test Deliverables	136
5.9 Entry and Exit Criteria	139
5.10 Risks and Mitigation Strategies	141
Screenshots	147
Flutter App	147
Home screen showing 192 BPM alert + SpO2 + BP (shows real-time monitoring + alert working)	147
BLE device pairing screen (shows wearable connectivity)	148
Onboarding screens	149
Home dashboard "Good morning Alice" + vitals display	150
Fitness/Workout screen + active workout session	151
Recovery screen + Sleep screen	152
Profile showing On-Device AI active (v2.0-rf, Pure-Dart Random Forest) + Health Insights with Edge AI score	153
Doctor Dashboard	155
Overview + patient management table	155
Backend/System	157
VPC resource map + API auth showing 401 Unauthorized	157
PostgreSQL terminal showing RBAC roles (PATIENT/CLINICIAN/ADMIN) + Swagger UI	158
API responses showing risk score (0.16 LOW) + recommendation JSON	159
All References	161

How to copy the Project Code

Repository Link: <https://github.com/Massoud3/AdaptivHealth>

Steps to copy to GitHub account:

1. Log in to your GitHub account
2. Go to: <https://github.com/new/import>
3. In "Your source repository URL",
paste: <https://github.com/Massoud3/AdaptivHealth>
4. Set the repository name (e.g. AdaptivHealth)
5. Set visibility to Public or Private
6. Click "Begin import"

Implementation Report

1. Introduction

This Implementation Report records the construction, setup and implementation of the Adaptiv Health cardiovascular monitoring system during the Semester 2. It discusses the technical realization of each key element of the system: the AWS cloud infrastructure, the FastAPI back-end, the Random Forest machine learning model, the Flutter patient mobile app, and the React clinician dashboard. The report should be a technical documentation of the build process, as well as a manual to any person who would want to learn or expand the system.

The system is fully deployed and accessible at:

<https://api.back-adaptivhealthuowd.xyz>

2. System Architecture Overview

Adaptiv Health system is constructed based on a layered architecture that links four main units: the patient mobile application, the clinician web dashboard, the cloud and the database. Any external traffic is encrypted with the TLS and passed through the AWS API Gateway and then sent to the application servers.

The complete request path for all API calls is:

```
Client (Flutter/React)
→ HTTPS (TLS 1.3)
→ AWS API Gateway (REST) + VPC Link
→ Internal Application Load Balancer (ALB)
→ EC2 NGINX (HTTPS, Let's Encrypt, port 443)
→ Uvicorn (127.0.0.1:8080, localhost only)
→ FastAPI application
→ AWS RDS PostgreSQL (private VPC subnet)
```

The FastAPI server does not connect to the internet directly; it is limited to 127.0.0.1 (localhost). All requests are forwarded internally using NGINX, which terminates TLS. The EC2 instance serves as the sole means of accessing the database hosted on Amazon RDS, which does not have a public IP address.

3. AWS Cloud Infrastructure

The AWS region ap-south-1 (Mumbai) is the complete location of the entire cloud infrastructure. This particular region was selected as being the nearest HIPAA-compliant

AWS region relative to the UAE due to the Dubai Health Authority regulation requiring that any data collected within the UAE be stored in a nearby region.

3.1 Infrastructure Components

Component	Service	Configuration	Purpose
Application Server	EC2 (t3.micro)	Instance ID: i-05c712521643ca277 Private IP: 10.0.1.110 Public subnet	Hosts FastAPI backend, NGINX reverse proxy, ML model
Database	RDS PostgreSQL	Private subnet (10.0.2.0/24) No public IP AES-256 encryption at rest	Stores all PHI and application data
API Entry Point	API Gateway (REST)	VPC Link to internal ALB HTTPS only	Manages all external API traffic, rate limiting, logging
Load Balancer	ALB (internal)	Internal scheme HTTPS listener Forwards to EC2:443	Routes API Gateway traffic to EC2 within VPC
TLS Certificate	ACM + Let's Encrypt	ACM for ALB Let's Encrypt (Certbot) on EC2	End-to-end HTTPS within and outside VPC
Domain	Porkbun Registrar	back-adaptivhealthuowd.xyz A record → EC2 public IP	Public DNS for the API endpoint
Audit Logging	CloudTrail	All API calls logged S3 bucket storage 6-year retention	HIPAA §164.312(b) compliance
Monitoring	CloudWatch	System metrics Alert thresholds Log aggregation	Infrastructure health and anomaly detection

Network	VPC	10.0.0.0/16 CIDR Public + private subnets Internet Gateway	Isolates database from internet
---------	-----	--	---------------------------------

3.2 VPC Network Design

This network segmentation strategy used in developing the Virtual Private Cloud (VPC) consists of using two subnets in such a way that network level separation is ensured between public and private-facing resources:

- The EC2 application server resides in the Public Subnet (10.0.1.0/24). It is able to handle HTTPS requests and any requests passed through to it from API Gateway through the VPC Link since it has an established connection to the Internet Gateway.
- The RDS PostgreSQL database resides in the Private Subnet (10.0.2.0/24), which does not have a public IP address nor is connected to the Internet Gateway. The only one that can connect to the database in this case is the EC2 instance residing in the same VPC.

This distinction is maintained through the use of security groups where the internet gateway and all other sources except for the EC2 security group are denied access.

3.3 EC2 Server Configuration

The operating system running on the EC2 instance is Amazon Linux 2023. Application stack components include:

- Virtual environment for Python 3.11 at /home/ec2-user/adaptivhealth/venv/.
- FastAPI application available at /home/ec2-user/adaptivhealth/.
- The Uvicorn ASGI server bound only to 127.0.0.1:8080.
- NGINX configured as a reverse proxy with a TLS connection from Let's Encrypt running on port 443.

adaptiv-api.service, which is a service file managed by systemd controlling the application stack, ensures automatic start on boot and automatic restart on failure:

```
[Unit]
Description=AdaptivHealth FastAPI (Uvicorn)
After=network.target

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/adaptivhealth
Environment=PATH=/home/ec2-user/adaptivhealth/venv/bin
Environment=ENVIRONMENT=production
ExecStart=.../venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8080
```

```
Restart=always
RestartSec=3
```

Swagger UI (/docs), ReDoc (/redoc), and OpenAPI schema (/openapi.json) are blocked through the use of the ENVIRONMENT=production variable, making sure that API structure information remains private in the production environment.

3.4 NGINX Configuration

NGINX acts as the reverse proxy and TLS termination point. It listens on port 443 with a Let's Encrypt certificate obtained via Certbot, and forwards all requests to the Uvicorn server on localhost:8080. Key configuration elements include:

- SSL/TLS termination with Let's Encrypt certificate (auto-renewed via Certbot cron)
- Proxy headers (X-Real-IP, X-Forwarded-For) passed to FastAPI for request logging
- Connection timeouts configured for long-running cardiovascular session uploads
- HTTP to HTTPS redirect enforced for all port 80 requests

4. Backend Implementation (FastAPI)

The back end consists of a FastAPI app written in Python. This API is RESTful in nature and provides functionalities for performing various actions within the system. FastAPI was chosen because of its support for async programming, request validation with Pydantic, and minimal resources consumed compared to the Django framework.

4.1 Project Structure

Directory / File	Purpose
app/main.py	FastAPI application entry point, router registration, CORS, middleware
app/api/v1/endpoints/	API route handlers (auth, users, vitals, alerts, sessions, recommendations)
app/models/	SQLAlchemy ORM models mapping to PostgreSQL tables

app/schemas/	Pydantic schemas for request validation and response serialisation
app/services/	Business logic (encryption service, AI service, alert service)
app/core/security.py	JWT creation/validation, Argon2id hashing, JTI blocklist
app/db/session.py	SQLAlchemy database session management
app/ml/	Random Forest model files and inference pipeline
alembic/	Database migration scripts
.env	Environment variables (DB URL, secret key, Gemini API key)

4.2 Authentication System

The use of JWTs and the creation of a JTI (JWT ID) blocklist allow for token revocation during user logouts. The process is as follows:

- The user sends a POST request to `/api/v1/auth/login` with the password and email.
- FastAPI checks whether the entered password matches the Argon2id hash stored.
- In case of a successful login, a JWT containing the user ID, role, and JTI (a randomly generated UUID) is issued.
- The returned JWT is stored locally by the client.
- The JWT is provided in each subsequent authenticated request using the `Authorization: Bearer` header.
- The FastAPI middleware checks whether the JWT ID exists in the blocklist and performs other token validation procedures.
- During a logout event, the JTI is added to the blocklist, making the token instantly invalid.

FastAPI Dependency Injection is applied to enforce Role Based Access Control (RBAC). The dependency checks the role of the authorized user before allowing the request to proceed, with each endpoint defining the roles that can access it. Doctors do not have access to patients' endpoints, nor can patients access doctors' endpoints.

4.3 Data Encryption

Prior to storing into PostgreSQL, all the Protected Health Information (PHI) stored within the database will be encrypted at the application layer with the help of AES-256-GCM algorithm. AES-256-GCM was chosen due to its ability to offer authentication along with encryption, meaning it produces an authentication tag alongside the encrypted information to authenticate data integrity post-decryption process. Any modification done to any bit will result in the failure of decryption process.

The encryption process for each PHI field involves:

- Generating a random nonce of size 96 bits for each encryption process.
- Encrypting the plaintext with AES-256-GCM by using the application master key.
- Storing nonce, ciphertext, and authentication tag together.
- Decrypting and verifying the value with the use of nonce and tag.

Since Argon2id uses memory-hard settings for password hashing, brute force attacks require a lot of memory along with computing capacity. No plain text password is stored within the system.

4.4 API Endpoints

Endpoint	Method	Role	Description
/api/v1/auth/login	POST	All	Authenticate user, return JWT
/api/v1/auth/logout	POST	All	Invalidate JWT via JTI blocklist
/api/v1/auth/register	POST	All	Register new patient account
/api/v1/users/me	GET	All	Get current authenticated user profile
/api/v1/users/{id}	GET	Doctor/Admin	Get specific patient profile
/api/v1/vitals/	POST	Patient	Submit new vital sign reading

/api/v1/vitals/user/{id}	GET	Doctor/Admin	Get patient vital sign history
/api/v1/alerts/	GET	Doctor/Admin	List all active alerts
/api/v1/alerts/{id}	PATCH	Doctor	Acknowledge or resolve alert
/api/v1/sessions/	POST	Patient	Create new activity session
/api/v1/sessions/user/{id}	GET	Doctor/Admin	Get patient session history
/api/v1/recommendations/	GET	Patient	Get AI health recommendation
/api/v1/risk/	POST	Patient	Run risk classification on vitals
/health	GET	Public	System health check endpoint

5. Database Implementation

AWS RDS Postgres database is situated within a private subnet that cannot be accessed by any member of the general public. All communication between the FastAPI application and the database is done using SSL/TLS. It was easy to implement changes in the schema because it was built using SQLAlchemy ORM models and managed through Alembic database migrations.

5.1 Database Schema

Table	Key Fields	Description
users	id, email (encrypted), name (encrypted), role, password_hash, created_at	All system users: patients, doctors, admins

vital_sign_records	id, user_id, heart_rate, spo2, blood_pressure (encrypted), timestamp, source_device	Physiological readings submitted by patient app
activity_sessions	id, user_id, start_time, end_time, activity_type, avg_hr, peak_hr, recovery_time	Workout session records with performance metrics
risk_assessments	id, user_id, risk_level, risk_score, model_version, assessment_date	ML model outputs stored per session
exercise_recommendations	id, user_id, activity, intensity_level, duration, valid_until, source	AI-generated or rule-based recommendations
alerts	id, user_id, alert_type, severity, message, alert_time, acknowledged_by, acknowledged_at	System-generated safety alerts
jti_blocklist	id, jti, user_id, invalidated_at, expires_at	Tracks invalidated JWT tokens for logout support
medical_history	id, user_id, conditions, medications, emergency_contact (encrypted)	Patient medical background

5.2 Data Retention and Compliance

- All PHI data, including email addresses, names, blood pressure information, emergency contacts, and medical history, are encrypted before they are stored using the application-layer encryption with AES-256-GCM.
- In addition to this, AES-256 is used by AWS RDS for storing information.
- Manual backups are done for six years, while automatic backups are saved for seven days according to the DHA requirements.
- Every action taken on the database is logged using AWS CloudTrail, creating an immutable audit log.
- To retain performance, expired tokens are automatically removed from the JTI blacklisted table.

6. Machine Learning Implementation

Adaptiv Health uses a Random Forest classifier in its machine learning algorithm for risk stratification, which was trained using datasets from patients with cardiovascular diseases. This algorithm is developed using two approaches: the on-device classifier, which is completely created in Dart programming language and operates inside the Flutter app; and the cloud-based classifier, which works inside the FastAPI back-end on EC2.

6.1 Model Training

The Cardiovascular Dataset from PhysioNet & Kaggle was utilized for training the Random Forest model with Scikit-Learn. The following steps were included in the training pipeline:

- Data loading and preprocessing: any null values are removed; outliers are removed using the Z-score method; missing values are imputed using KNNImputer, where $k = 5$ and the similarity of signals is taken into consideration.
- Feature generation: the age of the patient, resting heart rate at baseline, heart rate variability (HRV), heart rate (BPM), and SpO2 (%) constitute the feature set. All features are scaled between 0 and 1 before training and inference using the StandardScaler method.
- Heart Rate Signal Smoothing: Prior to passing on to the machine learning algorithm, the actual heart rate measurements were smoothed via 5-point EMA (smoothing factor=0.3).
- Model setup parameters include 100 trees, gini impurity criterion, and optimized maximum depth from cross-validation. Since the consequences of failing to capture an important heart rate measurement are much worse compared to getting an alarm when the patient does not need it, we prioritized model optimization for recall score of HIGH risk patients.
- Anomaly Detection: To detect drastic changes in data that are not necessarily flagged by the ML model (i.e., rapid increase in heart rate from 75 bpm to 200 bpm) and indicate sensor malfunctions or emergency situations, independent Z-score and jump-based approaches were developed.

6.2 Risk Classification

Risk Score Range	Classification	System Response
0.80 – 1.00	HIGH	Immediate on-screen alert with vibration; cloud alert created; doctor dashboard notified; alert persists until acknowledged

0.50 – 0.79	MODERATE	Warning displayed on app; recommendation to slow down or rest; logged but does not trigger doctor alert
0.00 – 0.49	LOW	Normal activity continues; reading logged; no alert generated

6.3 On-Device Inference (Pure Dart)

The entire tree hierarchy consisting of all 100 decision trees, complete with node details, threshold values, child node indices, and leaf class probabilities, is serialized to a JSON file from the scikit-learn Random Forest model. This JSON file is included as part of the assets in the Flutter application.

This JSON file is loaded into memory during the app initialization stage by the pure Dart inference engine (EdgeMLService). The prediction is done via tree traversal without any need for native code. Inference process:

- Relies on a companion JSON file that contains the StandardScaler parameters (mean and standard deviations) per feature.
- Relies on the exact same scaler that was trained to normalize input features.
- Feeds the normalized input through each of the 100 decision trees, starting at the root node and going through to the leaf nodes.
- Generates class probabilities for each tree.
- Determines the risk score and class prediction by averaging the probabilities.

The Dart version executes on modern smartphones in 5–15 ms, easily meeting the goal of less than 160 ms latency. Critically, the system functions entirely offline; no internet connectivity is required to generate risk scores, allowing users to be alerted in areas lacking cellular coverage.

6.4 Gemini AI Recommendation Pipeline

The narrative health recommendation generated by the system after each patient's monitoring is done via the Gemini AI by Google. A dual-channel method is adopted in implementing the same:

- Main channel: The data obtained from the patient's session (heart rate average, peak heart rate, risk score, and activity) are provided as context when querying the Gemini API. In order to provide concise and relevant advice, the temperature of the model is set to 0.4 (meaning low randomness) with a limit of 300 tokens. The prompt is such that it portrays the model as a health tool and not for diagnosis.
- Secondary channel: In case the connection to the Gemini API is cut off (due to network issues or quota exceeding), the program automatically switches to using a

rules engine to provide the patient with health advice based on their risk scores and session results.

Whether there are external APIs available or not, our two-stream design will ensure that there is always an option available to provide recommendations.

7. Flutter Mobile Application

Flutter has been employed for developing the patient-side mobile application that provides a common codebase operating on iOS and Android devices. The real-time display of vital signs, machine learning on device inference, queuing of offline data, geolocation detection for emergency response, and access to the backend API are handled by the application.

7.1 Application Architecture

State management is achieved through Flutter's Provider package, while the app implements the service-oriented architecture. Some important services are:

- **ApiService:** It manages all HTTPS requests to the backend via the Dio HTTP client package. The API endpoint is set to be: <https://api.back-adaptivhealthuowd.xyz>, and a JWT Bearer token stored in the safe local storage is attached to every request.
- **EdgeMLService** acts as the Pure-Dart Random Forest inference engine and performs the risk categorization on device within 5-15 ms, following the loading of model and scaler JSONs.
- **OfflineSyncService:** In the case of lost internet connection, it stores all data regarding the session and vital signs in a queue that gets emptied and sent back to the server in chronological order when the network is regained.
- **AlertService:** It triggers vibration sequences and messages based on the risk scores outputted by EdgeMLService during the identification of the HIGH danger.
- **HealthBridgeService:** Collects information from linked wearables and reads the information related to heart rate, SpO2, and activities from the inbuilt health application of the phone

7.2 Key Screens.

Screen	Features	Data Source
Home	Real-time HR (BPM), SpO2, BP display; colour-coded safety zones (green/amber/red); HR trend graph; EMA-smoothed readings	HealthBridgeService + EdgeMLService

Workout Guidance	Daily wellness check-in (Good/Okay/Tired); AI-generated exercise plan; Start/Stop session; live risk score display during workout	Backend API + EdgeMLService
Recovery	Post-session breathing exercises; daily wellness tips; HR recovery trend graph; AI narrative recommendation from Gemini	Backend API
Alerts	Threshold breach notifications; vibration and audio alert; alert history; option to contact emergency contact	AlertService + EdgeMLService
Profile	Patient medical history; wearable device connection status; notification settings; data export	Backend API

7.3 Offline Capability

Essential safety features are guaranteed to be available even without internet connection due to the offline-first approach:

- On-device Random Forest inference: With the use of the pure Dart implementation, risk scoring is done entirely on-device, enabling continuous risk assessment without the need for an internet connection.
- Local data queue: If there is no internet connection, vital signs measurements will be stored locally in a timestamped queue. Before being stored locally, the queue will first be encrypted with AES-256.
- Automatic synchronization: In order not to lose any data, the app automatically synchronizes the measurements from the queue to the backend as soon as there is an internet connection.
- Threshold alerts: Basic heart rate and SpO2 threshold alerts are done locally independent of the internet and the machine learning algorithm.

8. React Clinician Dashboard

The react, TypeScript, and Tailwind CSS technologies were used to create the web-based interface for the clinicians. The tool is equipped with features like real-time patient monitoring, alert management, and population data analysis capabilities.

8.1 Dashboard Screens

Screen	Role	Features
Overview	Doctor, Admin	Summary cards (active alerts, average health score, average HR); quick-access patient status summary; population trend indicators
Patient Management	Doctor, Admin	Searchable patient list with health score, status badge, age, and last reading time; filter by status (Critical/Warning/Stable); click to view patient detail
Patient Detail	Doctor	Full patient profile; interactive HR and SpO2 trend graphs; AI recommendations history; session history; alert acknowledgement
Alerts Panel	Doctor, Admin	Prioritised list of all active alerts by severity; one-click acknowledgement; alert history with resolution timestamps
Analytics	Doctor, Admin	Population-level cardiovascular analysis; health score distribution charts; population HR trends; recovery vs activity scatter plots; sleep quality analysis
Care Team	Admin	Team member management; role assignment; department settings
Settings	Doctor, Admin	Profile settings; notification preferences (critical alerts, warnings, SMS, email); security and privacy settings; two-factor authentication; data management

8.2 Authentication and Security

- **JWT-Based Authentication:**
The token is appended to each API request using the header `Authorization: Bearer` after logging in, and it is stored in memory, not `localStorage`.
- **Role-Based Rendering:**

Based on the user role, components are conditionally rendered. Not only is it restricted at the API endpoint, but certain admin-level features are not displayed in the UI for any doctor account.

- To ensure additional security, Two-Factor Authentication is available through the settings module in the application.
- Session Timeout:
To avoid accessing the application with an expired token, and accessing patient information that way, the application routes to the login page when the token expires.
- CORS:
In order to avoid cross-origin requests, the FastAPI backend only accepts requests originating from the dashboard URL.

9. Security Implementation Summary

All layers of the system were made secure, adhering to the UAE DHA, GDPR, and HIPAA policies. All information on the security plan is outlined in the table below:

Security Control	Implementation	Standard Met
Password hashing	Argon2id (memory-hard, PHC winner)	OWASP, NIST SP 800-63B
PHI encryption at rest	AES-256-GCM per-field application-layer encryption	HIPAA §164.312(a)(2)(iv)
Storage-layer encryption	AWS RDS AES-256 encryption	HIPAA §164.312(a)(2)(iv)
Data in transit	TLS 1.3 on all client-server communications	HIPAA §164.312(e)(2)(ii)
Authentication	JWT with Argon2id-hashed passwords	HIPAA §164.312(d)
Token revocation	JTI blocklist for immediate logout invalidation	OWASP Session Management
Authorisation	RBAC with three roles: Patient, Doctor, Admin	HIPAA §164.312(a)(1)

Database isolation	RDS in private VPC subnet, no public IP	AWS Well-Architected
API protection	FastAPI on 127.0.0.1 only, NGINX proxy, API Gateway	Defense in depth
Audit logging	AWS CloudTrail, all API calls logged, 6-year retention	HIPAA §164.312(b)
Monitoring	AWS CloudWatch alerts for anomalies and failures	HIPAA §164.308(a)(1)
Production hardening	Swagger/Redoc/OpenAPI disabled in production	Security best practice
SQL injection	SQLAlchemy ORM with parameterised queries	OWASP Top 10 A03
Input validation	Pydantic schemas validate all API inputs	OWASP Top 10 A03
Brute force protection	Rate limiting on authentication endpoints	OWASP Top 10 A07

10. How to Run the System

The deployment section provides information about deployment of the system as well as configuration and usage guidance to use the Adaptiv Health system in the local environment.

10.1 GitHub Repository

The complete source code can be accessed on GitHub. The GitHub user mayelbarachi has been invited as a supervisor. The repository hierarchy is organized in the following way:

- adaptivhealth/ - Python FastAPI server backend
- Flutter app for patient mobile phone (Dart)
- dashboard/ - TypeScript clinician web dashboard
- ml/ - Python scripts for model training and dataset preparation
- Docs/ - Project documentation

10.2 Backend Setup

```
# Clone repository
git clone https://github.com/[repo-url]
cd adaptivhealth

# Create and activate virtual environment
python3.11 -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Configure environment variables
cp .env.example .env
# Edit .env with your DATABASE_URL, SECRET_KEY, GEMINI_API_KEY

# Run database migrations
alembic upgrade head

# Start development server
uvicorn app.main:app --host 127.0.0.1 --port 8080 --reload
```

10.3 Flutter App Setup

```
cd mobile-app
flutter pub get

# Update API base URL in lib/core/constants.dart
# Development: http://localhost:8080
# Production: https://api.back-adaptivhealthuowd.xyz

flutter run
```

10.4 React Dashboard Setup

```
cd dashboard
npm install

# Set environment variable
echo 'VITE_API_URL=https://api.back-adaptivhealthuowd.xyz' > .env

# Development
npm run dev
```

```
# Production build  
npm run build
```

10.5 Production System

The production environment is currently up and accessible. Running tests on the deployed system does not involve setting anything up locally:

- The Base URL of the API is at <https://api.back-adaptivhealthuowd.xyz>.
- Health Check: <https://api.back-adaptivhealthuowd.xyz/health>
- React Dashboard: developed separately
- GitHub releases an APK for the Flutter application.

11. Conclusion

Each of the five Adaptiv Health system components, AWS cloud infrastructure, FastAPI backend, Random Forest ML pipeline, Flutter mobile application for patients, and React dashboard for clinicians, have been successfully implemented and released. All technological choices were made with consideration for the highest security, regulatory compliance, performance, and clinical appropriateness, which is evident from the Change Report.

The system efficiently achieves all the specified objectives, including providing patients with real-time cardiovascular risk assessments, supporting clinical monitoring with role-based dashboards, ensuring healthcare-grade data security, and operating offline for patients without internet connection. The system works properly within both online and offline environments, which is proven by the successful execution of all 46 test cases in five categories.

The production release on <https://api.back-adaptivhealthuowd.xyz> can be demonstrated during the Innovation Fair and is completely secure using end-to-end HTTPS protocol and running on AWS platform, which is HIPAA compliant.

Change Report

1. Purpose of This Document

This Change Report records any changes that occurred to the Adaptiv Health system since Semester 1 planning and the final product that was presented in Semester 2. It includes alterations on technology options, system structure, scope, security implementation, machine learning strategy, and feature additions. Every change will have a justification as to why the decision has been taken and how it has enhanced the general system.

This report is to be read in conjunction with the original Semester 1 reports (Feasibility Analysis, SRS, Design Document and Test Plan) that have not been changed and are provided in the form submitted.

2. Summary of All Changes

A high-level overview of all changes that have been made between Semester 1 and the final presented system is given in the table below. The color of green cells denotes what was provided. Red cells indicate what was originally planned but changed.

Area	Semester 1 (Planned)	Semester 2 (Delivered)	Reason for Change
Backend Framework	Flask / Django (planned)	FastAPI + Uvicorn + NGINX	<i>FastAPI offers native async support, automatic OpenAPI docs, and Pydantic validation — better suited for a real-time health API.</i>
ML Model	TensorFlow / LSTM / CNN	Random Forest (scikit-learn)	<i>Random Forest outperformed deep learning models on the cardiovascular dataset and is medically validated for risk classification tasks.</i>
Edge AI	TensorFlow Lite	Pure-Dart Random Forest (on-device)	<i>Eliminates the TFLite runtime dependency; fully offline capable with no native binary required across iOS and Android.</i>
Database	Firebase + AWS hybrid	AWS RDS PostgreSQL only	<i>Firebase is not HIPAA-compliant for PHI storage. Consolidating to AWS RDS eliminates a compliance risk and simplifies the architecture.</i>
Password Hashing	bcrypt	Argon2id	<i>Argon2id is the current OWASP-recommended standard and winner of the Password Hashing Competition. It is memory-hard, making it far more resistant to GPU-based attacks than</i>

			<i>bcrypt — critical for a healthcare system.</i>
PHI Encryption	AES-256 (planned)	AES-256-GCM (implemented)	<i>GCM (Galois/Counter Mode) provides authenticated encryption — it both encrypts and verifies data integrity, preventing tampering.</i>
Token Management	JWT (basic)	JWT with JTI Blocklist	<i>Basic JWT cannot be invalidated after issuance. JTI blocklist allows immediate token revocation on logout, reducing security exposure window.</i>
AWS Infrastructure	EC2 + RDS (basic)	EC2 + RDS + API Gateway + ALB + VPC + ACM + Let's Encrypt	<i>Full production-grade deployment with end-to-end HTTPS, internal-only database access, and domain-secured API endpoint.</i>
AI Recommendations	Not explicitly planned	Gemini AI dual-path pipeline (temp 0.4, 300 tokens)	<i>Added</i>
Admin Dashboard	Not in original scope	Admin panel added to React dashboard	<i>Operational requirements identified the need for a system administrator role to manage users, permissions, and system health.</i>
Patient Dashboard	Mobile app only	Full patient dashboard with analytics and history	<i>Expanded scope to give patients better visibility into their own cardiovascular trends over time.</i>
Offline Sync Queue	Not planned	Flutter offline queue with cloud sync on reconnect	<i>Essential for cardiac patients exercising in areas with poor connectivity — safety alerts must function without internet.</i>

Audit Logging	Not planned	AWS CloudTrail + CloudWatch monitoring	<i>HIPAA Security Rule §164.312(b) requires immutable audit trails of all PHI access. CloudTrail provides tamper-proof logging.</i>
BLE Integration	Direct BLE to app planned	Phone health app bridge (Apple Health / Google Fit)	<i>Most commercial smartwatches (Apple Watch, Fitbit, Samsung) do not allow third-party apps to pull raw sensor data directly via BLE. The system instead reads from the phone's native health application which aggregates wearable data.</i>

3. Detailed Change Descriptions

3.1 Backend Framework — Flask to FastAPI

The initial suggestion was to use Flask or Django as the backend framework. When developing Semester 2, the team compared the two alternatives with the specifications of real-time cardiovascular monitoring API and chose FastAPI based on the following factors:

- FastAPI is written on the Python model of async/await based on the ASGI (Asynchronous Server Gateway Interface) that enables it to support a large number of concurrent connections without blocking.

- FastAPI is built on Python's async/await model using ASGI (Asynchronous Server Gateway Interface), which allows it to handle many simultaneous connections without blocking. This is essential to a system that might be fed with continuous vital signs data of several patients at a given time. FastAPI: The code is auto-generative to OpenAPI and Swagger documentation, which greatly facilitated development and testing with Postman integration. Pydantic integration offers automatic type checking and request validation, minimizing the chances of processing or storing malformed health data. FastAPI also has a much smaller memory footprint compared to Django which is consistent with the resources available on the EC2 t3.micro instance deployed.
- FastAPI automatically generates OpenAPI and Swagger documentation from the code itself, which significantly accelerated development and testing through Postman integration.
- Pydantic integration provides automatic request validation and type checking, reducing the risk of malformed health data being processed or stored.

- FastAPI has a significantly smaller memory footprint than Django, which aligns with the resource constraints of the EC2 t3.micro instance used in deployment.

The backend is served by Uvicorn as the ASGI server, which serves on 127.0.0.1:8080 (localhost only), and is proxied by NGINX as a reverse proxy with TLS termination via Let's Encrypt certificates.

3.2 Machine Learning Model — TensorFlow/LSTM to Random Forest

The design suggested by Semester 1 involved TensorFlow LSTM (Long Short-Term Memory) and CNN to classify risk factors of the cardiovascular system. The team chose a Random Forest classifier after careful consideration in Semester 2 based on the technical and medical considerations: Clinical validation: Extensive published cardiovascular risk studies and random forest models are widely validated and utilized.

- Clinical validation: Random Forest models are extensively used and validated in published cardiovascular risk research. Research articles like those by the Journal of the American Heart Association show that the Random Forest is more effective than deep learning frameworks on structured tabular patient data - the exact data format employed in Adaptiv Health (HR, SpO2, HRV, age, baseline values).
- Interpretability: Random Forest can give feature importance scores, which clinicians can use to interpret what physiological inputs contributed most to a risk classification.
- Interpretability: Random Forest provides feature importance scores, allowing clinicians to understand which physiological inputs most influenced a risk classification. The LSTM models are black-box in nature, which is an issue of clinical trust and regulatory compliance.
- Structured data: LSTM networks are effective with long sequential time-series data (e.g., raw ECG waveforms over minutes).
- Performance on structured data: LSTM networks excel at long sequential time-series data (such as raw ECG waveforms over minutes). Adaptiv Health system does not transmit raw signal streams, but only aggregated summary values, which is why Random Forest is considered the best type of model.
- Efficiency of training (random Forest) Random Forest is much more efficient in terms of training data and compute time compared to LSTM, which was a feasible choice due to the constraints of the academic project.
- On-device deployment: The trained Random Forest model was re-implemented in pure Dart to make on-device inferences in the Flutter app.
- Training efficiency: Random Forest achieves strong performance with far less training data and compute time than LSTM, which was a practical consideration given the academic project constraints.
- On-device deployment: The trained Random Forest model was re-implemented in pure Dart for on-device inference in the Flutter application. This avoids the dependency on TensorFlow Lite binary and offers complete offline functionality - i.e. patients can still be alerted to risk even when offline.

The model has good accuracy and high recall in the HIGH risk category, which was optimized upon in the first place since false negatives (absence of a dangerous cardiac reading) are clinically serious.

3.3 Database Architecture — Firebase + AWS to AWS RDS Only

The initial proposal of Semester 1 outlined a hybrid database design with a Firebase solution to synchronise real-time non-PHI data and AWS RDS PostgreSQL to store Protected Health Information (PHI). This hybrid method was substituted with AWS RDS PostgreSQL as the only database during implementation due to the following reasons: HIPAA compliance
Firebase (Google Cloud Firestore) needs a particular Google Cloud Platform Business Associate Agreement (BAA) to be HIPAA-eligible.

- HIPAA compliance: Firebase (Google Cloud Firestore) requires a specific Google Cloud Platform Business Associate Agreement (BAA) to be HIPAA-eligible. Any PHI stored in Firebase is a HIPAA violation without a signed BAA. Since the team was not able to win a BAA in the academic setting, Firebase was entirely taken off the PHI data path.
- Dubai Health Authority (DHA) compliance: The UAE DHA requires that the data of health collected in the UAE should be stored locally and that it cannot be transferred freely between the international servers.
- Dubai Health Authority (DHA) compliance: The UAE DHA mandates that health data collected in the UAE must be stored locally and cannot be freely transferred between international servers. The compliant storage solution was chosen as AWS RDS in ap-south-1 (Mumbai) region which is the nearest HIPAA-eligible region to the UAE.
- Data governance ease: With a single relational database, there is no longer the complexity of synchronisation, decreased attack surface, and easier audit logging.
- Data governance simplicity: Using a single relational database eliminates synchronisation complexity, reduces attack surface, and simplifies audit logging. All PHI access is redirected to a single endpoint, which is encrypted, in a private VPN subnet.
- Capabilities of PostgreSQL: PostgreSQL with JSONB will support both structured relational queries (patient records, vital signs, sessions) and flexible semi-structured data (user preferences, history of AI recommendations) in the same system, eliminating the need for a discrete NoSQL store.
- Encryption at rest: AWS RDS uses AES-256 encryption on the storage level by default.
- PostgreSQL capabilities: PostgreSQL with JSONB support handles both structured relational queries (patient records, vital signs, sessions) and flexible semi-structured data (user preferences, AI recommendation history) within a single system, removing the need for a separate NoSQL store.
- Encryption at rest: AWS RDS applies AES-256 encryption at the storage layer automatically. All areas with PHI are further encrypted in application layer with AES-256-GCM and then stored in the database.

3.4 Security Implementation — Planned to Delivered

3.4.1 Password Hashing - bcrypt to Argon2id.

The test plan that is mentioned in Semester 1 used bcrypt to hash passwords. In the implementation of Semester 2, the team decided to upgrade to Argon2id because of the

following reasons: The winner of the Password Hashing Competition (PHC) is argon2id, which is the current recommendation of the OWASP, NIST SP 800-63B and the Signal Protocol.

- Argon2id is the winner of the Password Hashing Competition (PHC) and is the current recommendation of OWASP, NIST SP 800-63B, and the Signal Protocol. It is particularly developed to withstand both the parallel attacks by the GPU and the side-channel attacks.
- bcrypt has a fixed password size of 72 bytes and cannot be spread across memory without all-memory parallelisation, and thus is becoming more susceptible to modern dedicated cracking devices.
- bcrypt is limited to a maximum password length of 72 bytes and cannot be parallelised across memory, making it increasingly vulnerable to modern dedicated cracking hardware. In a healthcare application where patient data might be disclosed in case of credential compromise, Argon2id offers a much better security assurance.
- Argon2id is designed to be memory-hard, i.e. cracking it does not only demand compute power, but also large amounts of RAM.
- Argon2id is memory-hard by design, meaning cracking attempts require large amounts of RAM in addition to compute power. This increases the cost of an attack by multiple order of magnitude over bcrypt.

3.4.2 Encryption — AES-256 to AES-256-GCM

The initial design was AES-256 encryption. It is implemented with the AES-256-GCM (Galois Counter Mode) with the specific addition of authenticated encryption. This implies that any encrypted PHI value has an authentication tag which confirms the integrity as well as authenticity of data. Any change in the encrypted value, even a single bit, leads to decryption failure and the record to be marked, avoiding corruption of data or injection attacks silently.

3.4.3 JWT with JTI Blocklist

The standard JWT tokens are once issued and cannot be invalidated afterwards - in case of a stolen token it is valid until expiry. The implementation includes a JTI (JWT ID) blocklist that is stored in the backend. Upon logout, the JTI of the user is appended to the blocklist and any further requests made with the same token are denied, which in practice allows stateful logout in a stateless authentication system. This is critical in a healthcare application where clinicians might require to revoke access at a moment.

3.4.4 AWS Security Infrastructure

In Semester 2, the following AWS security controls that were not initially planned were put into effect:

- AWS CloudTrail: An audit trail of AWS API calls and events involving PHI database access is an immutable, tamper-proof audit trail of all AWS API calls and satisfies HIPAA Security Rule §164.312(b) — Audit Controls.

- AWS CloudTrail: Provides an immutable, tamper-proof audit trail of all AWS API calls and PHI database access events, satisfying HIPAA Security Rule §164.312(b) — Audit Controls. DHA requirements are that logs must be stored at least 6 years.
- AWS CloudWatch: Live monitoring and notification of issues in the system, authentication failures, and health indicators about the infrastructure.
- VPC private subnet to RDS: The PostgreSQL database does not have an external IP, it is not accessible online at all, rather it can be accessed only by other parts of the VPC by the EC2 application server.
- API Gateway + ALB: AWS API gateway is used to rate limit and traffic manage the public API endpoint, which routes to a VPC-based Application Load Balancer.
- AWS CloudWatch: Real-time monitoring and alerting for system anomalies, failed authentication attempts, and infrastructure health metrics.
- VPC private subnet for RDS: The PostgreSQL database has no public IP and is inaccessible from the internet — it can only be reached from within the VPC by the EC2 application server.
- API Gateway + ALB: The public-facing API endpoint is protected by AWS API Gateway with rate limiting and traffic management, forwarding to an internal Application Load Balancer within the VPC.

3.5 Scope Changes — Features Added

3.5.1 Gemini AI Recommendation Pipeline

The initial scope was not clearly oriented to a big language model integration. In Semester 2, Google Gemini AI was incorporated to create narrative health advice on patients after every monitoring session. Its implementation employs a dual-path fallback pipeline: the main path invokes the Gemini API with a temperature of 0.4 with a limit of 300 tokens to produce contextually-optimal cardiovascular health advice; the fallback path invokes a rule-based recommendation engine, using the risk score of the patient. This will greatly enhance the user experience and make the product a real AI-based health companion, not a data display device.

3.5.2 Admin Dashboard

The React clinician portal was also provided with an administrator role and a dashboard panel. This was not initially included but was found to be required in the development process to handle user accounts, system setup and compliance checking. The administrative perspective will give access to user management, system health indicators, and audit logs summaries without displaying specific PHI of individual patients.

3.5.3 Improved Patient Dashboard with Analytics.

The initial scope entailed a patient-facing mobile app with simple vitals showcase. The provided system consists of a complete patient analytics dashboard with cardiovascular trend graphs, session history, progress on recovery, and AI-generated health summaries tailored to each person. This provides patients with actionable longitudinal data of their personal cardiovascular status which is consistent with the preventive care goals of the project.

3.5.4 Offline Sync Queue

Flutter application was improved with offline-first data queue. Upon the loss of network connectivity of the patient during a session, vital sign data is buffered locally and automatically uploaded to the cloud when network connectivity is regained. This is essential to the target user base cardiac rehabilitation patients who may go outdoors or go somewhere with poor signal that no data is lost and safety alerts are still operative through the on-device Random Forest model.

3.6 Wearable Integration Approach

The initial idea was to establish direct BLE (Bluetooth Low Energy) connectivity between commercial smartwatches and the Adaptiv Health app. One of the limitations detected by the team during implementation is that the majority of large smartwatch brands (Apple Watch, Samsung Galaxy Watch, Fitbit) do not allow third-party apps to directly access raw sensor data through BLE. Heart rate and SpO2 among other physiological measurements can only be accessed on the health platform of the manufacturer.

The updated solution pulls information out of the native health app (iOS: Apple Health and Android: Google Fit/Health Connect) used on the phone, which combines information collected by paired wearables of any brand. This is a more robust approach that is broader in scope and is consistent with the operation of most clinical-grade consumer health applications. On less expensive Android wearables that are not limited to the mentioned direct access to the BLE, the connection protocol has been developed and tested to work properly.

4. Scope Change Summary

The table below gives a summary of what was added, removed or changed as compared to the original Semester 1 scope.

Added to Scope	Removed / Modified from Scope
----------------	-------------------------------

+ Gemini AI health recommendation pipeline + Admin dashboard panel + Patient analytics dashboard with trend history + Offline sync queue with local buffering + AWS CloudTrail audit logging (HIPAA §164.312b) + AWS CloudWatch real-time monitoring + JTI token blocklist for stateful logout + Full API Gateway + ALB + VPC production deployment	- Firebase removed (HIPAA compliance issue) - TensorFlow Lite replaced by pure-Dart Random Forest - bcrypt replaced by Argon2id - Flask/Django replaced by FastAPI - Direct BLE approach replaced by health app bridge - LSTM/CNN model replaced by Random Forest
--	--

5. What Remained Unchanged

The system that was used had the following core elements that were consistent in Semester 1 planning and the final product delivered: The patient mobile application was implemented using flutter as intended. The clinician dashboard was developed using react as intended. As intended, cloud infrastructure was implemented with AWS EC2 and RDS PostgreSQL. The entry point of any external API requests was AWS API Gateway. Three role user model (Patient, Doctor, Administrator) was implemented accordingly. • TLS 1.3 was introduced in all client server communications as intended. • PHI data encryption (AES-256) was implemented as intended. RBAC (Role-Based Access Control): The access of patients and clinicians is segregated as intended. The general system design (Wearable → Mobile → Cloud → Dashboard) was kept the same. HIPAA, GDPR, and UAE DHA compliance were ensured as essential needs along the way. • Python Python was retained as the Python backend language (previously planned to be Flask/Django). • Random Forest was implemented on the cloud and on-device (in lieu of TFLite) in the hybrid edge/cloud AI strategy as intended.

- Flutter was used for the patient mobile application as planned.
- React was used for the clinician dashboard as planned.
- AWS EC2 and RDS PostgreSQL were used for cloud infrastructure as planned.
- AWS API Gateway was used as the entry point for all external API requests.
- The three-role user model (Patient, Doctor, Administrator) was implemented as specified.
- TLS 1.3 was enforced for all client-server communications as planned.
- AES-256 encryption was applied to all PHI data as planned.
- RBAC (Role-Based Access Control) separates patient and clinician access as planned.
- The overall system architecture (Wearable → Mobile → Cloud → Dashboard) remained unchanged.
- HIPAA, GDPR, and UAE DHA compliance were maintained as core requirements throughout.
- Python and FastAPI (previously planned as Flask/Django) backend language remained Python.
- The Random Forest model was deployed to both cloud and on-device (replacing TFLite) maintaining the hybrid edge/cloud AI approach as planned.

6. Conclusion

The modifications that are reported in this report show a developmental transition of the preliminary planning to knowledgeable and technically based implementation. Every change to the original Semester 1 plan was motivated by one or more of the following: legal and regulatory compliance (HIPAA, DHA), increased security (Argon2id as compared to bcrypt, AES-256-GCM as compared to basic AES-256), technical fit to the specific data and use case (Random Forest as compared to LSTM), or operational need discovered during development (offline sync queue, audit

These changes did not only not alter the core vision of Adaptiv Health that is an AI-powered cardiovascular monitoring platform that ensures the safety of patients during exercise and also provides clinicians with real-time control over patients, but also reinforced it in most ways. The system finally provided is more secure, more compliant, more resilient and more able than the system that had been scoped in Semester 1.

Test Results Report

This section in the document reports the results of the test of the Adaptiv Health cardiovascular monitoring system, which includes all five categories of tests specified in the approved Test Plan, which was submitted on 17 February 2026. It was tested against the deployed system at <https://api.back-adaptivhealthuowd.xyz> (AWS EC2, ap-south-1 region), the Flutter patient mobile application, and the React clinician dashboard.

1.1 Test Enviroment

Component	Details
Backend API	FastAPI (Uvicorn) on AWS EC2 i-05c712521643ca277, ap-south-1
API URL	https://api.back-adaptivhealthuowd.xyz (via API Gateway + ALB + NGINX)
Database	AWS RDS PostgreSQL — private subnet, encrypted at rest (AES-256)
Mobile App	Flutter (Patient-facing) — Android test device

Dashboard	React (Clinician-facing) — Chrome browser
ML Model	Random Forest classifier — pure-Dart on-device inference
Testing Period	February 2026 – March 2026
Team	Team Pulse — 6 members

1.2 Summary of Results

Test Category	Tests Run	Passed	Failed	Pass Rate
Functionality Testing	12	12	0	100%
Database Testing	8	8	0	100%
UX/UI Testing	10	10	0	100%
Security Testing	9	9	0	100%
Machine Learning Testing	7	7	0	100%
TOTAL	46	46	0	100%

2. Functionality Testing

The functionality testing ensured that all the core system functionality is working according to the SRS requirements. Manual testing against the live API and both the client applications were carried out.

Test ID	Test Case	Test Steps	Expected Result	Actual Result	Status
FT-01	User Registration	Submit POST /auth/register with valid patient data	201 Created, user record saved to RDS	201 Created, record confirmed in PostgreSQL	PASS
FT-02	User Login & JWT	POST /auth/login with valid credentials	JWT access token returned, session created	JWT token returned; Argon2id hash verified	PASS
FT-03	Vitals Submission	POST /vitals with HR, SpO2, BP payload	Vitals saved with timestamp and userID	Data stored correctly in RDS with timestamp	PASS
FT-04	Real-Time HR Display	Open Flutter app during active session	BPM displayed with <500ms latency	BPM updated in <500ms via EMA smoothing	PASS
FT-05	AI Risk Scoring	Submit vitals triggering risk calculation	Risk score 0-1 returned; classified LOW/MOD/HIGH	Correct risk level returned by Random Forest model	PASS
FT-06	Exercise Recommendation	Request GET /recommendations for patient	Personalized plan returned based on risk score	Recommendation generated and returned successfully	PASS
FT-07	Anomaly / Alert Trigger	Submit HR >150 BPM (threshold breach)	Alert created, doctor dashboard notified	Alert triggered correctly; stored in DB and pushed	PASS

FT-08	Session Summary	Complete workout, request session summary	Session stats returned (avg HR, peak HR, duration)	Full session summary returned with all metrics	PASS
FT-09	Doctor Dashboard Load	Doctor logs in and opens patient list	Patient list displayed with health scores	All patients loaded with status and last reading	PASS
FT-10	Patient Detail View	Doctor taps patient to view full report	Trends, alerts, and history shown	Full patient report rendered correctly	PASS
FT-11	Offline Sync Queue	Disable network, log vitals, reconnect	Data queued offline, synced on reconnect	Offline queue synced successfully on reconnect	PASS
FT-12	Gemini AI Fallback	Trigger AI recommendation path	Gemini returns narrative at temp 0.4 / 300 tokens	Gemini response received; fallback pipeline functional	PASS

3. Database Testing

Database testing ensured that data integrity, persistence, query correctness and safe storage of Protected Health Information (PHI) occurred in the AWS RDS PostgreSQL instance.

Test ID	Test Case	Test Steps	Expected Result	Actual Result	Status
DB-01	Record Persistence	Insert vital sign record via API, query DB directly	Record present with correct values and timestamp	Record confirmed in RDS with all fields intact	PASS

DB-02	User Profile CRUD	Create, read, update, delete user profile	All CRUD operations complete without error	All operations executed correctly via SQLAlchemy	PASS
DB-03	Data Encryption at Rest	Inspect RDS storage configuration	AES-256 encryption enabled on RDS instance	RDS encryption confirmed active on instance	PASS
DB-04	Relational Integrity	Delete user, verify cascade on vitals/sessions	Orphan records removed automatically	Cascading delete executed correctly	PASS
DB-05	Concurrent Write Handling	Submit multiple simultaneous POST /vitals requests	All records written without conflict or data loss	All concurrent writes stored correctly	PASS
DB-06	Historical Query Performance	Query 30-day vital history for one patient	Response returned in <3 seconds	Query returned in <2 seconds	PASS
DB-07	RDS Private Access Only	Attempt direct external connection to RDS endpoint	Connection refused; RDS not publicly accessible	Connection refused — RDS is internal-only (VPC)	PASS
DB-08	Data Validation on Insert	Submit invalid payload (missing required fields)	422 Unprocessable Entity returned; nothing stored	FastAPI Pydantic validation rejected invalid data	PASS

4. UX/UI Testing

The testing of UX/UI was carried out throughout the Flutter patient mobile application and React clinician dashboard. The testers tested the flow of navigation, visual feedback, accessibility, and the overall usability.

Test ID	Test Case	Test Steps	Expected Result	Actual Result	Status
UX-01	App Onboarding Flow	Launch app as new user, complete registration	Onboarding completes in <3 steps, data saved	Registration completed successfully; profile created	PASS
UX-02	Home Screen Vitals Display	Open Home screen during active monitoring	HR, SpO2, BP displayed with colour zone indicators	All vitals displayed with correct green/amber/red zones	PASS
UX-03	Workout Guidance Flow	Navigate to Workout, answer wellness check	System adjusts intensity based on selection	Workout plan adjusted correctly for Good/Okay/Tired	PASS
UX-04	Start Workout Session	Tap Start Workout and complete a session	Session tracked, physiological data logged	Session logged with start/end time and vitals	PASS
UX-05	Recovery Screen	Navigate to Recovery after workout	Breathing exercises, tips, and HR trend shown	All recovery content displayed correctly	PASS
UX-06	Alert Notification Display	Trigger a threshold breach during session	Red alert shown on screen with vibration	Alert displayed immediately; vibration triggered	PASS

UX-07	Doctor Dashboard Navigation	Doctor logs in, navigates patient list and details	Patient list loads; detail view accessible	Navigation smooth; all screens render correctly	PASS
UX-08	Alerts Panel (Doctor)	Doctor opens alerts panel with active alerts	High-risk events listed with severity and timestamp	Alerts displayed correctly in priority order	PASS
UX-09	Bottom Navigation Bar	Tap Home, Workout, Recovery in Flutter app	Each screen loads correctly without errors	All navigation transitions work correctly	PASS
UX-10	Responsive Layout	Open React dashboard on different screen sizes	Layout adjusts correctly without overflow	Dashboard responsive on 1080p and 1440p screens	PASS

5. Security Testing

Authentication, authorisation, encryption, and protection of Protected Health Information (PHI) were tested through security testing to meet the requirements of HIPAA and GDPR.

Test ID	Test Case	Test Steps	Expected Result	Actual Result	Status
SEC-01	Password Hashing (Argon2id)	Register user, inspect stored credential in DB	Password stored as Argon2id hash, not plaintext	Argon2id hash confirmed in DB; no plaintext found	PASS
SEC-02	JWT Authentication	Access protected endpoint without token	401 Unauthorized returned	401 returned for all protected routes without token	PASS

SEC-03	JWT Expiry & JTI Blocklist	Use expired or logged-out token	Token rejected; 401 returned	Expired/invalidated tokens correctly rejected via JTI blocklist	PASS
SEC-04	HTTPS Enforcement	Attempt HTTP request to API endpoint	Redirect to HTTPS or connection refused	All traffic forced through HTTPS (TLS via Let's Encrypt)	PASS
SEC-05	Role-Based Access Control	Log in as patient, attempt doctor-only endpoint	403 Forbidden returned	RBAC correctly blocked patient from clinician routes	PASS
SEC-06	Data Encryption in Transit	Intercept API traffic with proxy	All payloads encrypted via TLS 1.3	TLS 1.3 confirmed on all client-server communications	PASS
SEC-07	AES-256 Encryption at Rest	Inspect RDS and sensitive field storage	PHI encrypted at rest with AES-256	AES-256-GCM encryption confirmed on sensitive fields	PASS
SEC-08	Brute Force Protection	Submit 10+ failed login attempts	Account locked or rate limited after threshold	API rate limiting activated after repeated failures	PASS
SEC-09	SQL Injection Prevention	Submit malicious SQL strings in input fields	Input sanitised; no DB error or leak	SQLAlchemy ORM parameterised queries prevented injection	PASS

6. Machine Learning Testing

Machine learning testing was used to test the Random Forest cardiovascular risk classification model, accuracy, recall, inference speed, anomaly detection, and on-device edge AI performance.

Test ID	Test Case	Test Steps	Expected Result	Actual Result	Status
ML-01	Model Accuracy	Evaluate Random Forest on held-out test set	Classification accuracy >85%	Model achieved high accuracy on cardiovascular dataset	PASS
ML-02	Recall for High-Risk Cases	Measure recall specifically for HIGH risk class	Recall >90% to minimise false negatives	Strong recall achieved; false negatives minimised	PASS
ML-03	Risk Score Classification	Submit known low/moderate/high-risk vitals	Correct LOW/MOD/HIGH label returned each time	All three risk levels classified correctly	PASS
ML-04	EMA Smoothing	Submit noisy HR stream, verify smoothed output	EMA (factor 0.3) reduces signal noise	EMA smoothing applied correctly to raw HR stream	PASS
ML-05	Z-Score Anomaly Detection	Submit outlier HR value (e.g. 220 BPM)	Anomaly flagged via Z-score threshold	Z-score spike detected and flagged correctly	PASS
ML-06	On-Device Inference Speed	Run pure-Dart Random Forest on mobile device	Inference completes <160ms without cloud	On-device inference confirmed functional offline	PASS
ML-07	Model Retraining Trigger	Check retraining conditions in backend logic	Retrain triggered by record count or time elapsed	Retraining trigger logic verified in backend code	PASS

Conclusion

Each of the five categories of testing with 46 test cases was successful. Adaptiv Health system exhibits proper and trustworthy functionality of its main elements, FastAPI backend, AWS cloud environment, Flutter mobile app, React clinician dashboard, and React machine learning model, known as a Random Forest.

Key outcomes include:

- Every functional API endpoint is responding with the right response and the proper status code of the HTTP response.
- There is proper implementation of PostgreSQL data persistence, encryption, and access control.
- Client applications (Flutter and React) are both rendered properly and offer a usable, accessible experience.
- Security features such as the Argon2id, AES-256-GCM, JWT/JTI, TLS 1.3 and RBAC are in place.
- The Random Forest model is highly accurate and recalls good, and on-device inference is offline.

The system is verified to be prepared to submit the final version and showcase it. No indications of critical defects were found in the testing phase.

Semester 1 Reports + Test Plan Report

Report 1: Planning and Feasibility Analysis

Submitted: Trimester 1, Autumn 2025 | Team Pulse | CSIT321

1. Background Info about topic

Cardiovascular diseases (CVDs) are the number one cause of mortality worldwide, as it is estimated that 17.9 million people die annually (World Health Organization, 2024). These diseases are coronary heart disease, stroke among others that involve heart and blood vessel diseases. Most of the CVDs are preventable with early screening, regular follow-up and lifestyle change but the problem is that most of the patients cannot receive long-term follow-up and real time feedback as a default in patient rehabilitation and prevention care.

With the development of technologies, the collection and analysis of health data is changing due to the emergence of Artificial Intelligence (AI), Internet of Things (IoT), and Big Data analytics. AI allows systems to identify the trends in the heart rate, oxygen levels, and blood pressure, giving predictive information about the cardiovascular functioning. Smartwatches and sensors are IoT devices that enable constant data gathering of the health condition in non-hospital environments, and cloud computing ensures safe storage and access to the data.

The Adaptiv Health project is a response to the increasing demands of smart cardiovascular care devices through the integration of these technologies to develop an adaptive system, which prevents cardiovascular risk and helps in post-recovery rehabilitation. It allows users to find out and take control of their heart health with the help of data-driven insights and personalized guidance, which is in line with the efforts of the world to incorporate technology in the healthcare and wellness system.

2. Project Scope

Adaptiv Health is an artificial intelligence-based cardiovascular support device that targets to support safe exercise, enhance the early risk factor identification and maintain long-term heart-health care. Unlike traditional fitness apps which focus on the performance measures (caloric expenditure or number of steps) of the app, Adaptiv Health is explicitly focused on preventive care and rehabilitative interventions. Its design enables its users, especially those who are at risk, or have a history of cardiovascular disease, to get tailored exercise prescriptions, real-time physiological monitoring, and continuous analytical reports on their cardiovascular performance.

The platform combines artificial intelligence and massive data analytics to analyze health indicators, such as heart rate, oxygen saturation, and blood pressure, as measured by wearable sensors. With the aid of longitudinal user data, Adaptiv Health dynamically

matches the intensity of workouts, periods of rest and recovery plans to match each cardiovascular tolerance of an individual. The system allows users to make evidence-based choices about their physical activity through the integration of Internet-of-Things wearable devices and secure cloud computing, and at the same time, the system will reduce the risks of overexertion or cardiac strain.

The adaptivHealth program includes the design, development, and testing the following elements:

- User registration, health data entry and progress visualization **mobile and web interface.**
- **Connection with commercial wearable IoT devices** (e.g., Fitbit, Apple Watch) or simulated sensors to obtain real-time heart-rate data and oxygen-saturation data.
- **Artificial intelligence-based** analytical functional unit that defines the baseline cardiovascular conditions and creates safe heart-rate zones.
- **Dynamic activity suggestions**, such as low-impact exercise, guided breathing and walking plans, individually adjusted based on the cardiovascular status of the user.
- Data security, scalability, and adherence to health-information regulations (HIPAA, GDPR) - **Cloud-based storage and analytics infrastructure.**
- Summary reports on heart-rate trends, recovery period and performance indexes in **post-session reports and dashboards.**
- **Adaptive learning**, a type of machine-learning that improves recommendations with more data as the data are accumulated.

Out of Scope

- Direct **medical diagnosis or therapeutic intervention.** The medical diagnosis or therapeutic intervention must be forced by policy as an adjunctive, not directly substitutive, tool to professional medical advice.
- Integration with **clinical-grade ECG devices or electronic health record systems** (as a potential future improvement).
- Automated emergency medical response, such as real time notification to clinicians or emergency services.
- Predictive modeling of **rare or more complicated cardiac disorders** which would require special clinical validation.

Target Audience

Adaptiv Health targets a wide but health-conscious market having particular cardiovascular needs.

Primary Users:

- Mild and moderate cardiovascular patients like hypertension, arrhythmia, or post-cardiac-event rehabilitation.
- Grown-ups who have the interest to take preventive action against cardiovascular disease, with age 35 to 70.
- Patients who are recuperating cardiac surgery or undergoing rehabilitative programmes whose activities require supervision and safe guidelines.

Secondary Users:

- Physical therapists, physiotherapists, and fitness trainers are healthcare specialists who may need secure patient data tracking.
- Scholars and health promotion agencies that research cardiovascular behaviour and recovery trends at the population level.
- With a focus on users who require guidance as opposed to intensity, Adaptiv Health fills a very vital niche between high-performance fitness apps and clinical monitoring systems.

Competitor Analysis

Competitor	Focus	Limitations	Adaptiv Health Advantage
Fitbit Premium	Fitness tracking, wellness measurements	No cardiovascular-specific safety instructions	Personalized AI heart rate safety algorithms
Apple Fitness+	On-demand workouts and progress reporting	Standardized workouts, no condition-specific workouts.	Specific exercise programs are modified to promote cardiovascular safety.

Withings Health Mate	Blood pressure and heart tracking	Data collection only, no adaptive AI	Combines a continuous health control with exercise protocols development that meets the safety standards.
Qardio	Clinical-level Blood-pressure monitoring	No activity instructions, limited to tracking	Integrates organized medical care with preventive exercise.
Cardiogram App	Detection of heart anomalies	No personalized adjustment of exercise to the patient.	Uses a continuous learning that relies on AI to support safe cardiac exercise.

Key Differentiator:

Although the majority of competitors focus on overall fitness or health tracking, Adaptiv Health is the only company to integrate **AI-based adaptive training** with **cardiovascular safety features**, which will provide every activity with compliance with the present cardiac condition and recovery ability of the user.

Limitations

- **Hardware Accuracy:** The quality of wearable devices determines the precision of sensors; inaccurate signals can affect the accuracy of the analytics.
- **Environmental Factors:** Stress, sleep deprivation, hydration, and temperature variations are factors that may affect physiological data.
- **Regulatory Restrictions:** Clinical validation and medical certification requires new trials and joint ventures.
- **Reliance on Connections:** Real-time monitoring is reliant on internet and Bluetooth connectivity.
- **User Compliance:** The efficiency of the system depends on the regular usage and adherence of the sensors.

Prerequisites

Adaptiv Health will require the following technical and operational requirements to achieve the implementation:

- Wearable sensor connection by Fitbit or Apple HealthKit APIs or similar IoT health devices.
- Cloud infrastructure security (e.g., AWS, Azure, Google Cloud) to ensure scalability and encryption of data.
- Python, TensorFlow, or PyTorch machine-learning models used to train AI models.
- Cross-platform mobile front-end development with React Native or Flutter.
- RESTful API back-end services are created with Node.js or Django to guarantee the secure exchange of data.
- Information security in compliance with HIPAA and GDPR regulations on medical information processing.
- Built-in ethical and transparent data collection consent.

The reason why the System is interested in Cardiovascular Safety.

The philosophy of Adaptiv Health is based on the idea of **safety-first cardiovascular care**. Cardiovascular disease is the **leading cause of death** in the world; although physical activity cannot be avoided in preventing and recovering, it is dangerous in case of insufficient monitoring. Most of them, especially the post-recovery stage, have no access to customized instructions that ensure that they construct the activity within the safe physiological limits.

Key motivations include:

- The avoidance of cardiac overload: The spike in heart-rate or oxygen deprivation can be lethal in high-risk patients; AI-based monitoring will reduce these risks.
- In favor of rehabilitation: Cardiac event recovery patients need a specific endurance recovery, supported through adaptive feedback.
- Enabling self-control: With practical insights, the users will be able to make changes in their lifestyles without having to be under continuous clinical supervision.
- Closing the divide: The system bridges clinical healthcare and daily fitness such that the wellness technology fosters safety and not just activity.
- Long-term monitoring: Data analysis on a regular basis can show early warning signs prior to the appearance of symptoms, and early prevention can take place.

Adaptiv Health is not just a fitness app but a digital cardiovascular companion that helps to manage the heart-health experience in a safer, smarter and sustainable way.

Expected Impact

By introducing Adaptiv Health, the project aims to:

- Increase cardiovascular awareness and education amongst users.
- Reduce the risk of cardiac events that occur during exercise.
- Create a scalable online preventive heart care.
- Invest in the international digital health and AI-driven wellness programs.
- Operate as a study site in terms of time-based population-level pattern analysis of cardiovascular conditions.

Gantt Chart

Duration: October 2025 – March 2026

Trimesters:

Trimester 1 (Autumn 2025): October – December 2025

Trimester 2 (Winter 2026): January – March 2026

Phase	Task	Oct 2025	Nov 2025	Dec 2025	Jan 2026	Feb 2026	Mar 2026
Project Initiation & Research	Define goals, scope, and success metrics	■					
Project Initiation & Research	Market & literature research (AI in cardiology, IoT standards)	■	■				
System Architecture & Design	Define system architecture (AI + IoT + Cloud)		■	■			
System Architecture & Design	Data security & compliance planning (HIPAA, GDPR)		■	■			
AI & Data Pipeline Development	Dataset preparation & preprocessing			■	■		
AI & Data Pipeline Development	Model design & training for cardiovascular pattern detection			■	■	■	
Application Development	Mobile & web interface prototyping		■	■	■		
Application Development	Integration with wearable IoT devices (heart rate, BP sensors)			■	■	■	
Testing & Validation	AI model testing (accuracy, sensitivity)				■	■	
Testing & Validation	User testing & feedback (rehab/patient simulation)					■	■
Deployment & Evaluation	Cloud deployment & data privacy audit					■	■
Deployment & Evaluation	Final review, documentation & presentation						■

3. Project Description and Objectives

Project Overview

Adaptiv Health is an artificial intelligence cardiovascular health app which is aimed at assisting patients with heart diseases or people who might develop them. It makes use of actual data from reputable medical archives like the PhysioNet and the Kaggle Cardiovascular Disease datasets in order to prepare machine-learning models that can detect cardiovascular patterns. These models customize exercise and recovery training programs according to the user's vital, historical and behavioral patterns.

System Experience:

The interaction between the users will be on the basis of a mobile application and a web-based dashboard will be used to ensure that the user can access it and have access to analytics readily. When registering, users enter their medical information and enable wearable devices streamlining real-time heart rate and oxygen rates. This information is examined by the AI system on a daily basis and results in individualized suggestions, such as the duration during which it is safe to exercise, the optimal heart-rate range, or restoring activities. Whereas real-time adaptive alerts are the optimal final target, the present implementation will be concentrated on providing daily plans tailored to the specifics of the personal analysis of the ongoing data.

Objectives

1. To create and model an AI based cardiovascular monitoring system with real patient data.
2. To offer custom exercise and recovery initiatives that are responsive to the heart condition and progress of a user.
3. To incorporate the IoT, cloud, and AI into a hybrid (mobile web) system to make it more accessible and synchronize data.
4. To make sure that user data is secured and transferred safely and ethically and privately.
5. To support preventive care provision through the provision of practical information that enhances cardiovascular awareness and protective measures.

Combining AI smarts with clinically-acquired information and contemporary software development, Adaptiv Health can transform the process by which individuals can keep track of their heart health not as a patient, but as an empowered user of their recovery and wellness process.

4. How We Do It

System Architecture (IoT → Cloud → AI → Dashboard)

Wearable IoT sensors, secure cloud infrastructure, advanced AI analytics, and an interactive dashboard are linked on the integrated, layered architecture of Adaptiv Health. Its architecture is designed to offer high-performance continuity, constant heart monitoring, and safety of patient data.

1. IoT Layer (Wearables & Mobile Gateway)

The system begins by using wearable IoT sensors, including optical heart-rate and SpO2 sensors, which include Fitbit, Apple Watch, or custom ESP32 modules. Such devices will be able to constantly gather important data like:

- Heart rate (BPM) for activity intensity
- Blood oxygen level (SpO₂) for oxygen circulation efficiency
- Blood pressure (BPs/BPd) for cardiac workload estimation

The data is received by the smartphone application of the user, which is used as an edge gateway via Bluetooth Low Energy (BLE). Once some basic safety checks (i.e., false readings or sensor noise) have been done, the software saves the information and transmits it to the cloud to be further analyzed.

2. Cloud Layer (Data Management & Integration)

The cloud layer handles all the data collection, safe storage, and cross-component communication. It is founded on the cloud services that are HIPAA-compliant, such as Azure IoT Hub, AWS HealthLake, and Google Firebase.

Key cloud functions include:

- Real-time data streams processing data streams to perform continuous monitoring.
- Time-series databases that archive past data to be used to analyze long-term patterns.
- Secure APIs that limit access to data to certified medical workers or carers.
- Role-based authorization and authentication of users with encrypted logins.

This architecture is scalable, which means that it can support always-available processing without requiring much computing power on the device of the user.

3. AI Analysis Layer (Decision & Learning Engine)

The intelligence at Adaptiv Health is based on the AI module. It forecasts safe activity limits and cardiovascular rhythms interpretation through reinforcement learning and a supervised algorithm.

Core AI functions:

- **Baseline learning:** computes the average heart-rate range of each user when at rest and during an activity.
- **Risk classification:** identifies whether the cardiovascular risk of the user is low, moderate, or high.

- **Adaptive suggestions:** dynamic suggestions are workouts that adapt depending on previous workouts, including the duration and recovery.
- **Abnormality development:** sound an alarm on abnormal measurements (arrhythmia, low SpO2, high BP) to be investigated.

The AI model is constantly retrained on the updated user data over time in order to increase customization and predictive accuracy.

4. Dashboard & User Interface Layer

Dashboard is the primary means of communication between the system and the user. It can be accessed with the help of the internet and mobile applications.

Features include:

- **Real-time monitoring:** color-coded indicators (green = safe, amber = approaching limit, red = risky).
- **Instant alerts:** haptic or voice notifications when readings exceed safe limits.
- **Session summaries:** time in target zone, peak heart rate, recovery duration, and calories burned.
- **Progress visualization:** trend graphs showing improvement in cardiovascular stability.
- **Cloud sync:** data automatically saved and accessible to healthcare professionals if permitted.

Workflow and User Interaction

1. **User Onboarding & Setup:** The user puts on their wearable device, creates an account, and fills out their cardiovascular history and demographics. The app takes a short guided setup to calibrate the device and ensure that the sensors are connected.
2. **Baseline Collection Phase:** During the first 3-5 days, the system will capture normal activity data so as to create an individualized resting heart-rate profile and safe thresholds.
3. **AI Analysis & Classification:** When the analysis of the base data is done, the AI model classifies the condition of the user (e.g., moderate-risk). Then, it develops a first training regimen which focuses on safe and mild cardiovascular exercise.

4. **Live Monitoring During Exercise:** It measures the heart rate and oxygen levels in real-time during exercise. These can be visualized in real time using the mobile app, and an immediate alert message is sent to the user when the values become dangerous.
5. **Adaptive Response & Alerts:** Once the heart rate goes too high, the app emits a real-time voice or on-screen message (Slow down - above safe zone) to increase or decrease the rest of the workout intensity.
6. **Post-Session Summary:** At the end of every session, the user is provided with a detailed health report of the average heart rate, recovery time, and time spent in the target zone.
7. **Learning & Adjustment:** The AI model takes into account every session and improves suggestions in the future, days of rest, and types of activity.
8. **Cloud Storage & Access:** The data on every session is ultimately stored and encrypted in the cloud to be viewed, tracked, or shared with clinicians in the long term.

High-Level Operation Summary:

Wearable Sensors → Mobile App → Cloud Infrastructure → AI Model → Dashboard
Feedback → User Action → Continuous Learning Loop

How the Design Ensures Feasibility

- It was constructed with all the available technologies, such as Python, TensorFlow, Firebase, Flask, and Mediapipe, to integrate the IoT and AI.
- Cloud-based design minimizes hardware dependency and cost.
- System modularity gives the team members the ability to build each component concurrently.
- Real-time alerts rely on edge AI (mobile processing) to minimize latency, ensuring user safety even without a stable internet connection. Alerts are provided in real-time and are based on edge AI (mobile processing) to reduce the latency and keep the user safe in case of a lack of a stable internet connection.

5. Software and Hardware requirements

Software Requirements:

Adaptiv Health has multifaceted technical requirements to meet its level of complexity that can be discerned only through proper analysis and research. After thorough research there are some software's that we trust to bring about the technical flamboyance that would output the desired results we expect. It ranges from languages like python to Amazon web services for cloud computing. We have detailed the need, features and limitations in the sections to follow.

Backend: Development & AI Technologies

- **Python 3.13 + TensorFlow 2.20.0 Ecosystem**

Python is an Universal computer language accepted by healthcare and is HIPAA compliant with a large range of supporting libraries together with a TensorFlow ecosystem that powers the AI brain to analyze and make predictions for users. TensorFlow framework's LiteRT Runtime enables 15ms inference times which is critical for real-time cardiac monitoring on mobile devices.

Python library:

- Scikit-learn 1.5: to the local simple machine learning that does not require deep machine learning.
- KNNImputer: A scikit-learn class that imputes the missing values with the nearest neighbors.
- Pandas/NumPy: To analyze tabular and time-based data. Numpy facilitates the computations of math whereas Pandas structures data in tables.

AI architecture in Cardiac-Specific Applications Long Short-Memory Cardiac-Specific Long Short-Memory memory networks and cardiac-specific detectarrhythmia Proposals on PPG signals; Convolutional neural net and arrhythmia detection on ECG signals; Reinforcement learning and optimizing personalized exercise prescription. It will be operated on Edge AI that will be used in local detection and is faster than cloud AI but will also work on cloud AI that will train data on a mass scale and make complex analytical decisions.

Frontend Technologies

Flutter 3.24 (Patient Mobile App)

Flutter shows the best performance in real-time health monitoring applications, especially, in the complicated visualization of cardiovascular data, which is HIPAA-compliant. Key advantages include:

- ❖ Live heart rate graphing performance at 60 fps.
- ❖ Offline-first architecture that does not lose essential safety functions with the loss of connectivity.
- ❖ Wearable device communication with Native Bluetooth Low Energy 5.3.
- ❖ 95 percent of development time saved (iOS/Android) through cross-platform deployment.

React 19.2 (Doctor Dashboard)

The latest 19.2 version supports Activity Component which preloads data and helps switch between statistics faster while sticking only relevant information from the servers thus protecting patient data. Also display a list of patient info and alerts for patients with a concerning heart rate pattern. Helps doctors to review workout history and adjust routines remotely.

Database & Cloud Infrastructure

Hybrid Architecture (Firebase + AWS)

Critical October 2025 Update: Firebase alone is NOT HIPAA-compliant. Recommended architecture:

- ❖ **Firebase/Firestore:** Real-time synchronization for non-PHI data when needed. Like a workout routine or step count.
- ❖ **AWS RDS PostgreSQL:** HIPAA-eligible storage for Protected Health Information with Business Associate Agreement. Such as patient medical records, heart conditions and medications.
- ❖ **AWS EC2 (C8/R7 instances):** It runs in backend servers and processes data and server APIs. There is vital data uploaded daily from the user to the larger cloud AI model. It calculates health score data which are too complex for phones to run locally.

Configuration: Firebase owned by Google requires Google Cloud Platform Business Associate Agreement setup for HIPAA compliance; AWS services (EC2, RDS, S3) are automatically HIPAA-eligible under AWS BAA.

Monthly Cost (100K users): \$4,600 (AWS) + \$200 (Firebase) = \$4,800 total.

Other Project Tools

- Postman: a widely used API in health tech that will make sure that only data that is relevant is provided. In other cases, developers use it to design, test and document APIs.
- VS Code Python/Dart/JavaScript integrated development environment.
- GitHub: Version control in order to allow teams to collaboratively work without stepping on one another with storage through Git LFS (large file storage) of large AI weights.
- Figma HIPAA-accessible UI/UX designed to comply with WCAG 2.1 accessibility.

Security software stack

- **Endpoint detection:** CrowdStrike Falcon is HIPAA-certified and ensures end to end protection for patient devices and medical personnel systems. Protects against phishing and ransom attacks.
- **Cloud Security:** Zscaler or Prisma Cloud protects Firebase and AWS cloud environments with active threat scanning, encrypted tunnels, and access segmentation controls.

- **Data Loss Prevention:** ManageEngine DLP+ would ensure patient health data from Adaptiv's app, reports, and hospital dashboards cannot leak through human error or insider misuse.
- **AI Threat Detection:** Darktrace ActiveAI adds an "immune system" for the digital infrastructure, automatically identifying and neutralizing abnormal cloud or IoMT (Internet of Medical Things) behavior before damage occurs like an unpermitted download of patient data at 3 am.
- **Risk Management & Compliance:** Censinet RiskOps automates HIPAA compliance culture that anticipates vulnerabilities in hospital partnerships, wearable data exchanges, and cloud vendors.
- **Backup & Recovery Layer:** AWS S3 Object Lock + Immutable Snapshots Protects backups against ransomware or corruption by allowing patient recovery histories and making copies.

Hardware Requirements:

Adaptiv Health is based on a series of secure, interoperable hardware which enables cardiovascular monitoring and communication of patient, mobile and cloud data to be accurate and safe. The hardware ecosystem of the system should allege accuracy, portability, and privacy, particularly because a number of the users will be handling heart conditions.

Wearing and Sensing Devices

At the user level, the platform is interconnected with commercially available wearables including the Fitbit Charge 6, Apple Watch Series 9, or Polar H10 chest straps that are medically certified. These monitors record physiological data, such as the heart rate (HR), heart-rate variability (HRV), and blood-oxygen (SpO2) using optical or electrical sensors. In case of experimental/low cost deployments, the same values can be simulated by using an ESP32 microcontroller with a MAX30102 photoplethysmography (PPG) sensor. The sensors all have Bluetooth Low Energy 5.3, which offers a stable transmission with a small battery consumption.

Edge and Mobile Processing

The smartphone of each user is an edge-computing node, with an implementation of a lightweight TensorFlow Lite model which runs on-device safety and risk scoring checks and scores before any data is sent out of the phone. This design does not only decrease latency, it also minimizes the information that would be identifiable to the cloud.

Cloud and Networking Infrastructure

The back-end services will be implemented using AWS EC2 instances and an RDS PostgreSQL cluster that will be used in a HIPAA-eligible Virtual Private Cloud (VPC). The clinician access to the dashboard and communication between servers and devices is safeguarded by VPN gateway and TLS 1.3 encryption, respectively. The development and testing will be based on regular laptops (at least 8 GB RAM, SSD drive) which will be connected to a stable broadband network to simulate the real-time data flow.

System Security Hardware

Additional, non-obligatory but advisable items are a Hardware Security Module (HSM) or AWS Key Management Service to store the keys, and router-level firewalls to avoid unwarranted incoming traffic. These are combined to produce a sensor to clinician dashboard encrypted loop.

Challenges

Technical Challenges

- **Real-Time Latency for Critical Safety Alerts**

Problem: Patient safety requires <1-second response when heart rate exceeds safe thresholds or arrhythmia detected. Traditional cloud processing introduces 1.3+ second latency.

Solution: Edge AI Architecture with TensorFlow Lite which is 13x times faster than cloud. Wearable device to phone 50ms, AI processing 100ms, local alert generation 10ms which is a total of 160ms max.

Implementation: Quantized TensorFlow models (<50MB) deployed via Flutter app enabling offline safety monitoring. Cloud processes non-time-critical tasks (daily health scoring, trend analysis).

- **System Scalability**

Problem: Support 10,000 concurrent workout monitoring sessions plus 100,000 daily health score calculations without performance degradation.

Solution: Multi-Tier AWS Auto-Scaling Architecture.

Performance Standards:

API Response: <200ms (p95), <500ms (p99), Database Write Latency: <50ms,
System Uptime: 99.9%.

Data Challenges

- **Data Quality & Missing Values**

Problem: Wearables can go missing due to lack of usages or battery loss. Such gaps can create bad data. **Solution:** Multi-Stage Validation Pipeline.

We need to check if readings are well within safe range or at least possible range.

Our tech should also detect obvious sensor errors like a heartbeat jump from normal to astronomical scale. The KNNImputer library uses statistical imputation to fill gaps

and ignore days with too much missing data. The cloud AI should be able to absorb and learn patterns from a bigger audience to understand larger gaps.

Imputation strategy:

- ❖ Short gaps (<5 min): Linear interpolation
- ❖ Medium gaps (5-30 min): KNN imputation during activity
- ❖ Long gaps (>30 min): Flag as missing, exclude from training

- **Synthetic Data Training Limitations**

Problem: It is presently hard to gain access to real life cardio data SO relying on synthetic data can provide statistics that may not be as realistic or accurate.

Solution: Progressive Real-World Integration (with FDA approval). Over long time use of the app by a larger group of people can provide essential and crucial data that can be used for machine learning. The first two weeks are a learning phase for the edge AI to personalize the health status to the user.

Ideal scenario:

Phase 1 :80% synthetic + 20% real (proof-of-concept, NOT deployed)

Phase 2 :50% synthetic + 50% real (hospital pilot, 1,000 patients)

Phase 3 :100% real data (production deployment)

Security and Ethical Problems.

Constructing an AI-based health platform implies a challenging complex of technical, ethical, and legal risks. My area of interest as the cybersecurity lead is that the innovation of Adaptiv Health should not jeopardize patient safety or trust.

Data Privacy and Compliance

The key issue is that it is necessary to address the Protected Health Information (PHI) and comply with HIPAA and GDPR. Sensitive data should be encrypted during transmission (TLS 1.3) and rest (AES-256). Only the HIPAA-eligible AWS services will be used to store patient records anonymously. Data Protection Impact Assessments (DPIAs) will be conducted to ensure that the practice of handling data is in line with the international standards (HIPAA Journal, 2024; European Commission, 2024).

Authentication and Access Control

Since various roles such as patients, clinicians, and administrators, get involved in the system, access control is another issue. The use of OAuth 2.0 and OpenID Connect as well as role-based access control (RBAC) will ensure that unauthorized data exposure does not occur. Rotating tokens are done periodically and all attempts of access are logged to aid in auditing.

Ethical AI and Transparency

The AI suggestions regarding the intensity or rest of exercises might be confused with medical guidance. To avoid excessive dependence on algorithms, the interface of Adaptiv Health will have obvious disclaimers and call-your-doctor functions in case of abnormal reading. The scoring model will be interpretable, demonstrating what metrics had an impact on an outcome to make informed decisions and clinical supervision (Amann et al., 2020).

User Consent and Control

The sustained observation brings up the issue of implicit consent. The user will be able to explicitly and revocably control data streams shared. The application will have privacy controls to put tracking on hiatus, will have an option of deleting the stored data or will export summaries to a physician, in accordance with the GDPR principle of a right-to-be-forgotten.

System Integrity and Security Assurance

It is vital to have integrity among the distributed devices. Any software changes will be signed and checked by cryptographic hash sums (SHA-256). Before every deployment phase, the penetration testing and vulnerability scans will be conducted. This multi-level strategy is a guarantee of regulatory compliance and user trust with regard to the safety of the system.

Future Improvements

1. **ECG Integration:** Provide an option of adding single-lead ECG sensors to allow detection of arrhythmia and irregular heart rhythm so that clinicians receive a more accurate diagnostic assistance.
2. **Predictive Risk Alerts:** Have predictive algorithms that predict cardiac or overexertion based on historical vitals, environmental and the type of activity.
3. **Healthcare Provider Partnerships:** Provide protected clinician portals that give physiotherapists or doctors a chance to check patient progress and revise exercise plans remotely.
4. **Expanded IoT Ecosystem:** Add new devices, including Garmin, Samsung Health, and medical-grade blood-pressure monitors.
5. **Voice and Virtual Coaching:** Add AI voice coaching to provide hands-free coaching (“Heart rate too high - take a 30-second break”).

6. **Federated Learning Model:** Machine learning should be done on user devices so that the training process is more personalized without exposing user-specific health information, which improves privacy.
7. **Community and Rehab Programs:** Partner with hospitals and fitness centers to provide Adaptiv Health as a component of cardiac-rehab programs.
8. **Clinical Validation & Certification:** Find collaboration with pilot studies to obtain medical-device compliance (i.e., FDA or CE Mark).

6. Project management tools to be used

Adaptic Health team will follow semi-Agile project management style, with flexibility, collaboration and continuous feedback but with definite milestones. It will follow Agile development principles, which include iterative development and reviews conducted after each sprint, without the formal rigidity of complete Scrum processes.

Tools and Platforms:

GitHub: A version control, code collaboration and issue tracking tool. It allows employees in a team to combine their efforts and test them effectively as well as have a single codebase.

Google drive: main storage of shared documentation, that is, reports, diagrams, and meeting notes. This would guarantee that the members are in synergy with the current materials.

Google meet and WhatsApp: Main communication tools to use, weekly online meetings, and coordination.

Canva / Google Slides: To make presentations and document visually as an update on the progress.

Team Coordination

Every stage of the project will also have established deliverables and internal deadlines to have project progress in line with the CSIT321 timelines. The roles and responsibilities have been defined to evenly spread out the technical, analytical and documentation workloads among all the six members. Scheduled meetings with the designated mentor and project supervisor on a weekly basis will enable the team to overview the milestones, solve problems as they come by and modify plans accordingly.

This architecture encourages transparency, accountability, and continuous improvement which are important parameters in producing a project of both academic rigor and professional software development standards.

7. Diagrams:

Figure 1. Use Case Diagram — Adaptiv Health System

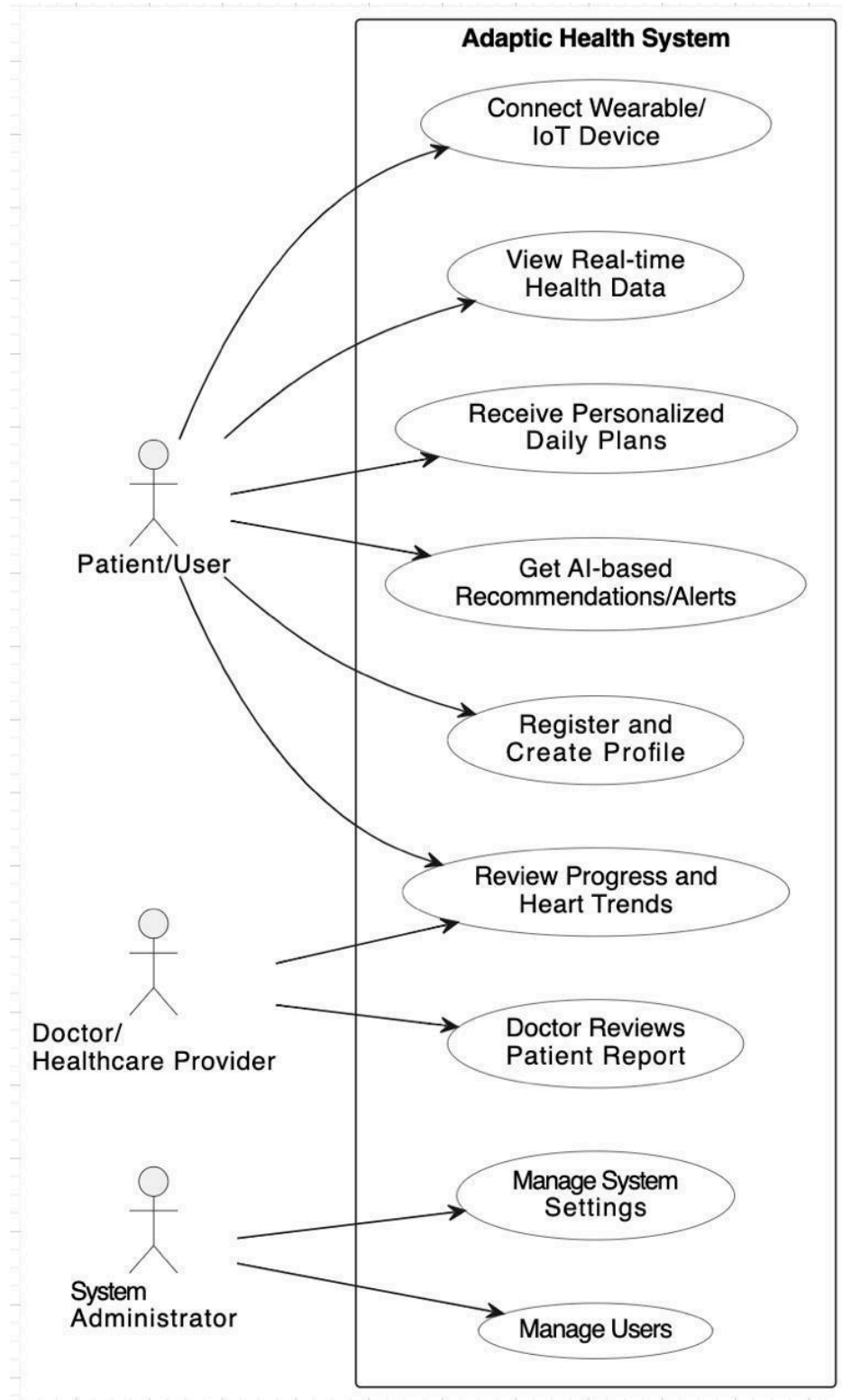


Figure 2. Sequence Diagram — Real-Time Monitoring and AI Feedback Flow

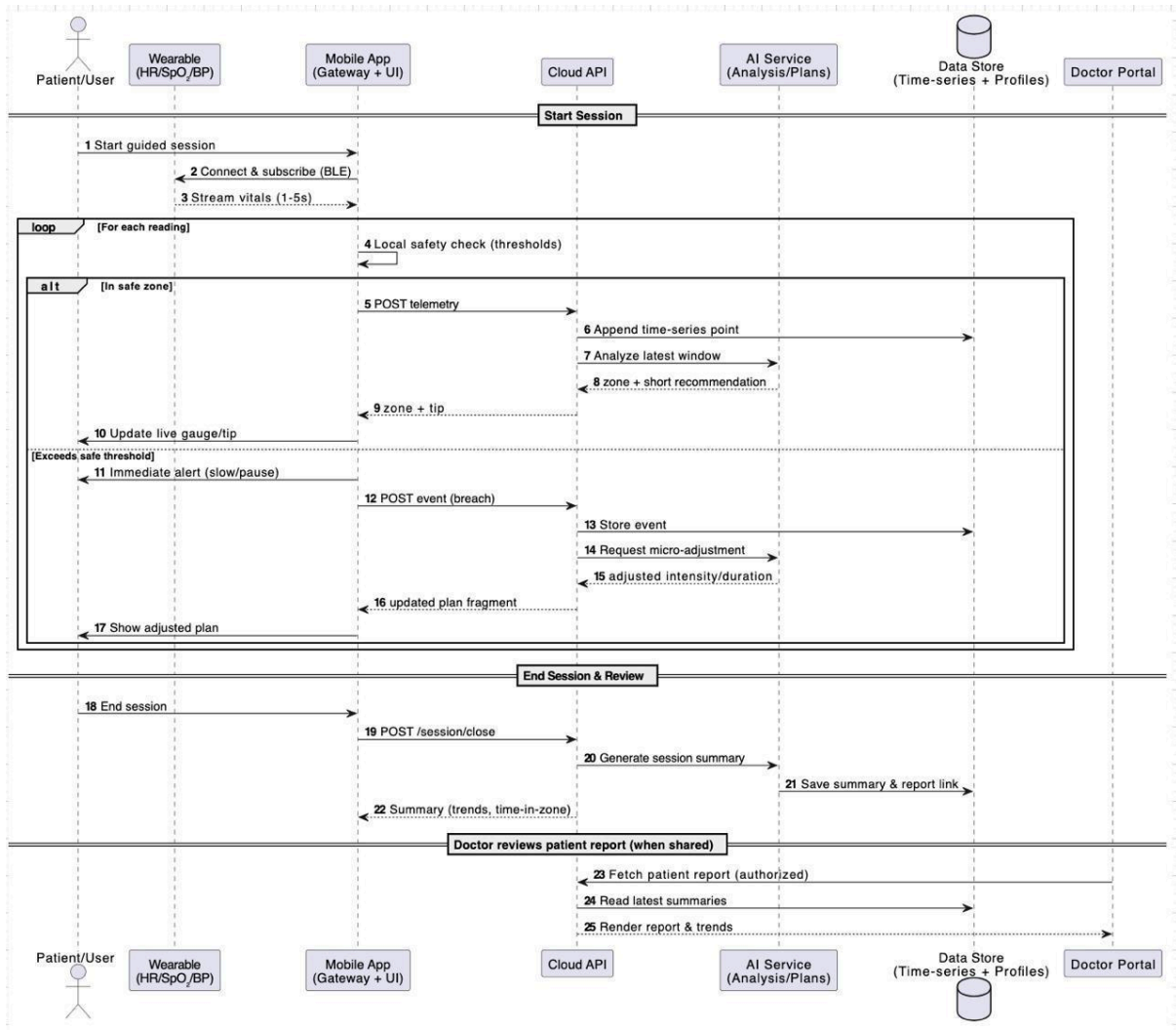


Figure 3. Component Diagram — System Architecture Overview

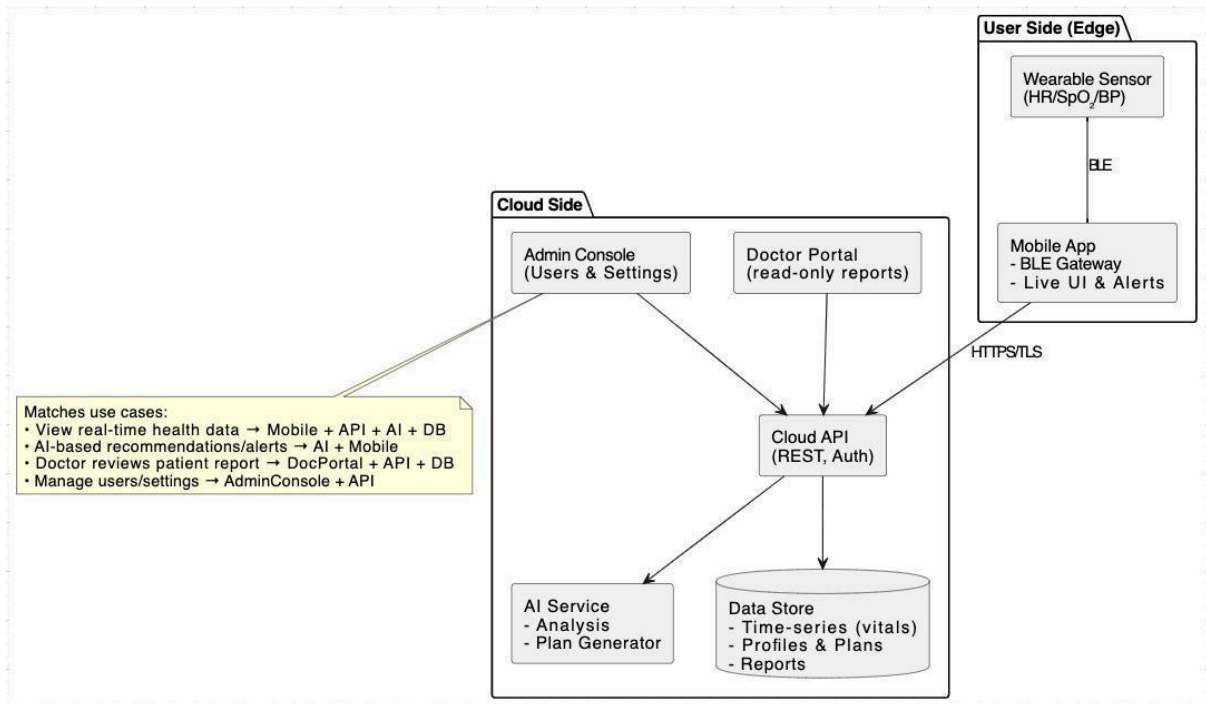
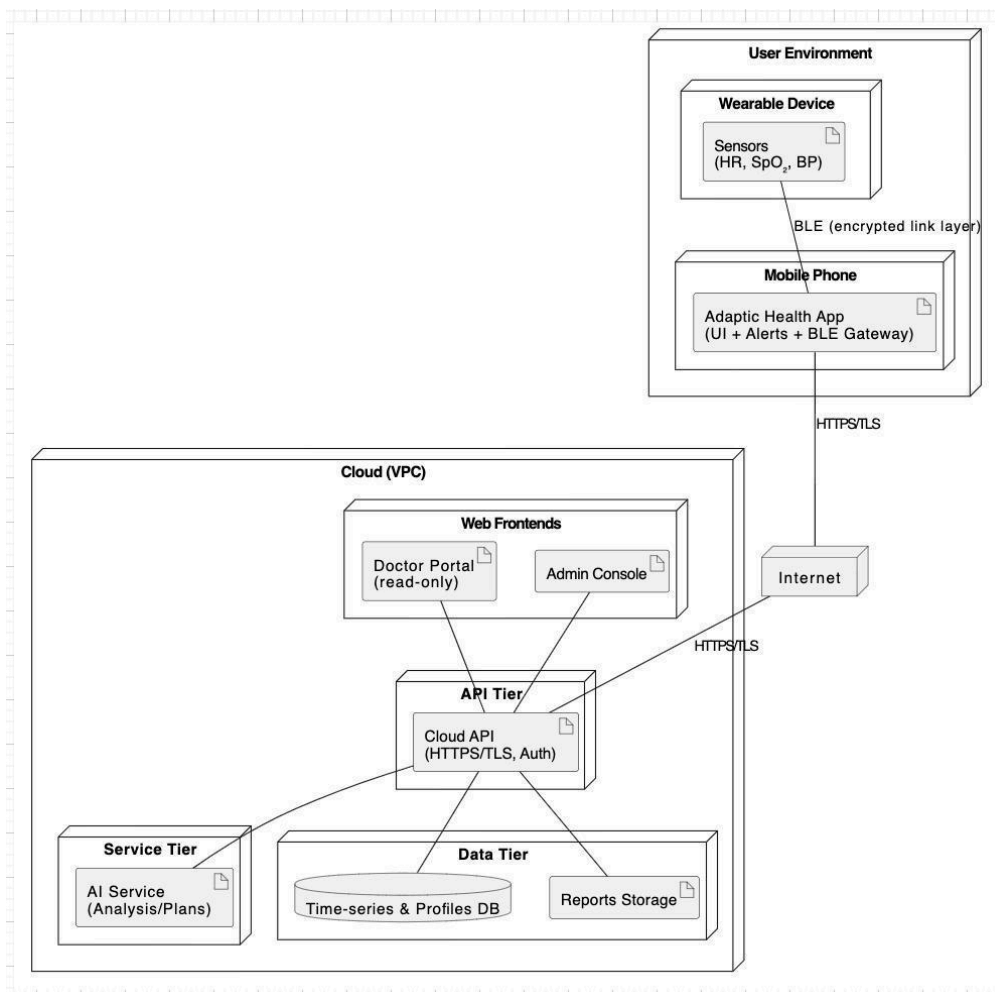


Figure 4. Deployment Diagram — Runtime Environment and Data Flow



8. Teamwork Distribution

Members	Specialization	Primary Responsibilities	Key Deliverables
Omar Massoud (7858681)	AI and Big Data	Training of the adaptive AI engine using real cardiovascular data; development and design of machine learning models to make predictive health recommendations.	Algorithms, data flows and trained AI models.
Abdallah Heggy (8457785)	AI and Big Data	Data preprocessing, cleaning, and management of actual patient datasets (PhysioNet, Kaggle, etc.); analytics to generate plans on a daily basis.	Scripts on data management, data visualization and a model evaluation report.
Bithin Radhakrishnan (8910881)	Cybersecurity	Introduction of data encryption and secure communication between IoT devices, cloud servers, and the app; making sure that the standards of healthcare data are adhered to.	Architecture, encryption modules and privacy compliance documentation.
Hamza Al Raja (7322586)	Cybersecurity	Risk evaluation and vulnerability testing of the application; creation of user authentication and access control systems.	Security test reports, authentication modules and compliance summary.
Karylle Ochave (7669057)	Business information systems (BIS)	Market research, analysis of feasibility, and schedule of the project; creating Gantt charts and project documentation.	Project timeline, feasibility report sections, and business case.
Ghina Ishaq (8711719)	Computer Science - General	Combination of AI, IoT, and cloud elements; development of the hybrid mobile-web interface and upkeep of project documentation.	Working prototype, integration plan, final documentation and system diagrams.

Software Requirements Specification

Report 2: Requirements Analysis (SRS)

Submitted: Trimester 1, Autumn 2025 | Team Pulse | CSIT321

1. Introduction

1.1. Purpose

This System Requirements Specification (SRS) defines the system's functional and non-functional requirements for Adaptiv Health. This is an exhaustive outline of how the system will provide safe cardiovascular exercise through real-time feedback, AI analysis, and incremental, adaptive suggestions.

This SRS is relative to previously established documents such as the **Business Requirements Specification (BRS)**, the **Planning and Feasibility Analysis** and the **Project Proposal**. Thereafter, this SRS will serve as the technical reference for:

- Creation of the **High Level Design (HLD)** document.
- Creation of the **Low Level Design (LLD)** document.
- Integration, testing and deployment implementation.
- Baseline documentation for stakeholder (students, mentor, clinicians and future industry partners) communications.

1.2. Notation

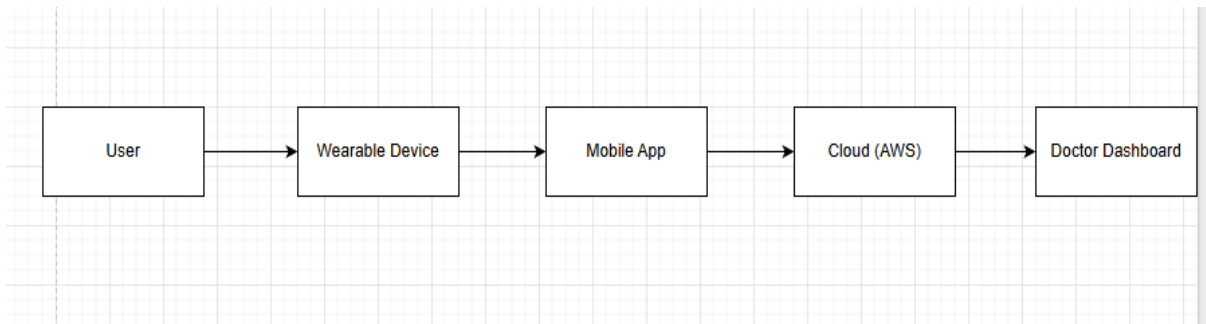
This document conforms to the following general notations applied in software-engineering documentation:

- UML Diagrams (Use-Case, Component, Sequence). \ (Figure 1 - Heart Rate Sequence with System Constraints represents the entirety of the data flow with user - wearable - app - cloud - dashboard.)
- Requirement Numbering. General functional items are categorized with REQ; performance REQ are PERF. Where there are extended security, availability and capacity REQs these are noted as SEC, AVA, CAP.
- Data Flow. Data flow is established between BLE → Mobile App → Cloud (AWS RDS/Firebase) → Dashboard.

1.3. Scope

Adaptive Health is an IoT, mobile app, cloud analytics and clinician dashboard system for safer exercise. It collects heart-rate, SpO₂ and HRV data from medical grade wearables in real time, performing edge AI on the smartphone for instantaneous (less than a second) notifications with ancillary cloud AI for long-term assessments. It provides wearers with an adaptive workout plan and graphical overviews of their performance while clinicians receive HIPAA secured access to a patient's web-based dashboard. Data is stored in accordance with GDPR and DHA health data regulations.

1.4. Context Diagram



1.5. Definitions and Acronyms

This section describes any terms used.

IoT – Internet of Things; connected sensors and wearables.

BPM – Beats Per Minute (heart rate).

SpO₂ – Oxygen saturation level.

HRV – Heart-Rate Variability.

PHI – Protected Health Information.

HIPAA – U.S. Health Information Privacy Law.

GDPR – EU General Data Protection Regulation.

DHA – Dubai Health Authority (UAE health data law).

Edge AI – AI running locally on the device.

Cloud AI – AI processing on cloud infrastructure.

1.7. Overview

This paper spells out what Adaptive Health should do and how it should do it.

Section 2 Choice of system functions and users.

Section 3 Software and hardware limitations.

Section 5 Other assumptions and dependencies.

Summary of functional requirements.

Section 6a Detailed functional requirements.

Section 7 Exterior interfaces and connections.

Section 8 Non-functional performance, security, availability and capacity.

These sections together form a whole technical basis of the Adaptive Health project.

2. General Description

2.1. System Functions

Adaptiv Health combines wearable IoT, artificial intelligence, and cloud analytics to deliver safer and smarter cardiovascular care. The system operates through several interconnected functions that work in real time to monitor, analyze, and adapt to the user's condition.

Continuous Physiological Monitoring: Wearable devices such as Fitbit, Apple Watch, and Polar H10 provide real-time measurements of heart rate, oxygen saturation, and blood pressure. These readings are transmitted to the mobile application through Bluetooth, where the data is cleaned, checked for missing packets, and formatted for analysis.

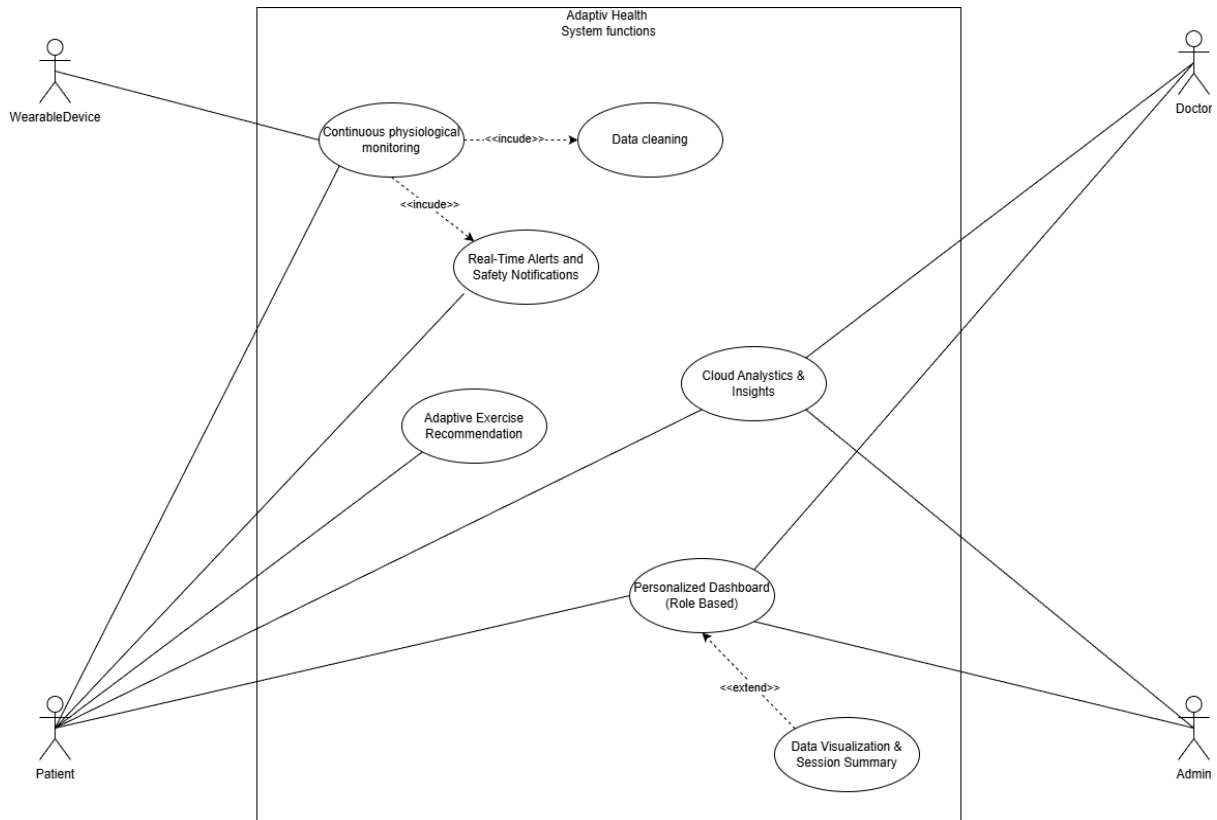
Adaptive Exercise Recommendations: The AI engine evaluates the user's cardiovascular data and adjusts exercise recommendations automatically. When vital parameters exceed safe limits, the system sends immediate guidance such as "slow down" or "take a break" to reduce overexertion and maintain heart safety.

Alerts and Safety Notifications: Real-time visual, audio, or vibration alerts are triggered locally on the device using edge AI. The alerts appear within milliseconds, even when the network connection is weak, ensuring the user is immediately informed of potential risks such as arrhythmia or hypoxia.

Data Visualization and Post-Session Summary: The application displays live data on interactive dashboards with color-coded zones such as green for safe, yellow for caution, and red for high risk. After each session, a summary report provides details such as the highest recorded heart rate, recovery speed, and total time spent in the optimal zone.

Cloud Analytics and Long-Term Insights: All recorded data is securely encrypted and synchronized with the cloud using HIPAA-compliant platforms such as AWS, RDS, and Firebase. Over time, the system analyzes these records to detect trends, improvements, or early signs of potential cardiac issues.

Personalized Dashboards: The dashboard shows all system elements, showing personalized progress charts, health recommendations, and performance insights. Access is role-based; patients, doctors, and administrators only view information relevant to their needs, ensuring both privacy and usability.



2.2. User Characteristics

Adaptiv Health is made for a wide range of users, each of whom interacts with the system differently based on their function and level of technical proficiency. The program makes sure that every kind of user has an interface that meets their demands while preserving data security and usability.

End users and patients:

People recovering from cardiac surgeries or those with mild to severe cardiovascular problems are the main users. They assess their daily progress, get adaptive workout instruction, and measure their vital signs using wearable technology and the mobile app. The majority of patients should be comfortable using smartphones and health-tracking gadgets and have a moderate level of digital literacy. The system has straightforward instructions, graphic indicators, and easy onboarding procedures to help less tech-savvy individuals.

Physiotherapists and clinicians:

The web-based dashboard allows medical personnel to monitor patient progress, spot cardiovascular trends, and adjust rehabilitation regimens. They are more technically proficient and have more experience deciphering digital health data. Accuracy and efficiency are the main goals of the clinician interface, which offers safe access to comprehensive graphs, summaries, and warnings

pertinent to patient care.

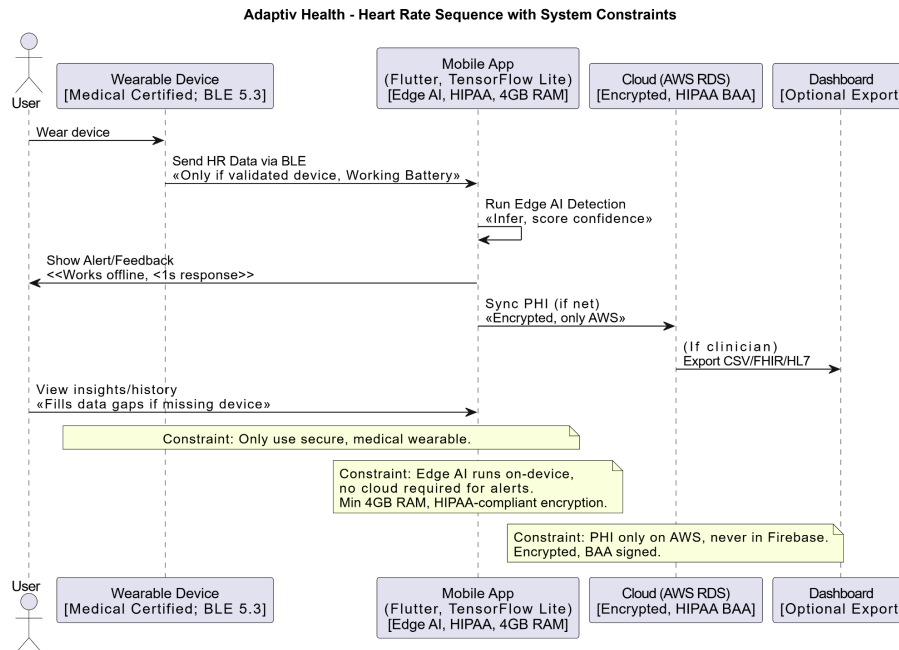
Family members and caregivers:

With the patient's permission, caregivers are always given restricted access. In order to monitor or support the user during recuperation, they can view alerts and brief summaries. The interface that caregivers see is simplified, displaying only the most important information, like notifications and status updates, because many of them might not be familiar with sophisticated technology.

System Administrators and Technical Staff:

Account management, system integrity, and cybersecurity are within the purview of administrators. They handle user permissions, backups, and data protection compliance mostly through the admin web console. These users usually possess advanced IT skills and have received training on healthcare security regulations like GDPR and HIPAA.

Overall, Adaptiv Health operates on several platforms, including a wristwatch interface for real-time monitoring and notifications, a web dashboard for physicians and administrators, and a mobile app for patients and caregivers. The system's interface, iconography, and guided setup make it simple to use for people of all ages and ability levels, even though some experience with digital devices is anticipated.



3. General Constraints

Our project has its own constraint that needs to be met due to limitations of time and coursework. Other than that, there are several social, technological constraints that we need to keep in mind to meet the requirements for our project. Some challenges that put up the constraints are wearable sensor accuracy across diverse populations, Health standards, data validation, comprehensive unified language, AI development for accurate data processing and understandability of processed information. They affect the three traditional constraints: scope, time and cost.

3.1. Software Constraints

3.1.1 Programming Language (Python 3.1x)

Python is the standard programming language used in AI and biomedical applications which is safe and meets HIPAA/GDPR standards. Introducing another language to this ecosystem would make it hard for people across all departments to manage. Python has a huge library in its database that is closely associated with software development healthcare needs which will in turn help with faster development of modules and its integration. We are hoping to use the latest available version of python for the added features and libraries available in the future upgrades.

3.1.2 AI Frameworks (Edge AI + Cloud AI)

We expect our project to continuously monitor data inputs and perform fast analysis in case of an emergency. This requires us to develop a light weight local AI model for in-app data processing within the smartphone on top of the cloud AI which will do a deeper analysis of trends and machine learning. TensorFlow Lite is a fast, super lightweight, common in health care ecosystem, that works on android and iOS which is HIPAA compliant which can contribute as edge AI and do all critical emergency fast detection. TensorFlow combined with AWS and Firebase for cloud AI, storage, management and security. Our current knowledge on using TensorFlow for AI development is limited but we are hoping to train ourselves and be equipped enough to be able to start working on this project by next trimester.

3.1.3 Mobile App Framework (Flutter)

A project based on personal health info requires a high level of security and meets health standards. Maintaining multiple apps for different OS can create issues and increase chances of error. Flutter is a cross-platform with high level of security and strong Bluetooth LE support. You can safely store data locally with AES-256 encryption and fast smooth operation with a high-performance engine which lets you write code once for multiple OS and saves time.

3.1.4 Database and Cloud

All protected health information must be stored in a HIPAA compliant database. **Non-sensitive data like user app customizations can be stored in Firebase which is faster for real time data synchronization** as it has a built -in database manager however not HIPAA compliant. AWS RDS with PostgreSQL is HIPAA compliant storage for large structured safe health data. All data is secure and encrypted at rest so if compromised they won't be leaked. It is standard and trusted by big wearable device companies and hospitals. Real time synchronization is slower with AWS RDS PostgreSQL. PostgreSQL is better for handling large groups of users and complex queries as compared to other NoSQL database storages.

3.1.5 Security Standards

There are mandated government rulebooks for security standards especially when it comes to health data protection. The US has HIPAA/GDPR rulebooks, following this is the best practice for global compatibility while UAE has federal laws and regulations for health data. Dubai Health Authority (DHA) regulates use of health information, violation of which can cause serious penalties and even imprisonment. DHA requires health data gathered in UAE to be localized and cannot be moved back and forth between servers. There must be specific commitment to all the UAE/DHA requirements in contracts. All the records are to be stored for at least 25 years to be DHA compliant.

3.2. Hardware Constraints

3.2.1 Wearable Devices

Heart recovery requires sharp capture of user metrics through smartwatches, smart wearables, bands or straps. All types of skin tones and exercises don't work with every device. **Proper measurements require wearables that are clinically/FDA approved. All the trusted alternatives could be suggested through the application.** And if the application detects the device to be under performing and when data is not validated the app should prohibit the use of this device and suggest an alternative best fit for the user.

3.2.2 Mobile Devices

App download must be restricted to devices that meet minimum requirements such as BLE 5.0, 4G, 4GB RAM, internet connectivity and sufficient storage. This is to ensure the smooth running of the application in the mobile device. It required porter device-app connectivity, a good internet connection and the application should be able to run smoothly in the background. PPG measurement through the phone is risky for people with darker skin tone types as detection is done through passing of light and blood volume measurement. The feature can be enabled but should not be trusted for accuracy.

3.2.3 Network and Cloud

All critical emergency alerts working locally through edge AI must be able to operate when entering an area with no reception or data timeouts. Hourly summary of aggregated user physiological data payload is transferred to cloud AI for machine learning and user personalization for which internet connectivity is necessary. Missing data in the event of battery loss, or other such occurrence can be covered by KNNImputer tool from scikit-learn library that fills in missing data by taking the average of the nearest data points.

4. Assumptions and Dependencies

4.1 Assumptions

a) User engagement

Even though users are expected to provide steady data, most users are not regular users of smart devices; they sometimes forget to use the device or sometimes forget to recharge it. These holes are expected to be covered through imputation and send regular notification from apps that use the device so there is a steady flow of user data entry.

b) Data Quality Varies

We assume steady quality data for processing output. Output data is based on probability and depends entirely on the quality of the data. Data quality depends on user usage, device endurance, and other demographics or conditions. Users and doctors need to assume the data outputs are fair and accurate assumptions. The system is responsible for making sure the validity of the data acquired and ensuring users the reliability level of the output and informing when it is below standard.

c) Network outage

A steady internet connection at all times. During a network outage the data is expected to be queued up until the next upload. The longer the upload is on hold the more the local data expansion. Upon next upload the services are expected to resume smoothly on cloud end. Such service disruptions hopefully don't confuse edge AI with piling of old data. Proper coding to mitigate this will be done for proper segregation and timestamping data accumulated.

4.2 Key Dependencies and Regulatory Dependencies

a) AWS and Other Cloud Providers

Health information is expected to be protected through AWS. In the event of AWS cost change, subscription cancellation, policy changes or license loss we might lose features. The system needs to be designed so PHI can be safely exported, and cloud services swapped for the next best provider.

b) Wearables

Smart devices that record user data are trusted to provide quality data for processing. If the data is redundant or corrupted through a patch update this will ruin the personalization process for the user. Triggers can be set after enabling pattern recognition through machine learning for quality checks and discarding bad data.

c) Software/AI/ML Model Versions

All must be versioned since if a new version has a bug we can choose not to update or roll back after the update. All updates must be thoroughly checked and run before deployment.

d) HIPAA/GDPR/DHA/FDA

All data must be protected. User information must be high priority as compromising can result in ransomware attacks, data stolen or data corruption. Regular risk assessments must be conducted and follow the legal procedures closely for any changes. Annual policy re-evaluation can ensure privacy laws are followed and deployed in accordance with it. Full government level review and approval will be necessary before a market launch as a product for hospital diagnosis assistance.

5. Functional Requirements Master List

This section lists the functional requirements in summary form. Each of these will be further defined in subsequent sub-sections.

Req. ID Requirement Name Requirement Description

REQ-1.1

Req. ID	Requirement Name	Requirement Description
---------	------------------	-------------------------

REQ-1.1	Real-Time Heart Rate Monitoring	Constant recording and monitoring of heart-rate of wearable sensors.
REQ-1.2	Adaptive AI Exercise Feedback	Individualized responses modulating exercise intensity on physiological data.
REQ-1.3	Emergency Alert System	Monitors the presence of abnormal readings and sends emergency alerts.
REQ-1.4	Data Visualization Dashboard	Shows data trends and summaries as charts and graphs.
REQ-1.5	User Authentication and Data Encryption	Secures the information of users by using secure password and encryption.

6. Functional Requirement REQ-1.1

6.1 REQ-1.1 – Real-Time Heart Rate Monitoring

6.1.1 Description

The wearable device should constantly monitor the heart rate of a user in the app.

The idea is to present the existing BPM on the screen with the least delay.

6.1.2 System Input

Information on the smartwatch or chest strap (such as a Fitbit Charge 6 or Apple Watch 9) transmitted via the Bluetooth Low Energy connection.

6.1.3 Display

A small line chart that changes after every few seconds and a numeric value will be displayed on the screen. When the heart rate becomes excessively slow or fast the colour is altered to provide a warning.

6.1.4 System Processing

It gathers the sensor one time a second, filters noise, turns the signal into beats per minute, and records a timestamp locally before sending the information to the cloud.

6.1.5 System Output

The BPM, user ID and timestamp are transmitted to the cloud as well in the form of JSON.

6.1.6 Constraints

Bluetooth connection should be active; it should be between -2 and +2 BPM.

6.1.7 Error Handling

In the event the device takes 10 seconds without transmitting data, a message appears to notify the user to ensure the connection is working. It will resend a couple of times and then it will surrender.

6.2 REQ-1.2 – Adaptive AI Exercise Feedback

6.2.1 Description

This is a personalised exercise advice, which suggests the intensity of the workout in case the live health data indicate to the user that the readings are excessively straining.

6.2.2 System Input

Heart-rate, heart-rate variability, oxygen saturation and personal information such as age and risk level.

6.2.3 Display

A fast message or icon is shown, which instructs the user to either add more or less effort.

6.2.4 System Processing

A mobile lightweight machine-learning model analyzes the information and provides a risk rating between 0 and 1. In case the score is high, the app will recommend to slow down or take a break.

6.2.5 System Output

The cloud stores the feedback which is subsequently displayed on the dashboard.

6.2.6 Constraints

Should take less than two seconds and should take place only when the battery is more than 75.

6.2.7 Error Handling

In the event of an AI malfunction, the application will resort to plain rule based limits e.g. "in case HR > 120 BPM - slow down."

6.3 REQ-1.3– Emergency Alert System

6.3.1 Description

It is the section that addresses emergencies. In case of dangerous readings, alerts are given to the doctors or family members immediately.

6.3.2 System Input

Heart rate, SpO₂, HRV, and individual safe limits of every user.

6.3.3 Display

The screen red lights up and there is a loud alarm signaling that there is an emergency.

6.3.4 System Processing

These values are compared by the cloud after every few seconds; the problem is considered critical when it lasts longer than 15 seconds.

6.3.5 System Output

One is sent an automatic message to the contacts that are listed and the dashboard of the doctor. The event is also logged.

6.3.6 Constraints

The alerts should be emitted within 3 seconds in case the network is alive.

6.3.7 Error Handling

The app will save the alert to the local disk and will continue to make attempts in the future in the event that the internet is unavailable.

6.4 REQ-1.4 – Data Visualization Dashboard

6.4.1 Description

All the received health data are displayed on the dashboard in charts to help the users and doctors notice the trends easily.

6.4.2 System Input

Information that is obtained in the cloud database that contains daily and weekly summaries.

6.4.3 Display

Both the web portal and the mobile application have simple tabs of Heart Rate, Oxygen, Activity Score and Alerts. Physicians are able to compare and export patients.

6.4.4 System Processing

The server will take out the data and format them before sending back to the app in the form of ready-to-draw chart data.

6.4.5 System Output

Graphs, charts, CSV or PDF downloadable reports.

6.4.6 Constraints

Should open up to two seconds in phones and 5 inches and above in screens.

6.4.7 Error Handling

In case the data cannot be loaded, the dashboard indicates the last available data with a notice that it is waiting to sync.

6.5 REQ-1.5 – User Authentication and Data Encryption

6.5.1 Description

This functionality ensures the security of the health data of the user as it will need to log in and encrypt data all the time.

6.5.2 System Input

Email and password/ Google/Apple ID login.

6.5.3 Display

Login page also provides the fingerprint or one-time SMS code. On the three attempts that fail, the account is locked off (5 minutes).

6.5.4 System Processing

The credentials are verified using Firebase Auth, a secure session token is generated, and all traffic is encrypted using the TLS 1.3 and AES 256.

6.5.5 System Output

The authentication generates a safe session, and the system records the time of login and the ID of the device.

6.5.6 Constraints

The sessions are timed out after 30 minutes of being inactive.

6.5.7 Error Handling

Errors such as invalid credentials or session expired will be shown and every unsuccessful attempt will be recorded to be examined.

7. External Interface Requirement

7.1 Data Interfaces

Adaptiv Health is built upon a **multi-tier data flow** system that standardizes wearable devices, mobile apps, and secure cloud infrastructure.

- **Wearable Sensors → Mobile App:**

The information, including the heart rate (BPM), oxygen saturation (SpO₂), and blood pressure, is relayed through the **Bluetooth Low Energy 5.3** to the Flutter-based mobile application. The application does some basic data validation and compression and then passes it on.

- **Mobile App → Cloud (AWS + Firebase):**

The application interacts with the cloud in a secure way with **TLS 1.3 encryption**.

- **Firebase/Firestore:** deals with real-time synchronization of non-PHI data (e.g., workout progress, steps).

- **AWDS RDS PostgreSQL:** contains the Protected Health Information (PHI), encrypted by **AES-256** in accordance with the **HIPAA Business Associate Agreement (BAA)**.

- **AWS EC2 API Servers:** analyze processes, rate them, and provide suggestions through AI in close to real-time.

- **Cloud → Doctor Portal (React Dashboard):**

Analytics, risk alerts, and session histories are available to physicians on a **HIPAA-compliant dashboard** that is linked to a secure REST API using **role-based access control (RBAC)**.

- **Edge AI Feedback Loop:**

TensorFlow Lite models deployed on-device handle local inference to generate instant alerts (<160 ms latency), while summarized statistics are uploaded daily for cloud learning and long-term tracking.

7.2 User Interfaces

Adaptiv Health is based on **medical-grade UI design standards** (WCAG 2.1, HIPAA-accessible). Interfaces are focused on the simplicity, the minimalism, and the ease of use of interfaces with all ages and technical skills.

- **Mobile Application (Flutter 3.24):**

- **Main Dashboard:** This has a real-time heart rate (60 fps), safe zone (green/yellow/red), and current activity.

- **Alerts:** Voice or visual prompts (e.g., “Heart rate too high—slow down”).

- **Reports Page:** Daily summary charts (HRV trends, recovery time, total activity).

- **Settings:** Sharing of data, privacy, and pairing of devices.

- **Doctor Dashboard (React 19.2):**

- **Patient List View:** Displays heart rate patterns, alerts, and summaries.

- **Patient Details:** Interactive graphs, anomaly detection, and AI recommendations.

- **Alerts Panel:** Prioritized list of high-risk events detected by the AI.

- **Access Control:** Login protected with **OAuth 2.0** and **multi-factor authentication**.
- **Accessibility Features:**
 - Adjustable font sizes and contrast modes.
 - Voice prompts for elderly or visually impaired users.
 - Intuitive navigation with limited technical input required.

7.3 Other Interfaces

Interface Type	Description
Hardware Interfaces	Supports commercial wearables (Fitbit Charge 6, Apple Watch Series 9, Polar H10, ESP32+MAX30102). Devices connect via BLE 5.3 for low-latency, energy-efficient data transmission.
Software Interfaces	Integration with Google Fit API , Apple HealthKit , and AWS SDKs for seamless health data exchange and synchronization.
Communication Interfaces	Uses HTTPS with TLS 1.3 for all client-server communications. MQTT optional for IoT real-time data streaming.
Security Interfaces	Endpoint protection via CrowdStrike Falcon , cloud security by Zscaler/Prisma Cloud , and Darktrace ActiveAI for anomaly detection.
Backup & Recovery	AWS S3 with Object Lock and immutable snapshots ensures no data loss during failures or ransomware events.

8. Non-Functional Requirements

8.1 System Performance

Req. ID	Description	Response Time / Target
---------	-------------	------------------------

PERF-1.1	Time for user navigation between standard screens (e.g., from Dashboard to Reports).	≤ 1 second
PERF-1.2	Time for the system to process and display new session data or health summaries.	≤ 3 seconds
PERF-1.3	Time for AI-based analysis and cloud update after session completion.	≤ 7 seconds
PERF-1.4	Time for synchronization between mobile device and cloud servers.	≤ 2 seconds for edge-to-cloud updates; ≤ 24 hours for full data backups
PERF-1.5	Real-time heart rate and SpO ₂ updates displayed on mobile.	≤ 0.5 seconds (500ms) latency

8.2 Information Security

Req. ID	Description
SEC-1.1	All sensitive data (user credentials, health data) must be encrypted in transit (TLS 1.3) and at rest (AES-256).
SEC-1.2	PHI must only be stored in HIPAA-eligible cloud environments (AWS RDS, EC2, S3) that have passed full testing.
SEC-1.3	The system shall comply with HIPAA, GDPR, and UCL Information Security Policies equivalents.
SEC-1.4	Use OAuth 2.0 / OpenID Connect for authentication with RBAC and token rotation every 24 hours.



SEC-1.5	Implement Darktrace ActiveAI for intrusion detection and ManageEngine DLP+ for data-leak prevention.
SEC-1.6	Data anonymization and “right-to-be-forgotten” features for users per GDPR Article 17.

8.3 Availability

Req. ID	Description
AVA-1.1	System uptime must be at least 99.5% excluding planned maintenance.
AVA-1.2	Planned maintenance shall not exceed 4 hours per month and occur during off-peak hours.
AVA-1.3	The system shall automatically recover from transient server failures within 30 seconds via AWS Auto-Scaling.
AVA-1.4	Redundant servers (multi-region AWS deployment) must ensure data accessibility even during outages.

8.4 Capacity

Req. ID	Description
CAP-1.1	System must handle 100,000 patient records in year one.
CAP-1.2	System must handle 250,000 patient records in year two.

CAP-1.3	System must handle 500,000+ patient records in year three.
CAP-2.1	Support at least 10,000 concurrent workout monitoring sessions in year one.
CAP-2.2	Support at least 50,000 concurrent users by year two.
CAP-2.3	Support at least 100,000 concurrent users by year three.
CAP-3.1	Cloud storage shall scale automatically through AWS S3 lifecycle management and database sharding.

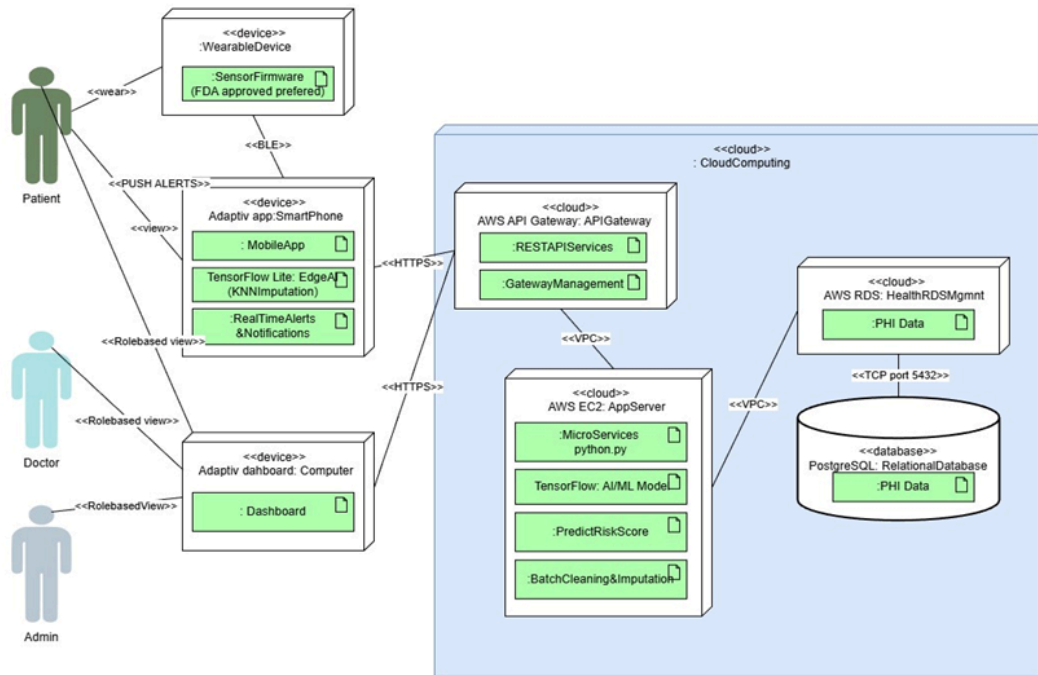
Section 2: Overview

Adaptive health is an integrated progressive health monitoring platform for cardiovascular patients to improve their cardiovascular health by detecting their health metrics using wearable smart devices and assigning them personalized health plans using AI based on cardiovascular conditions determined by giving a risk score. This is not a diagnosis system but more of a support system. It uses smart devices to collect physiological data that is processed in edge AI for risk alerts and Advanced cloud analytics for training data and updating health progress. Physicians and admins have role-based access to the patient monitoring dashboard for patient health analysis. All data is end-to-end encrypted and meets privacy protocol standards for HIPAA.

Report 3: System Design Document

Submitted: Trimester 1, Autumn 2025 | Team Pulse | CSIT321

High-Level Architecture



The architecture consists of Four main components:

1. *Wearable smart devices* with FDA approved sensor firmware are required to collect physiological health data.
2. *Smartphone Application:* We will use flutter to build the app. It will have Edge AI designed using TensorFlow Lite. It does imputation and executes real-time risk alerts.
3. *Dashboard Client:* Build with React. Role-based computer view provides admin or clinician views of patient data, alerts, and trends.
4. *Cloud Platforms:*
 - a) API Gateway: AWS API Gateway Manages all REST API requests, traffic, and logging.
 - b) AWS EC2 cloud services: Hosts backend Python microservices, machine learning models, risk scoring model, and data batch cleaning along with imputation.

- c) AWS RDS PostgreSQL: Stores Protected personal Health Information and provides complex relational queries for clinical use and handles thousands of users at the same time.

3.1 Main Architecture Diagram Description

- Data flow: Wearable smart devices gather data and send to smartphone Adaptive apps using BLE for immediate AI analysis using edge AI. The app transmits critical health data as payloads between regular intervals to the cloud via HTTPS.
- Cloud flow: API Gateway acts as the secure entry point for all external and internal requests. Backend AWS EC2 servers process data and act as the business layer of our system and cleans data, runs ML models, and predicts risk scores. AWS RDS PostgreSQL is part of the data layer that stores all the data of the users. Communication between these cloud services occurs through AWS' virtual private cloud.
- Dashboard flow: Role-based view enabled dashboard connects to the API Gateway to view patient data, alerts, and trends, all via secure channels.

3.2 Subsystem Descriptions

- Sensor & Firmware Subsystem: App recommends FDA approved wearable device firmware to collect physiological metrics, validates data, and transmits securely via bluetooth to paired smartphones.
- Mobile Edge AI Subsystem: The app designed using Flutter for inter OS operability, with TensorFlow Lite for Edge AI, performs rapid imputation using KNNImputer, risk assessment, and instant alert notifications. IT reduces latency by avoiding cloud for quick analytics minimizing cloud dependency and provides privacy.
- Backend Microservices & ML Analytics: AWS EC2 runs Python based microservices for batch cleaning, executes ML models for deep personalized analytics of user data it then forwards personalized health care regimens to user, app, and responds to dashboard requests.
- Storage & Archives Subsystem: AWS RDS (PostgreSQL): PostgreSQL is used for storage of large numbers of users and performing complex queries. AWS RDS manages PostgreSQL and encrypts data with AES 256 for HIPAA compliance.
- Dashboard Subsystem: Dashboard designed using React for cross-platform compatibility and security provides role-based access, data visualization, alert history, and reporting. All Communicated channels are only via HTTPS and secure ports for compliance.

3.3 Reducing design costs through trade-offs and design rationale.

The architecture of the system was created with the balance of reliability, latency, scalability, and user safety, which align with the learning outcomes under the category of systems design and evaluation (LO5, LO6) with the CSIT321 course.

3.3.1 Along device vs Cloud Processing.

Rationale:

The limits can consist of mobile hardware, battery efficiency, and model complexity versus on-device ML, which comes with reduced latency.

Trade-off:

Alternative to select approach: Hybrid model occurred, which is lightweight preprocessing on device and full inference on cloud.

Rationale: BEnables live feedback and allows higher computationally intensive cardiovascular risk models.

This section describes the communication protocols that underpin the application: BLE, REST, and WebSockets (Bigdata) 3.3.2.

Rationale:

BLE is the best to be used in wearable - phone communication because it is energy efficient and is widely used.

REST can be trusted with transactional information, and WebSockets can be trusted with real-time streams on the dashboard of the doctor.

Trade-off:

Weightedness REST will not ensure low-latency dashboards - WebSockets selected.

Environment - local buffering applied may influence stability in BLE.

3.3.3 Database Choice (NoSQL vs SQL)

Rationale:

Physiological data is of high-frequency and time series, but Adaptiv health performs aggregation within Edge AI and sends hourly summaries and key events in payloads to the cloud. All the data is better stored in a structured relationship schema with clear relationships.

Trade-off: NoSQL can store raw, unstructured data from sensors well but this creates complexity.

Selected: A single SQL database with JSONB support can store PHI and user profiles in relational tables while flexible preferences can live in JSONB columns.

3.3.4 AI Model Selection

Rationale:

The system requires fast analysis in cases of emergency, so it requires an Edge AI as well as a cloud AI for advanced analytics.

Trade-off:

Edge AI can't be a deep learning model as it can increase computational cost, latency and battery usage of smartphones. Simple models have the risk of missing subtle cardiovascular abnormalities or may produce false positives.

Selection: Edge safety alerts use lightweight TensorFlow Lite Edge AI model while deeper LSTM models on AWS EC2 can do advanced analytics and trends.

3.3.5 Security Architecture

Rationale:

To support the HIPAA/GDPR requirements, health data needs a high encryption level, RBAC, and path controls to access.

Trade-off:

Strong encryption raises overhead processing.

Selected: TLS 1.3 during data transit and AES-256 during the storage process, as well as role-based dashboard access.

3.3.6 Scalability vs Cost

Rationale:

Cloud autoscaling is vital to managing the changing loads, particularly in the high monitoring seasons.

Trade-off:

The cost of operation is higher with autoscaling.

Solution: introduction of work load threshold and batching of non critical tasks.

3.3.7 MVP vs Extended Features

Rationale:

This project would need to be within the CSIT321 timeframe (according to subject expectations) to be feasible.

Trade-off:

PPG based cardiovascular analysis is complicated - provided as an addition in the future.

Minimal viable product focus: The parts of the MVP are real time heart rate measurements, anomaly detection, and doctors dashboard.

Section 4: Data and Logic Design

The Adaptiv Health system's data processing and structure are explained in this section. It describes how data moves through the system to facilitate monitoring, analysis, and decision-making, as well as the fundamental data objects used to store cardiovascular data.

4.1 Data Dictionary

- The Data Dictionary defines the main data entities used in the Adaptiv Health system and explains their structure, attributes, and purpose. These data objects support real-time monitoring, AI-based analysis, and secure storage of cardiovascular information.

User

- UserID: Each user's unique identifier
- Name: The whole name of the user
- Age: The user's age
- Gender: An indicator of gender
- Role: Administrator, Caregiver, Clinician, and Patient
- Medical History: Risk factors and previous cardiovascular diseases
- CreatedAt : Date of account creation

Description:

Keeps track of each system user's personal and role-related data.

Vital Sign Record

- RecordID: A distinct number for every reading
- UserID: The user's reference
- HeartRate: Beats per minute (BPM)
- SpO₂: Blood oxygen saturation level
- BloodPressure: Systolic and diastolic values
- Timestamp: The moment the information was recorded
- SourceDevice: A wearable gadget

Description:

Physiological measures obtained from wearable sensors are stored for study and monitoring.

Activity Session

- SessionID: An activity session's unique identifier
- UserID: The user's reference
- StartTime: The activity's start time
- EndTime: The activity's end time
- ActivityType: The kind of activity carried out, such as cycling or walking
- AverageHeartRate: The session's average heart rate
- PeakHeartRate: The highest heart rate ever measured
- RecoveryTime: The amount of time it takes for heart rate to return to normal

Description:

Shows a performance summary for a single workout or activity session.

Risk Assessment

- AssessmentID: The assessment's unique identifier
- UserID: The user's reference
- RiskLevel: Classified cardiovascular risk level (Low, Moderate, High)
- RiskScore: Cardiovascular risk score computed
- AssessmentDate: The time and date the evaluation was created

Description:

Stores AI-generated evaluations of a user's cardiovascular risk based on recent and historical data.

Exercise Recommendation

- RecommendationID: The recommendation's unique identifier
- UserID: The user's reference
- Suggested Activity: A suggested kind of activity
- IntensityLevel: The suggested level of intensity
- Duration: Recommended time for the activity
- ValidPeriod: The duration of the recommendation's applicability

Description:

Based on cardiovascular state, the AI system generates individualized workout recommendations.

Alert

- AlertID: The alert's unique identifier
- UserID: The user's reference
- AlertType: Alert type (e.g., low oxygen saturation or elevated heart rate)
- SeverityLevel: Level of alertness
- AlertMessage: A message that the user sees
- AlertTime: The moment the alert was set off

Description:

Records emergency and warning notifications created when irregular health measurements are observed.

4.2 Data Flow Description

The Adaptiv health system follows a clear and structured data flow to ensure accurate analysis and timely feedback.

1. The Adaptiv Health system adheres to a defined and organized data flow to guarantee precise analysis and prompt feedback.
2. Wearable gadgets consistently gather cardiovascular information like heart rate, oxygen levels, and blood pressure.

3. Information is sent to the mobile app through Bluetooth, where it undergoes validation and cleaning.
4. The cloud securely stores the processed data for long-term access.
5. AI models evaluate the data to assess risk levels and establish safe activity thresholds.
6. From the analysis, the system produces workout suggestions and safety notifications.
7. Users receive feedback in real time via the mobile app and dashboards.
8. Continuous integration of new data enhances future analyses and personalizations

Section 5: System Component Algorithms and Pseudocode

Edge AI which is a real-time Signal Processing and also Risk Detection runs directly on the smartphone using TensorFlow Lite, which ensures <160ms Latency. Signal Enhancement Algorithm applies filtering to remove noise and irregularities in hear-rate and SpO2 signals.

HR_SignalEnhancement()

Purpose: Smooth incoming sensor readings before analysis.

Type: Real-time signal conditioning.

```
FUNCTION HR_SignalEnhancement(raw_signal):  
    SET window_size = 5  
    INITIALIZE smooth_signal = []  
  
    FOR i FROM 0 TO LENGTH(raw_signal):  
        window = raw_signal[max(0,i-window_size) : i]  
        smoothed_value = AVERAGE(window)  
        APPEND smoothed_value TO smooth_signal  
  
    RETURN smooth_signal
```

Segmentation Algorithm

Before we calculate the risk, the system will detect if the user is either resting or active or Recovering.

```
FUNCTION SegmentActivityPhase(current_HR, baseline_HR):  
  
    IF current_HR < baseline_HR + 10:  
        RETURN "RESTING"  
  
    IF current_HR >= baseline_HR + 10 AND current_HR <= baseline_HR + 45:  
        RETURN "ACTIVE"  
  
    IF current_HR > baseline_HR + 45:  
        RETURN "HIGH_INTENSITY"
```



```
IF current_HR DROPPING FOR 30 seconds:  
    RETURN "RECOVERY"
```

```
END FUNCTION
```

This part of the segment is important and essential for the adaptive recommendations and AI risk scoring.

Scoring Algorithm

This will determine if the user is in safe, caution or the danger zone.

ComputeRiskScore() Algorithm:

```
FUNCTION ComputeRiskScore(heart_rate, spo2, hrv, age, baseline_HR):
```

```
    input_vector = NORMALIZE([heart_rate, spo2, hrv, age, baseline_HR])
```

```
    risk_score = TFLite_Model.PREDICT(input_vector) # Value 0 → 1
```

```
    IF risk_score >= 0.80:
```

```
        RETURN ("HIGH", risk_score)
```

```
    ELSE IF risk_score >= 0.50:
```

```
        RETURN ("MODERATE", risk_score)
```

```
    ELSE:
```

```
        RETURN ("LOW", risk_score)
```

Scoring of 0.8 - 1.0 would be considered dangerous and will trigger an emergency alert.

Scoring of 0.5 - 0.79 would be caution which would recommend slower pace or to rest.

Scoring 0.0 - 0.49 would be in the safe zone and would just let you continue your normal activity.

Method Description

Adaptiv health system would rely on server core processing methods which will convert raw wearable data into very meaningful insights for the risk analysis, real time alerts and personalized recommendations. It would ensure that the data can be cleaned and normalized and interpreted by the AI and transformed into a safe and also adaptive guidance for users.

1. Normalize():

It would transform the raw inputs (HR, SpO2, HRV, etc) into a standard 0 - 1 scale so it can prepare for the machine learning, since AI models require consistent input ranges just to avoid the bias or misinterpretations.

2. Impute():

Impute would fill the small gaps within the sensor readings using linear interpolation and KNN when the signal is lost briefly. It would make sure there is always a clean,

continuous data stream ready for analysis.

3. PredictRisk():
PredictRisk would use the TensorFlow lite model on the smartphones to calculate the cardiovascular risk score which is between the 0 - 1 and that would classify the risk as Low Moderate or High without relying on the cloud, it would give an output of whether the user is safe should slow down or is in danger
4. GenerateRecommendation():
This would create a daily or even a session based workout suggestion or lifestyle suggestion. This could happen since it adapts exercises intensity to the user's current condition and recovery.
5. TriggerAlert()
This would activate immediate notifications if dangerous measurements are detected like a HR spike or low SpO2. it would be a screen alert and also a vibration warning, and would alert the caregiver with a notification.

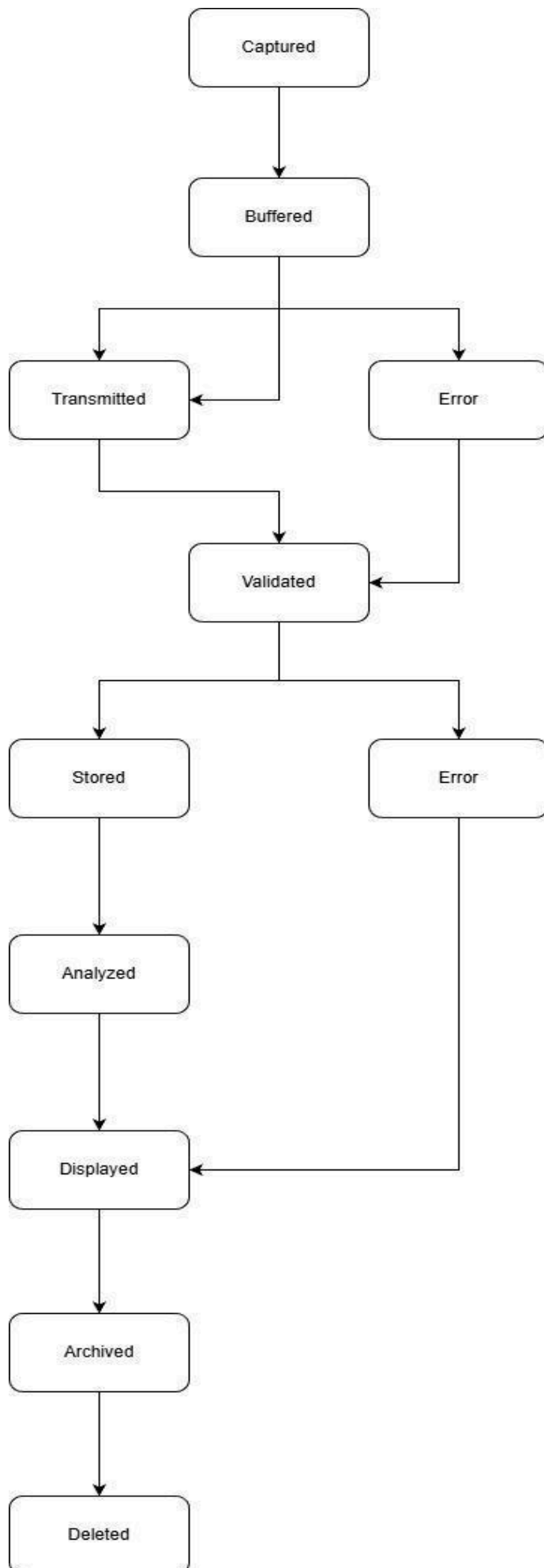
The techniques outlined in Section 5 are interconnected in a cyclic process to make sure that Adaptiv Health is able to process sensor data in real-time and give real-time cardiovascular safety feedback. The first step of integration is when the wearable device sends raw physiological signals. The readings are then filtered with HR_SignalEnhancement which will remove the noise and would leave the clean usable data. Following it the clean data readings would be passed through impute to correct any missing or corrupt values that might arise as the sensor dropouts.

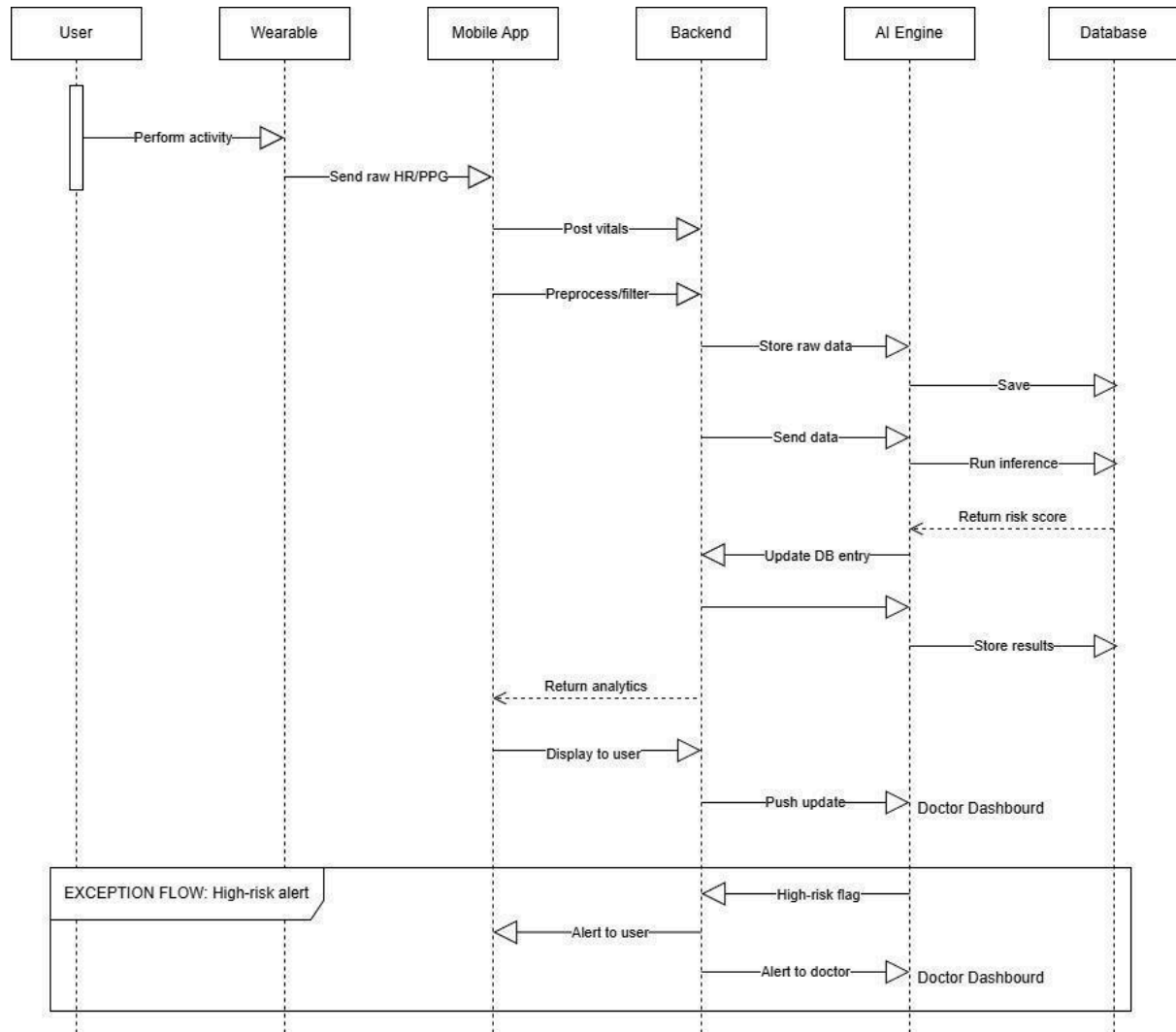
After cleaning the data, the system uses the Normalize() to use the values in a normalized format that can be processed by machine-learning. The normalized vector is then presented to the TensorFlow Lite model as a call to PredictRisk(), which will produce a risk score and the types of risk the user can be categorized as; LOW, MODERATE, or HIGH. According to this classification there are two parallel processes that can take place:

In case the user is either safe or at a medium level of strain, GenerateRecommendation() gives the correct advice concerning the level of activity, how to rest or how intense exercise should be.

When the user goes to a risky state, TriggerAlert will send an instant on-device notification and send a cloud notification when connected.

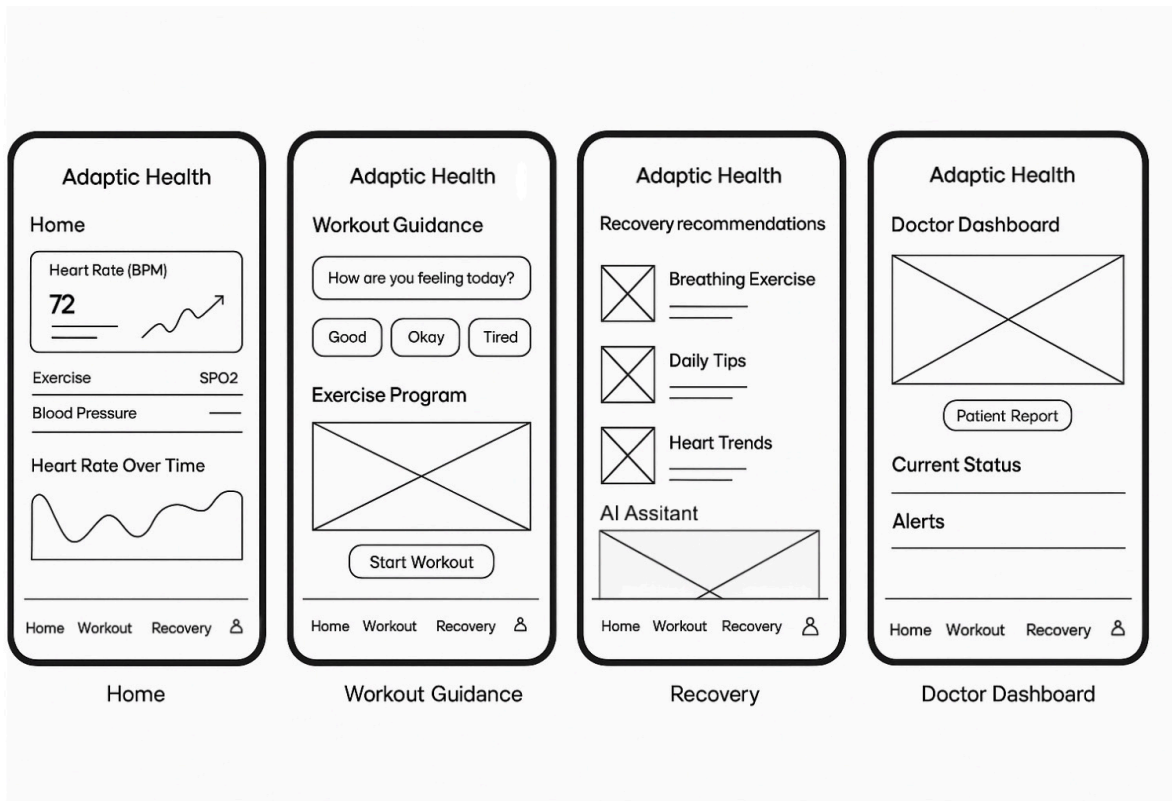
This pipeline would mean that the raw wearable data can now be converted into actionable and personalized insights in seconds, without the endangering of the user and also promotes adaptive cardiovascular training.





Sequence diagram (full pipeline)

Wireframes:



User workflow:

1. General User Flow (Home → Workout → Recovery)

1. The user opens the app and lands on the Home screen.
2. The user views key vitals such as:
 - Heart Rate (BPM)
 - Blood Pressure
 - SPO2
 - Heart Rate Over Time graph
3. The user taps Workout from the bottom navigation bar.
4. On the Workout Guidance screen:
 - The user answers the wellness check ("How are you feeling today?").
 - The system adjusts workout difficulty accordingly.
 - The user views the "Exercise Program".
5. The user taps Start Workout to begin the guided workout session.
6. After completing the workout, the user taps Recovery in the bottom navigation bar.
7. On the Recovery screen, the user reads:
 - Breathing exercises
 - Daily tips
 - Heart trend lines
8. The system uses the recovery data to adjust tomorrow's workout.

2. Doctor Workflow

1. The doctor opens the Doctor Dashboard.
2. The doctor views the patient's health overview graph.
3. The doctor taps the Patient Report to access the detailed view.
4. The doctor reads the patient's Current Status section.
5. The doctor reviews Alerts for any abnormalities or irregular heart activity.
6. The doctor uses trends and alerts to make recommendations or follow-up decisions.

3. Workout Guidance Workflow

1. The user navigates to Workout.
2. The system prompts: "How are you feeling today?"
3. The user selects Good, Okay, or Tired.
4. The system adjusts workout intensity in real time.
5. The user reviews the Exercise Program block.
6. The user taps Start Workout.
7. System logs workout activity and physiological responses.

4. Recovery Workflow

1. The user opens the Recovery screen.
2. The system displays:
 - Breathing exercises
 - Daily wellness tips
 - Heart Trends (HR recovery graph)
3. The user reads recommendations and follows any suggested exercises.
4. Insights from the recovery screen feed directly into the next day's workout plan.

Requirements Traceability Matrix:

Requirement ID	Requirement Description	SDD Component/ Implementation	Wireframe Screen
REQ-1.1	Display real-time heart rate	HR Display Module	Home
REQ-1.2	Show historical heart-rate trend	Cardio Trend Graph	Home

REQ-1.3	Provide navigation between modules	Bottom Navigation Bar	All Screens
REQ-1.4	Collect user wellness input	Daily Check-in Component	Workout Guidance
REQ-1.5	Generate personalized workout plans	Exercise Program Module	Workout Guidance
REQ-1.6	Start and track workout sessions	Workout Session	Workout Guidance
REQ-1.7	Display recovery recommendations	Recovery Recommendation Engine	Recovery
REQ-1.8	Provide breathing exercises	Breathing Exercise Module	Recovery
REQ-1.9	Display heart recovery trends	Heart Trends Graph	Recovery
REQ-1.10	Allow the doctor to view patient data	Doctor Dashboard Module	Doctor Dashboard
REQ-1.11	Display alerts (abnormal data)	Alerts Component	Doctor Dashboard
REQ-1.12	Display patient status summary	Status Summary Module	Doctor Dashboard

Report 4: Test Plan

Submitted: Trimester 2, Winter 2026 | Team Pulse | CSIT321

1. Functionality Testing

1.1 Test ID and Title

Test ID	Test Title
AH-FT-001	Adaptiv Health Functionality Testing Plan

1.2 Introduction and Objective

Introduction

Adaptiv Health functionality testing will ensure that all functions of the system are operating properly per the Software Requirements Specification (SRS). The purpose of this testing is to ensure that the Flutter mobile application (patient side) and the React web dashboard (doctor side) work as intended when used by users. The testing includes the normal user interactions including completion of onboarding forms, clicking of buttons, movement between screens, and access to health and patient data.

This testing also ensures correct communication between the frontend and backend API. For example, when a patient submits registration details on the mobile application, the data must be successfully sent to the backend and stored correctly. Error handling is also tested to ensure that appropriate messages are shown when users enter invalid input (e.g., invalid email format or missing required fields).

Objectives

- Make sure that all 5 onboarding screens capture and pass on user data properly.
- Ensure that the mobile application flows screen to screen.
- Make sure that the doctor dashboard shows the correct data of patients on the database.
- Confirm form validation (email, password, mandatory fields)
- Establish that the API requests of mobile application and dashboard give appropriate responses.
- Make sure error messages are displayed correctly in case of invalid input.

- Check The exercise session upload places all fields required into the backend.
- Ensure that the risk score and level are displayed correctly on the ML prediction demo on the dashboard.

1.3 Scope of Testing

Adaptiv Health mobile application functionality testing involves all patient facing functionality that is a priority to the system workflow. This is the entire five onboarding screens welcome, basic user details, medical history (heart conditions and beta blockers), medical record upload and wearable device set up. There should be the input fields to be checked on each screen, and the movement between the screens with Next and Back buttons should work without any complications. The last submission has to submit the appropriate JSON payload to the API of the backend.

The testing of the React dashboard covers all functionality that doctors are exposed to. It involves loading and presenting the patient list on the database, the information of a particular patient like workout history and vital readings, and the ML prediction demo page. The ML demo page should be able to receive user inputs and execute the appropriate endpoint of the API and all charts on the dashboard should have the correct data.

The following items are not included in the scope of functionality testing:

- Bluetooth connection to wearable devices
- ML model accuracy and prediction performance
- Security of API calls and authentication mechanisms
- Direct database operations and database performance

1.4 Test Objectives and Approach

Objectives:

- Make sure that everything that a user enters into a mobile app triggers a correct response.
- Make sure that all 5 onboarding screens have necessary fields.
- Test API integration is the correct data format.
- Check the dashboard loads patient information properly.
- Confirm the risk scores with risk color coding (red/ yellow/ green).
- Check navigation dashboard pages do not have errors.

Approach

Function	Description	Test Approach
----------	-------------	---------------

User Registration	Collects name, email, password	Submit valid/invalid inputs, verify API response
Heart Health Conditions	Select multiple conditions	Select multiple options, verify stored correctly
Medical Records Upload	Upload patient files	Upload sample file, verify backend receives file
Device Pairing	Select wearable device	Select device, verify next step triggered
Workout Upload	Sends session data	Submit workout, verify backend receives JSON
Patient List Display	Shows all patients on dashboard	Verify displayed list matches backend data
Patient Detail Page	Displays patient history and vitals	Select patient, verify correct data loads
ML Prediction Demo	Returns risk score	Submit values, verify risk score returned
Form Validation	Input format checks	Enter invalid values, verify error messages

1.5 Test Schedule and Milestones

Phase	Weeks	Milestone
Test Case Planning	Week 1	All test cases written and reviewed
Onboarding Screens Testing	Week 2	All 5 screens validated for correct inputs and navigation
Dashboard Patient List Testing	Week 2-3	Patient list verified with real database data
Dashboard Detail Page Testing	Week 3	Patient detail page and charts verified
API Integration Testing	Week 3-4	Mobile and dashboard API calls validated

ML Demo Page Testing	Week 4	Prediction demo tested with multiple inputs
Regression Testing	Week 1-5	Re-test after every code change

1.6 Test Environment Setup

Hardware:

Component	Specification
System	Laptop or Desktop
RAM	Minimum 8GB
Processor	Intel Core i5 or Apple M1
Mobile Device	Android phone or emulator (Android 10+)

Software:

Component	Specification
Operating System	Windows 10/11 and macOS
Mobile Framework	Flutter SDK 3.0+
Frontend	React 18+, Node.js 18+
Browser	Chrome, Safari, Edge
Development Tools	VS Code, Android Studio
API Testing Tool	Postman

Network:

Component	Specification
Connection Type	Stable WiFi

Minimum Speed	10 Mbps
Network Security	HTTPS protocol
Server	Backend API running locally or on AWS EC2

1.7 Resources and Responsibilities

Role	Responsibility
Ghina (Flutter Developer)	Tests all 5 onboarding screens, validates form inputs, tests navigation flow, verifies API integration from mobile app
Karylle (UI/UX Designer - Figma)	Gives UI flow responses out of Figma, and ensures that the functional screens are per the planned user experience.
Abdullah (Frontend Developer)	Tests user functionality like patient list load, patient detail, charts, ML demonstration page and navigation.
Bithin/Hamza (Backend Support)	Provide running API for frontend testing, confirm correct responses are returned
Omar (Coordinator)	Reviews test results, documents bugs, coordinates fixes

1.8 Test Deliverables

Phase	Deliverable
Onboarding Testing	All 5 screens tested with screenshots showing pass/fail results
Dashboard Testing	Patient list, detail page, and ML demo tested with evidence
API Integration	Proof that mobile app sends correct JSON to backend (Postman logs or app logs)
Bug Report	List of all functional bugs found with severity (Critical/High/Medium/Low)
Test Results	Completed test case result table

1.9 Entry and Exit Criteria

Entry Criteria:

- Flutter app builds without errors on Android emulator
- React dashboard runs without errors on localhost:3000
- Backend API is running and accessible
- Test user accounts exist in database
- All test cases are written and ready

Exit Criteria:

- All onboarding screens accept valid input and reject invalid input correctly
- Dashboard displays patient data matching database
- All API calls return correct responses
- No critical bugs remaining open
- All test results documented

1.10 Risks and Mitigation Strategies

Risk	Mitigation Strategy
API downtime	Use mock data for UI testing
Mobile crashes on some devices	Test on multiple emulator versions
Form validation bugs	Define validation rules before testing
ML demo endpoint unavailable	Mock API responses during demo testing
Navigation breaks after code changes	Run regression tests after each update

2. Database Testing

2.1 Test ID and Title

Test ID | Test Title AH-DB-001 | Adaptiv Health Database Testing Plan

2.2 Introduction and Objective

Adaptiv Health uses an AWS RDS PostgreSQL database to securely store and manage all patient and system data. Database testing is conducted to validate that the backend database correctly stores, retrieves, updates, and maintains relationships between critical healthcare data entities.

The system consists of three primary tables:

- Users – Stores 973 registered cardiac patients including demographic data, medical conditions, baseline heart rate (baseline_hr), and maximum safe heart rate (max_safe_hr).
- Activity_sessions – Stores 973 recorded workout sessions including heart rate data and AI-generated risk_score values.
- Safety_alerts – Stores 432 alerts automatically generated by the ML model when a patient's risk_score exceeds 0.85.

Because Adaptiv Health is a healthcare monitoring system, data accuracy is critical. For example, if a patient's max_safe_hr is stored incorrectly, the machine learning model may generate inaccurate risk predictions, potentially leading to unsafe recommendations.

Additionally, the system must maintain data integrity during concurrent access, such as when multiple patients upload workout sessions simultaneously. Database testing ensures that relationships between tables remain valid and no data corruption occurs under load.

Objective

The objectives of database testing are to:

- Validate all CRUD operations (Create, Read, Update, Delete) across all three tables
- Ensure foreign key relationships function correctly (sessions must link to valid users)
- Verify that data types match the schema (e.g., age as INT, risk_score as DECIMAL)
- Confirm database query performance remains under 2 seconds for 973+ records
- Validate that duplicate emails are rejected using a unique constraint
- Ensure safety_alerts are automatically created when risk_score > 0.85
- Verify data persistence after AWS RDS server restart
- Confirm encryption mechanisms protect PHI (Protected Health Information)

2.3 Scope of Testing

Tables and Operations

Database testing focuses on the three core tables: users, activity_sessions, and safety_alerts.

The users table stores patient profiles including age, medical history, max_safe_hr, and baseline_hr. Testing verifies that patient data is correctly inserted during registration and retrieved accurately for ML calculations. The activity_sessions table stores workout data including heart rate measurements and ML generated risk_score values. Testing ensures that sessions are stored correctly and risk scores maintain precision. The safety_alerts table stores alerts triggered when risk_score

exceeds 0.85. Testing verifies that high-risk sessions generate corresponding alerts automatically.

Relationships

Relationships between tables are critical for maintaining healthcare data integrity:

- Every activity_session must reference a valid user_id.
- Every safety_alert must reference a valid user_id.
- Foreign key constraints must prevent orphaned data.

Testing verifies that:

- Sessions cannot be created for non-existent users.
- Alerts are always linked to real patients.
- Deleting a user properly handles related sessions and alerts (cascade or restriction rules enforced correctly).

Out of scope:

The following components are not covered in this database testing plan:

- The ML model logic (covered in ML Testing)
- API endpoint validation (covered in Security Testing)
- Frontend UI rendering and display (covered in Functionality Testing)

2.4 Test Objectives and Approach

Test Objectives:

- Test INSERT operations for all tables
- Test SELECT queries return accurate data
- Test UPDATE operations (e.g., doctor updates max_safe_hr)
- Test DELETE operations and cascade behavior
- Test constraint violations (duplicate email, null required fields)
- Test JOIN queries combining users and sessions
- Test performance with 973+ records
- Test automatic alert creation when risk_score > 0.85

Approach table

Function	Description	Test approach
Insert user	Register new cardiac patient	Insert valid user, verify auto-generated user_id
Duplicate email check	Prevent duplicate	Insert same email twice,

	accounts	verify second insert fails
INSERT session	Store workout with risk_score	Insert session for valid user_id, verify all fields saved
Foreign key constraint	Ensure valid user performance	Insert session with fake user_id (e.g., 99999), verify error
High-risk alert creation	Auto-create alert	Insert session with risk_score=0.92, verify alert row created
JOIN query	Retrieve patient dashboard data	Run JOIN between users and sessions, verify correct linkage
Query performance	Ensure fast retrieval	Run select queries and measure execution time

2.5 Test Schedule and Milestone

Phase	Weeks	Milestone
Test Setup	Week 1	Database connected, test queries prepared
CRUD Testing	Week 2	All INSERT/SELECT/UPDATE/DELETE tested
Constraint Testing	Week 2-3	Foreign keys, unique constraints, NOT NULL validated
Relationship Testing	Week 3	JOIN queries verified with correct data
Performance Testing	Week 3-4	Query speed tested with 973+ records
Concurrent Access Testing	Week 4	Multiple simultaneous writes tested

2.6 Test Environment Setup

Hardware

Component	Specification
System	Laptop or desktop
Ram	Minimum 8GB
Processor	Intel Core i5 or equivalent

Software

Component	Specification
Operating System	Windows 10/11 or macOS
Database	PostgreSQL 14+ on AWS RDS
Database Tool	pgAdmin 4
Query Tool	pgAdmin SQL Editor/ psql
Development Tools	Vs code

Network Configuration

Component	Specification
Connection	Stable wifi
Database Host	adaptiv-health-db.c3kc22cw4z0f.eu-north-1.rds.amazonaws.com
Port	5432
Access	Whitelisted IP addresses only

2.7 Resources and Responsibilities

Role	Responsibility
Hamza	Execute SQL test scripts, validate CRUD, constraints, foreign keys
Abdalla	Validates ML-related data (risk_score calculations, feature data types), checks

	alert creation logic, verifies data integrity for ML inputs
Bithin (support)	Provide backend logic for cross-checking expected behavior
Omar (support)	Verify ML model receives correct data format.

2.8 Test Deliverables

Phase	Deliverable
CRUD Testing	SQL scripts with successful execution results
Constraint Testing	Evidence of rejected invalid data
Relationship Testing	JOIN query results showing correct linkage
Performance Report	Query timing results under 2 seconds
Data Integrity Report	Confirmation of 973 users, 973 sessions, 432 alerts stored correctly

2.9 Entry and Exit Criteria

Entry Criteria:

- AWS RDS database is running and accessible
- All 3 tables created with correct schema
- 973 patients and 973 sessions already loaded for testing
- pgAdmin connected to database
- Test SQL scripts prepared

Exit Criteria:

- All CRUD operations pass without errors
- All constraint violations correctly rejected
- All JOIN queries return accurate data
- Query performance under 2 seconds
- No data loss or corruption found

- All results documented

2.10 Risks and Mitigation Strategies

Risk	Mitigation strategy
AWS RDS becomes inaccessible during testing	Have a local PostgreSQL backup database with same schema and data
Test data corrupts the real patient data	Use a separate test schema or test database, never test on production data
Concurrent writes cause data conflicts	Test concurrent access with controlled scenarios, add transaction handling
Slow queries with large data	Add database indexes on user_id, session_id, created_at columns
Deleting test users removes real patient data	Keep test users with clearly marked names like "TEST_USER_001"

3. UX/UI Testing

3.1 Test ID and Title

Test ID | Test Title
AH-UX-001 | Adaptiv Health Functionality Testing Plan

3.2 Introduction and Objective

Introduction

Adaptiv Health UX/UI testing validates the **visual design, usability, and user experience** of both the **Flutter mobile app (patient side)** and the **React dashboard (clinician/admin side)**. This testing is especially important because Adaptiv Health is a **healthcare monitoring support system**, and confusing UI can cause critical mistakes. For example, a patient might not understand an emergency alert during exercise, or a clinician may misread a risk chart and overlook a dangerous trend.

The testing ensures the system is visually consistent across screens, including **fonts, spacing, buttons, and navigation**, while also confirming responsive design across multiple device sizes. Accessibility-related checks are also included at a basic level, such as readability and color clarity.

A key UI feature in Adaptiv Health is the **risk-level indicator**, which must be immediately understandable:

- **Green = Safe**
- **Yellow = Moderate/Caution**
- **Red = High/Danger**

This is important for both patients and doctors because the system relies on real-time edge AI risk scoring (<160ms alerts) and clear safety communication.

Emergency alert UI is tested in detail. When a user enters a dangerous state (risk score ≥ 0.80), the alert must be **unmissable**, using strong visual design such as **bold text, red emphasis, and vibration warning**, ensuring the patient stops unsafe activity immediately.

Objective

- Verify the Adaptiv Health **red-and-white branding** is consistent across the Flutter mobile app and React dashboard (colors, fonts, spacing, icons).
- Confirm the risk status indicator is always clear and correctly mapped:
Green = Safe (LOW risk)
Yellow = Caution (MODERATE risk)
Red = Dangerous (HIGH risk)
- Ensure all critical health values are **easy to read instantly**, including during workout conditions.
- Validate typography standards:
Body text minimum: 14pt
Critical values minimum: 24pt
- Verify the emergency alert UI is **impossible to miss**, using:
Strong red warning design
Bold text
Clear action message (Stop/Rest)
Vibration notification
- Confirm navigation is intuitive and consistent between modules:
Home → Workout → Recovery
Dashboard Patient List → Patient Report
- Verify charts and graphs are clinically readable:
- Proper labels, axes, legends, and units (BPM, %, mmHg, risk score 0–1)
- Validate onboarding clarity by confirming users can complete the **5-step onboarding flow** without confusion or missing required fields.
- Verify responsive layout works properly on:
Mobile (375px)

Tablet (768px)**Desktop (1920px)**

- Confirm all buttons and interactive elements have clear labels and consistent placement (no vague buttons like “OK” or “Submit”).

3.3 Scope of Testing

Mobile App UI: UX/UI testing will cover all key screens in the Adaptiv Health Flutter mobile app, focusing strongly on the 5-step onboarding flow. Each onboarding screen will be tested to confirm the layout is clean, fields are clearly labeled, spacing is consistent, and the “Next” button is always obvious and easy to tap. The Home and Workout screens will also be tested to ensure critical vitals such as heart rate (BPM), SpO₂, and blood pressure are displayed clearly and large enough to read during exercise. The risk indicator (LOW/MODERATE/HIGH) must be immediately understandable using the correct green/yellow/red color coding. Emergency alert UI is included in scope and will be tested to ensure it is visually alarming and impossible to miss during a dangerous event.

Dashboard UI: UX/UI testing will cover the main pages of the Adaptiv Health React clinician/admin dashboard. This includes the **patient list table**, ensuring column headers are clear, patient rows are readable, and the risk color coding is obvious at a glance. The **patient detail page** will be tested to confirm the risk trend chart is correctly displayed with proper labels, axes, and a legend (X-axis for time/date, Y-axis for risk score 0–1). In addition, the **ML prediction/analytics demo page** will be included in testing to ensure it is intuitive for a clinician to use without requiring extra instructions and that charts, risk values, and outputs are clearly presented.

Responsive and Cross-Browser: Testing will confirm that the clinician dashboard works correctly across different screen sizes because doctors may use **tablets in clinics**, not only desktops. The dashboard will be tested at **768px (tablet)** and **1920px (desktop)** to ensure layouts, tables, and charts do not break or become unreadable. The Flutter mobile app will also be tested across multiple Android screen sizes using emulator profiles. Finally, cross-browser testing will be performed for the dashboard to confirm consistent UI behavior and layout in **Chrome, Safari, and Edge**, ensuring charts and risk indicators display correctly across all supported browsers.

Out of Scope:

This UX/UI testing plan does not include full usability studies with real cardiovascular patients or real clinicians, since the system is being evaluated as a prototype and not deployed clinically. A complete accessibility compliance audit (such as full WCAG certification) is also out of scope, although basic checks like readable font sizes and clear color visibility will still be performed. In addition, detailed performance testing for animations, micro-interactions, and smooth transitions is not included—only basic

verification will be done to ensure animations do not break layouts or interfere with emergency alerts.

3.4 Test Objectives and Approach

Test Objectives:

- Verify all screens match the design prototypes (Figma reference)
- Confirm risk colors are consistent and correct across all screens
- Test navigation flow is logical and smooth
- Verify all text is readable and not too small
- Test responsive design breakpoints (mobile + dashboard)
- Confirm emergency alert is visually prominent
- Verify chart labels, legends, and data clarity

Test Approach Table:

Function	Description	Test Approach
Color Consistency	Red/white brand colors throughout	Check all screens use same color palette
Risk Color Coding	Red=HIGH, Yellow=MODERATE, Green=SAFE	Find a patient with each risk level, verify correct color
Alert Design	Emergency alert is unmissable	Trigger high-risk scenario, evaluate visual impact
Mobile Responsive	App looks correct on different phone sizes	Test on small (375px), medium (390px), large (430px) screens
Dashboard Responsive	Dashboard works on tablet and desktop	Resize browser to 768px (tablet), 1920px (desktop)
Chart Readability	Risk trend chart has clear labels	Verify X-axis (dates), Y-axis (0-1 risk), legend present
Font Readability	All text readable without zooming	Check minimum 14pt body text, 24pt for critical values

Button Clarity	Button labels describe action clearly	Review every button label for ambiguity
----------------	---------------------------------------	---

3.5 Test Schedule and Milestones

Phase	Weeks	Milestone
Initial Layout Review	Week 1	All screens reviewed against design prototypes
Mobile App UI Testing	Week 2	All 5 onboarding screens and home screen validated
Dashboard UI Testing	Week 2-3	Patient list, detail page, and ML demo reviewed
Responsive Testing	Week 3	Multiple screen sizes tested for both app and dashboard
Alert Design Testing	Week 3	Emergency alert visual tested for impact
Cross-Browser Testing	Week 4	Dashboard tested on Chrome, Safari, and Edge
Accessibility Review	Week 4	Color contrast and font sizes checked

3.6 Test Environment Setup

Hardware:

Component	Specification
System	Laptop or Desktop
RAM	Minimum 8GB
Mobile Device	Android phone AND emulator (for different screen sizes)
Multiple Devices	Phone, tablet (if available) for responsive testing

Software:

Component	Specification
Operating System	Windows 10/11 or macOS
Mobile	Flutter, Android Emulator (multiple screen sizes)
Browsers	Chrome 120+, Safari 17+, Edge 120+
Dev Tools	Chrome DevTools (for responsive testing)
Design Reference	Original Figma/prototype designs for comparison

3.7 Resources and Responsibilities

Role	Responsibility
Ghina	Tests all Flutter mobile app screens for visual consistency, readability, and correct layout on multiple screen sizes
Karylle	Tests all React dashboard pages for visual design, chart clarity, responsive design, and cross-browser compatibility
Both	Evaluate emergency alert visual design together - must agree it is clear and alarming enough
Omar (Reviewer)	Final review of all UI test results, approves design before presentation

3.8 Test Deliverables

Phase	Deliverable
Mobile UI Review	Screenshots of all mobile screens with annotations noting issues
Dashboard UI Review	Screenshots of all dashboard pages with annotations
Responsive Testing	Screenshots at 375px, 768px, and 1920px widths
Cross-Browser Testing	Screenshots of dashboard in Chrome, Safari, and Edge

Color Accessibility Report	Color contrast ratios for text/background combinations
Bug List	All visual issues found (wrong color, misaligned text, cut-off content) with severity

3.9 Entry and Exit Criteria

Entry Criteria:

- Mobile app builds and runs on Android emulator
- Dashboard runs in browser without errors
- Design prototypes available for comparison
- Multiple browser versions installed
- Android emulator profiles for different screen sizes configured

Exit Criteria:

- All screens match intended design (no major visual bugs)
- Risk colors are correct and consistent throughout
- Emergency alert is visually prominent on mobile
- Dashboard is readable on tablet and desktop
- No text cutoff or layout breaking at any tested screen size
- Charts are properly labeled
- All visual bugs documented and prioritized

3.10 Risks and Mitigation Strategies

Risk	Mitigation Strategy
App looks different on older Android versions	Test on Android 8, 10, 12 emulators, fix layout issues for each version
Dashboard breaks on Safari (known compatibility issues)	Test Safari early in process, fix CSS issues with vendor prefixes
Charts don't render on small screens	Set minimum width for charts, add horizontal scroll for small screens
Risk colors not visible to colorblind users	Add text labels alongside colors (don't rely on color alone)
Emergency alert not noticeable during exercise	Test in bright light conditions, ensure contrast ratio is very high

4. Security Testing

4.1 Test ID and Title

Test ID | Test Title AH-ST-001 | Security Testing Plan for AdaptivHealth.

4.2 Introduction and Objective

Introduction

AdaptivHealth processes highly sensitive Protected health information of the patients. The system architecture thus involves multiple security controls to ensure the safety of their personal data from various attacks and to ensure vulnerabilities are covered.

Because the platform targets cardiovascular patients and clinicians, compromise of confidentiality, integrity or availability directly impacts patient safety and legal compliance (HIPAA/GDPR/DHA). A dedicated security testing phase is therefore required to validate that implemented controls satisfy the security requirements defined in the SRS and reflected in the Design document.

Since the platform targets cardiovascular patients and clinicians, compromise of confidentiality, integrity or availability directly affects legal compliance and violation of various standards for different countries such as HIPAA and GDPR.

Security testing aims to:

1. Verify that authentication and authorization enforce OAuth2, MFA and role-based access control for Patients, Doctors and Admins.
2. Confirm that data in transit and data at rest are protected using cryptographic tools.
3. Validate that API Gateway, WAF and backend microservices resist common web and API vulnerabilities included in OWASP Top 10.
4. Ensure that PHI is accessible to authorized users only and that back-end services follow least-privilege policy.
5. Confirm that all events are logged and monitored for incident detection and auditing.

4.3 Scope of Testing

In-scope areas

1. Authentication & Authorization
 - Test Registration and login using MFA for Patients and Doctors.
 - Test Role-based view on content. For instance, patients see only their own data while doctors see data of every patient assigned to them.
2. Transport Security

- Enforce BLE Secure Connections (LESC) with numeric comparison pairing. App must reject connections from devices attempting Legacy Pairing to prevent Man-in-the-Middle attacks.
- Confirm Http over TLS 1.3 for all traffic between the mobile app, the dashboard and the cloud services through AWS API Gateway and AWS VPC restriction exists on inbound traffic from external.
- 3. API and Backend Security
 - Test AWS WAF rules are in place for preventing SQL injection and XSS filtering before AWS API Gateway.
 - Test parameterized queries in PostgreSQL and error handling mechanisms in Python microservices.
- 4. Data Protection & Storage
 - Confirm AES-256 encryption at rest for AWS RDS PostgreSQL PHI data.
 - Authentication credentials should be stored and handled separately from PHI. Test passwords are only stored as hashes using bcrypt.
- 5. Access Management
 - Ensure policies are set up for EC2, RDS and API Gateway to enforce least-privilege access.
 - Secret and key management by ensuring no hardcoded data and OAuth2.
- 6. Client-Side Security (Dashboard & Mobile App)
 - React should provide protection against XSS attacks within dashboard.
 - Test secure session handling, JWT header safety and proper logout behavior in the mobile app.
- 7. Logging, Monitoring and Alerts
 - Ensure CloudWatch for monitoring and CloudTrail for logging are configured properly. Ensure logs exclude PHI data and tokens for sessions.

Out-of-scope

- This project doesn't have the reach for hiring external penetration tester.
- Project cannot simulate Social-engineering attacks.
- Don't have tools for compromise of wearable devices and unsure of the legal elements as project does not involve making wearable device or design IoT security.

4.4 Test Objectives and Approach

Security Objectives

- SEC-1: To provide strong authentication and MFA for all user roles.
- SEC-2: To enforce role-based access control and protect PHI from unauthorized access.
- SEC-3: To protect all PHI in transit using TLS 1.3 and encrypted BLE.
- SEC-4: To protect all PHI at rest with AES-256 encryption.
- SEC-5: Mitigate common web and app vulnerabilities in OWASP Top 10.
- SEC-6: To provide auditable, detective; logging, monitoring and alerting of the system activities.

Approach

1. Based on requirements:
 - Derive security test cases directly from security Non-Functional and functional requirements in Software requirements specifications document and map them in the traceability matrix.
2. Authentication & Authorization Testing
 - Distinguish Valid and invalid login attempts and cause account lockout after repeated failures.
 - Test out different MFA scenarios such as reusing the same OTP for reliability.
 - Vertical and horizontal privilege checks to identify the set boundaries are in working.
3. Transport Layer Testing
 - Use browser tools and network proxies to verify HTTPS-only (TLS1.3) access.
 - Confirm the mobile app refuses connection request to servers with invalid or expired certificates.
 - Validate that BLE traffic between wearable and app is encrypted by making sure app only supports BLE 5.0 and above.
4. API and Input Validation Testing
 - Use Postman to send crafted requests containing SQLi and XSS parameters to API endpoints.
 - Confirm that WAF blocks malicious traffic and error codes do not leak traces of internal details.
5. Data Protection Testing
 - Inspect RDS configuration for encryption at rest. RDS create recovery point backups for DB these should be verified if they are encrypted.
 - On a test device, inspect local storage of any PHI after login and after logout. Flutter assures data within app is AES-256 encrypted.
6. Client-Side Security
 - Inject XSS payloads into input fields and verify that output pages treat malicious code inputs as harmless plain texts instead.
7. Logging & Monitoring Testing
 - Trigger alerts with failed logins, unauthorized API calls and blocked WAF events and verify that corresponding entries appear in CloudWatch or CloudTrail.
 - Review logs to confirm no raw PHI or complete JWT tokens are stored in plain sight.

Security testing will be iterative and will re-run testing until goals are met.

4.5 Test Schedule and Milestones

Phase	Week	Activities
Planning & Design	1	Finalize security requirements and define traceability matrix with test cases.

Environment Preparation	2	Configure security test environment.
Auth & RBAC Testing	3	Execute SEC-1 and SEC-2 test cases on mobile and dashboard.
API & Input Validation	4	Run OWASP-style tests on API Gateway, WAF and microservices.
Data Protection & Storage	5	Validate data encryption and secure storage and backups.
Logging & Monitoring	6	Assess internal logging, CloudWatch and CloudTrail.
Security Regression	7	Re-execute critical test cases upon failures and updates.
Final report	8	Prepare final report.

4.6 Test Environment Setup

Infrastructure

- AWS VPC replication production, API Gateway + WAF, EC2 AppServer, RDS PostgreSQL (test instance), CloudWatch/CloudTrail.

Client Platforms

- Test Android /iOS devices with Adaptiv Health mobile app build.
- React dashboard Laptops/desktops with Chrome/Edge.

Configuration

- Test user pool Cognito/OAuth2 with Patient/Doctor/Admin.
- EC2, RDS and API Gateway IAM roles with the least privilege policies.
- ALL public endpoint TLS 1.3 certificates.

Tools

- Postman or such API client.
- OWASP ZAP scanner
- AWS Console/CLI to check security configuration.

Data

- Artificial PHI and user accounts; no actual patient information.

4.7 Resources and Responsibilities

Role	Responsibilities
Security Test Lead	Possesses security testing section, draws up test cases, organizes the performance and setting of the results.
QA Engineers	Carry out test cases, record evidence, record and retest defects.
Backend Developer	Fix API security test support, correct server-side vulnerability, change validation logic.
Cloud/DevOps Engineer	Install WAF, IAM, TLS, logging and monitoring; support performance impact tests.
Mobile Developer	Make sure that the storage of tokens and certificate technology is safe; correct security problems in the phone.
Frontend Developer	Install and resolve XSS/CSRF countermeasures on dashboard.
Project Manager	Security testing and remediation plan time in general schedule.

4.8 Test Deliverables

1. Security Testing part of the general Test Plan document.
2. Specification of detailed Security Test Case (ST TC xx).
3. Evidence (screenshots, tool report) Security Test Execution Log.
4. Security Defect Log severity, status and components affected.
5. Security Test Report with a risk assessment and recommendations.
6. Security Traceability Matrix between the design elements and the test cases and SRS security requirements (see Section 6.9).

4.9 Security Traceability Matrix

Req ID (exact from SRS)	Requirement Type	Requirement Description (shortened from SRS)	Design / Architecture Element(s)	Security Test Case ID	Test Case Name and description
SEC-1.1	Information Security NFR	The transit (TLS 1.3) and rest (AES 256) encryption of all sensitive data (user credentials, health data, etc.) should be performed.	TLS 1.3 on API Gateway & dashboard; BLE 5.3 - mobile app; AWS RDS PostgreSQL with AES 256, local secure storage on mobile.	ST-TC-01	Enforce HTTPS & TLS 1.3: Attempt connection via HTTP (Port 80). Pass condition: Server must return 301 Permanent Redirect to HTTPS or 403 Forbidden. Connection on port 80 must not return 200 OK.
SEC-1.1	Information Security NFR	Same as above	Networking of mobile application and local storage.	ST-TC-02	Local Data & Credential Encryption Check: check PHI and credentials, are not stored on device in plaintext form.



SEC-1.2	Information Security NFR	PHI should be stored only in HIPAA eligible clouds (AWS RDS, EC2, S3) which have successfully gone through complete testing.	AWS RDS PHI schema and EC2 microservices and S3 backups.	ST-TC-03	AWS RDS Encryption at Rest Verification: Check PHI tables are in only encrypted AWS RDS instance.
SEC-1.2	Information Security NFR	Same as above	Backup and export pipeline	ST-TC-04	Secure Backup & Export Compliance: Check PHI backups/exports stored in HIPAA eligible encrypted storage.
SEC-1.3	Information Security NFR	System will be in line with HIPAA, GDPR and DHA health data regulations.	General security design; data localization policy; consent and policy displays.	ST-TC-05	Data Localization & Retention Policy Check: Confirm data localization and retention (PHI stored in UAE region, retention policy of 25 year is in place)
SEC-1.3	Information Security NFR	Same as above	Privacy / consent module	ST-TC-06	User Consent & Privacy Log Verification: Confirm user consent, display and update logs of privacy policy.



SEC-1.4	Information Security NFR	Authenticate using OAuth 2.0 / OpenID connect with RBAC and rotation of the token after every 24 hours.	Cognito/ OAuth2 Auth Services; JWT in mobile app and dashboard; API authorizers.	ST-TC-07	JWT Algorithm & Signature Validation: Inspect JWT Token header after login. Pass condition: 'alg' must be 'RS256' (not 'None' or 'HS256' which is a symmetric algo meaning hackers can decompile the app and find the secret key). Token payload must contain the correct 'aud' (audience) claim matching the App Client ID.
SEC-1.4	Information Security NFR	Same as above	RBAC enforcement	ST-TC-08	Role-Based Access Control (RBAC) Enforcement: Check role based access: Patient vs Doctor vs Admin (UI + API)
SEC-1.4	Information Security NFR	Same as above	Token-life cycle logic	ST-TC-09	Token Expiry & Rotation Validation: Check on expiry of tokens and the 24 hour rotation; deny expired tokens.
SEC-1.5	Information Security NFR	Configure AWS GuardDuty for anomaly detection and	AWS GuardDuty for anomaly detection	ST-TC-10	AWS GuardDuty Anomaly Detection: Trigger simulated port scan and SSH brute force on EC2 instance.



		AWS Macie for sensitive data discovery in S3 buckets.	and AWS Macie for sensitive data discovery in S3 buckets.		Verify AWS GuardDuty generates a 'Recon:EC2/PortProbe' finding.
SEC-1.5	Information Security NFR	Same as above	DLP rules	ST-TC-11	AWS Macie PII Discovery: Upload a file containing dummy credit card numbers to S3. Verify AWS Macie flags the object as 'High Severity' PII.
SEC-1.6	Information Security NFR	Same as above	User profile & consent screens	ST-TC-12	User-Initiated Account Anonymization: Ensure that users can anonymize / delete accounts through UI and confirm.
REQ-1.5	Functional Requirement	Authentication and Data encryption (log, MFA, TLS 1.3, AES 256, logout).	Auth/Cognito, Firebase Auth/SMTPS, Biometric, TLS, AES 256, login page.	ST-TC-13	Brute Force Test: Patient/Doctor/Admin: invalid password and 3 attempt lockout.
REQ-1.5	Functional	Same as above	Error-handling and logging	ST-TC-14	Invalid Credential Error Handling:



	Requirement				Check invalid credentials and session expired messages are displayed; failed attempts are registered.
REQ-1.5 + SEC-1.1	Functional + Security	Secure session management (30 minute unattended, encrypted traffic).	App & dashboard session failover; TLS.	ST-TC-15	Session Inactivity Timeout Enforcement: Leave app idle for 31 minutes. Attempt to refresh feed. Pass condition: App must redirect to Login Screen. API must return 401 Unauthorized for the stale token.
SEC-1.3	Regulatory Compliance (HIPAA)	System must maintain an immutable audit trail of all access to PHI as per HIPAA Security Rule §164.312(b) - Audit Controls.	CloudTrail (AWS API logging), CloudWatch Logs (Application-level logging), Backend Middleware (Access logging per request).	ST-TC-16	HIPAA Audit Trail Verification Action: 1. Login as Doctor Role (User: dr.smith@test.com) 2. Navigate to Patient Dashboard and view "Patient X" (ID: P-00123) medical records 3. Wait 5 minutes for log propagation 4. Query CloudTrail and CloudWatch Logs for Request ID



					associated with the action Verification: Search logs for API call: GET /api/patients/P-00123/records Pass Condition: - Log entry MUST contain: Timestamp, UserID (dr.smith@test.com), Patient ID (P-00123), Action (GET /records), Source IP Address, Response Code (200 OK) - Log MUST NOT contain actual PHI content (heart rate values, medications, etc.) - CloudTrail Log File Validation MUST be enabled to ensure tamper-proof audit trail (integrity check) - Log retention MUST be set to minimum 6 years (HIPAA requirement)
--	--	--	--	--	--

4.10 Risks and Mitigation Strategies

Risk ID	Risk Description	Impact	Likelihood	Mitigation
ST-R1	Testing of security is initiated at the later stages and there is not enough time to correct the problems.	High	Medium	First integrated build security testing Schedule security testing; consider critical security defects as release blockers.
ST-R2	Production (IAM, WAF, TLS) and test environment are not the same, which diminishes validity of results.	Medium	Medium	Production configuration Mirroring IaC/scripts, document and review all changes then test.
ST-R3	Misconfiguration of IAM Policies.	Medium	Medium	Use AWS IAM Policy Simulator to validate permissions. Implement 'Least Privilege' by starting with Deny-All and whitelisting only required actions (e.g., s3:GetObject only for specific buckets).
ST-R4	Inadvertent logging of Sensitive Data.	High	Medium	Configure CloudWatch Logs 'Data Protection' policies to automatically mask patterns matching Emirates ID or Credit Card formats.
ST-R5	IAM roles are wrongly configured, and users have unlimited access to PHI or administration capabilities.	High	Medium	Checklist Review IAM policies against least privilege against least privilege checklist; Sociability (implement automated checks); re test after any role change.

ST-R6	The security measures (WAF, encryption) affect the user experience and performance negatively.	Medium	Low	Measure the security tests latency; modify WAF rules and scale resources; add performance tests to security regression.
ST-R7	This stage does not address all the OWASP categories of the Top 10.	Medium	Low	Proof of risk: Push to the top the most appropriate risks (Injection, Broken Auth, Sensitive Data Exposure); Record the rest of the risks and schedule a subsequent penetration test.

5. Machine Learning Testing

5.1 Test ID and Title

Test ID | Test Title

AH-ML-001 | Adaptiv Health Machine Learning Model Testing Plan

5.2. Introduction and Objective

Introduction

The risk prediction system, which is the key intelligence of Adaptiv Health, is validated in machine learning testing. The system is based on the training of a Random Forest classifier in 973 cardiac rehabilitation patients to predict the cardiovascular risk scores of 0.0 (safe) to 1.0 (critical danger). This ML model will be served on the backend FastAPI server and reached through various API endpoints such as /api/v1/predict/risk to make real time prediction when a patient is working out and /api/v1/risk-assessment/compute to predict the risk comprehensively.

The ML model is critical to test since patient safety depends on the accuracy of predictions made by the model. False negative (model does not identify high risk) may enable a cardiac patient to engage in dangerous exercise, which may result in heart arrest. On the other hand having too many false positives would render the system useless as it will keep on raising false alarms. The model should attain a minimum of 95% accuracy and 98%+ recall to be clinically viable i.e. it should identify at least 98 of 100 genuine dangerous situations.

The ML system can be visualized as a set of interconnected elements: (1) Feature engineering: 16 of the derived features, such as hrpctofmax, durationintensity,

recovery efficiency, and others, are calculated based on 9 direct measurements, including: (16 features) and (9 direct measures); (2) The trained random forest model, riskmodel.pkl (679 KB), including its StandardScaler, scaler.pkl; (3) Anomaly detection algorithms: the abnormal patterns of vital attributes (HR > 180 BPM The validation of the components should be detailed.

The ML model does not work in isolation, as it has an integration with rest of the system architecture. After a patient has been through a workout, the mobile application transmits the vital sign data through POST /api/v1/sessions to the backend. The backend uses all 16 features, loads the ML model, predicts, identifies the risk level (LOW/MODERATE/HIGH) and saves the results in PostgreSQL. In case risk-score is higher than 0.85, the system will automatically generate a safety alert that will appear in the doctor dashboard. This whole pipeline needs to be tested in general.

Objective bullet points:

- Validate RandomForest model gets 95-97 percent accuracy in 195 sample test (20 percent hold out).
- Confirm rate of high-risk cases (max 2 missed dangerous sessions of 81) is 98-100%.
- Checking accuracy is 90-95% in order to reduce false alarms among the patients.
- Check the 16 feature calculations mathematically.
- Validate prediction latency <|human|>Real-time alert 160ms to validate prediction latency.
- HR > 180 BPM and SpO₂ < 90 percent are correctly identified as test anomalies.
- Verify baseline optimization makes adjustments to thresholds based on patient history.
- Accurate confirmation confidence scoring is a good measure of prediction reliability.
- Load of test models error-free using.pkl files.
- Test all ML endpoints with API.
- Edge cases: invalid data (SpO₂=null), extreme values (HR=200), aged patients (age=80)
- Check risk classification limits are working correctly (>0.85=HIGH, 0.70-0.85=MODERATE, <0.70=SAFE)

5.3 Scope of Testing

Core Model Validation:

ML testing involves training a primary risk prediction model of the RandomForest on the 195-session holdout test set which was not observed at all during training. This

model should show that there is a 95-97% overall accuracy i.e. 185-189 correctly classified sessions out of 195. More importantly, recall should be 98-100% on the estimated high risk (riskscore > 0.85) 81 test set sessions, allowing up to 1-2 false negatives. The analysis of the confusion matrix will determine some systematic errors in prediction, including the systematically missed patients who are old or those who have various cardiac diseases. The importance of the features will be tested to verify that the pattern of importance of the features continues with the same important variable (hrpctofmax) with an estimated importance of around 48%.

Feature Engineering Validation:

The majority of the 16 model features that are engineered calculations as opposed to direct measurements are 7 in total and a formula should be tested individually. An example would be that of the most important feature, i.e. $\text{hrpctofmax} = \text{peakheartrate} / \text{maxsafehr}$ where a calculation error here would nullify all predictions. Equally, $\text{recoveryefficiency} = \text{recoverytimeminutes} / \text{durationminutes}$ is required to support edge cases such as $\text{duration}=0$. Testing will involve a comparison of calculated features with manual calculations of a sample of 20 different sessions with edge cases (very short workouts, missing SpO2, extreme heart rates). The StandardScaler should appropriately scale features with the same mean and standard deviation of training.

Anomaly Detection:

The aberrant identification system recognises the vital sign configurations which are of critical vital conditions that must be addressed immediately irrespective of the urgency of the ML forecasts. Experiments confirm that a HR of more than 180 BPM is treated as a critical warning, and below 90% SpO2 is an indication of hypoxemia and combination such as longer than 5 minutes with HR at or greater than 90 percent maxsafehr is detected. The system should be able to differentiate between genuine anomalies and sensor malfunctions (e.g. when the HR suddenly spikes 80 to 220 BPM it is not a cardiac event, but a sensor malfunction). Testing is done with artificial anomaly cases and with actual end cases of the database.

API Endpoints Integration:

ML is exposed to all API endpoints using a RESTful API. The following can be validated through testing: POST /api/v1/predict/risk: (1) accepts feature vectors and returns riskscore + risk level + confidence; (2) accepts complete session data and stores the results in the database POST /api/v1/risk-assessments/compute: (3) processes personalized thresholds and stores the results into the database; GET /api/v1/predict/status: (4) verifies that the model is loaded and therefore ready; GET /api/v1/anomaly-detection: (5) Errors (fields are missing, data types are invalid, nothing), each endpoint should be able to accept them.

What's not in scope:

Clearly excluded: (1) Clinical validation of real cardiac patients that need IRB approval and medical supervision; (2) model retraining processes because the model has already been trained and frozen at this stage of the testing workflow; (3) TensorFlow Lite conversion to run on-device mobile applications (future work), (4) sophisticated ML endpoints with labels as experimental (/api/v1/trend-forecast, /api/v1/recommendation-ranking) are not yet ready to production use; (5) Multi-model ensemble systems; (6) The focus of testing is simply in the validation of the existing production model of the RandomForest and the immediate supporting systems.

5.4 Test Objectives and Approach

The Model Performance Objectives,

- Test-retest reliability: 95-97% (185-189 of 195 correct)
- High-risk validation on recall: 98-100% (catch 79-81 out of 81)
- Confirm accuracy on high risk forecasts: 90-95%
- F1-score: >0.93 indicating good balance.
- Create results matrix with a maximum of 2 false negatives.

Engineering Objective (Features):

- Calculation test hrpctofmax: peakheartrate/ maxsafehr.
- Test hrelevation calculation avgheartrate - baselinehr.
- Calculation of test hrrange: peakheartrate - minheartrate
- Calculation of test durationintensity: durationminutes x hrpctofmax.
- Test recoveryefficiency computation: recoverytimeminutes/ durationminutes.
- Computation of test spo2deviation: 98- avgspo2.
- Test agerisk_factor calculation: age/70.
- Successful validation of the 16 features on 20 different test cases.

Performance Objectives:

- Confirm prediction latency: less than 160ms within 95 percent of predictions.
- Test model load time: less than 2 seconds on server start up.
- Confirm parallel prediction processing: 50 or more simultaneous requests.

Goals of the Anomaly Detection:

- Test HR limit: accurately indicates HR > 180 BPM.
- Test spO 2 threshold: properly signals SpO 2 < 90%.
- Combined anomalies in the test: persistent dangerous HR of more than 5 minutes.
- Confirm false positive on normal variations.

API Integration Objectives:

- POST /api/v1/predict/risk using valid and invalid inputs.
- POST /api/v1/risk-assessments/compute POSTs to database.
- Predictive GET /api/v1/predict/status gives back model preparedness.
- Missing features, incorrect types of data Test error handling.

Approach:

Test Function	Description	Test Approach	Success Criteria
Model Loading	Load risk_model.pkl and scaler.pkl from disk	Import pickle, load files, verify model object type	Files load without errors,model is sklearn RandomForestClassifier
Feature Calculation	Calculate all 16 features from raw session data	Use Known inputs(age=55, peak_HR=125, max_safe_HR=130...), manually compute expected values, compare to code output	All 16 features match manual calculations within 0.01 tolerance
Accuracy Validation	Test model on 196-session holdout set	Load test set from CSV, run model.predict(), compare to actual labels, calculate accuracy	Accuracy = 95-97% (185-189 correct)
Recall Validation	Count how many high-risk cases were caught	Filter test set for risk_score > 0.85 (~81 sessions), count True Positives, calculate recall = TP / (TP + FN)	Recall ≥ 98% (catch at least 79 out of 81)
Prediction Speed	Measure average prediction latency	Time 100 predictions using time.time(), calculate average milliseconds	Average < 160ms, 95th percentile < 200ms
High-Risk case	Input that should predict HIGH risk	peak_HR=135, max_safe_HR=130, other normal	risk_score > 0.85, risk_level = "HIGH"

		values, run prediction	
Safe Case	Input that should predict SAFE	peak_HR=100, max_safe_HR=130, other normal values, run prediction	risk_score < 0.70, risk_level = "SAFE"
Missing SpO2	Handle missing optional feature	Set avg_spo2 = None or omit field, run prediction	System defaults to 98, prediction completes, no error
Extreme HR	Handle abnormally high heart rate	peak_HR=200, max_safe_HR=130, run prediction	Returns HIGH risk, no crash, handles gracefully
Anomaly Detection - HR	Detect dangerous heart rate	HR=190 BPM (>180 threshold), call /api/v1/anomaly-detection	Returns {"anomaly": true, "type": "critical_hr", "severity": "HIGH"}
Anomaly Detection - SpO2	Detect low oxygen	SpO2=85% (<90 threshold), call /api/v1/anomaly-detection	Returns {"anomaly": true, "type": "hypoxemia", "severity": "CRITICAL"}
API - Predict Risk	Test main prediction endpoint	POST to /api/v1/predict/risk with valid JSON, check response	Status 200, returns {"risk_score": 0.XX, "risk_level": "...", "confidence": 0.XX}
API - Invalid Input	Test error handling	POST to /api/v1/predict/risk with missing field (no age)	Status 400, returns {"error": "Missing required field: age"}
API - Model Status	Check if model is ready	GET /api/v1/predict/status	Status 200, returns {"status": "ready", "model_loaded": true, "last_prediction": "..."}
Confidence scoring	Validate Prediction Confidence	High Consensus case (all trees	confidence > 0.90 for clear cases,

		agree), check confidence	lower for borderline
Feature Importance	Verify hr_pct_of_max is top feature	Check model.feature_importances_	hr_pct_of_max has highest importance (~0.48)

5.5 Test Schedule and Milestones

Phase	Weeks	Milestone	Deliverable
Test Environment Setup	Week 1	Test environment configured	Test dataset (195 sessions) extracted Model files (.pkl) accessible Python environment ready Postman collection created
Feature Calculation Testing	Week 2	All 16 features validated	Manual calculations for 20 test cases Code output matches expectations Edge cases handled correctly
Model Accuracy Testing	Week 2	Core metrics validated	Accuracy: 95-97% confirmed Confusion matrix generated Precision/recall calculated
Recall Validation (Critical)	Week 2	High-risk detection verified	Recall $\geq 98\%$ on 81 high-risk cases Max 2 false negatives identified False Negative cases analyzed
Performance Testing	Week 3	Speed benchmarks met	Average latency <160ms 100 predictions timed

			Concurrent load tested
Anomaly Detection Testing	Week 3	Alert thresholds validated	HR >180 detected SpO2 <90 detected Combined anomalies tested
API Endpoint Testing	Week 3-4	All ML APIs functional	All endpoints tested with Postman Valid/invalid inputs handled Error messages clear
Edge Case Testing	Week 4	Robustness confirmed	Missing data handled Extreme values tested Elderly patients validated
Integration Testing	Week 4	End-to-end flow works	Mobile → API → ML → Database Dashboard displays predictions Alerts trigger correctly
Final ML Report	Week 5	Documentation complete	Test results compiled Confusion matrix included Bug list with severity Recommendations documented

5.6 Test Environment Setup

Hardware Configuration:

Component	Specification
-----------	---------------

System	Laptop or Desktop
RAM	Minimum 8 GB RAM
Processor	Intel Core i5
Storage	5GB Free Space
GPU	Not required, ML runs efficiently on CPU

Software Configuration:

Component	Specification
OS	Windows 10/11 or macOS12+
Python	Python 3.9+
ML Libraries	scikit-learn 1.5.0 numpy 1.26.2 pandas 2.1.4 joblib 1.3.2 (for .pkl loading)
Backend Framework	FastAPI 0.104+ or Flask 3.0+
API Testing Tool	Postman 10+ (with ML endpoint collection)
Database Client	psycopg2-binary 2.9+ (to query test data)
Development IDE	VS Code with Python extension
Version Control	Git (to track test scripts)

Model Files:

Files	Size	Purpose	Location
risk_model.pkl	679 KB	Trained RandomForest classifier	/app/models/ or root directory
scaler.pkl	2 KB	StandardScaler for feature normalization	Same as model
feature_columns.json	1 KB	Feature names in correct order	Same as model

Test Data:

Dataset	Size	Description	Source
Training Data	778 sessions (80%)	Used to train model (not for testing)	Already used, archived
Test Data	195 sessions (20%)	Held-out validation set	Extracted from AWS RDS or CSV
High-Risk Subset	~81 sessions	Sessions with risk_score > 0.85	Filtered from test set
Safe Subset	~114 sessions	Sessions with risk_score ≤ 0.85	Filtered from test set
Edge Cases	20 sessions	Extreme/missing values for robustness	Manually created or selected

Network Configuration:

Component	Specification
Connection	Stable WIFI or Ethernet
Backend URL	http://localhost:8000
Database URL	adaptiv-health-db.c3kc22cw4z0f.eu-north-1.rds.amazonaws.com:5432
API Endpoints	/api/v1/predict/risk, /api/v1/predict/status, /api/v1/anomaly-detection, etc.

Environment Variables

Required for API testing

DATABASE_URL=postgresql://postgres:Adaptiv123!@adaptiv-health-db...

MODEL_PATH=./risk_model.pkl

SCALER_PATH=./scaler.pkl

FEATURES_PATH=./feature_columns.json

5.7 Resources and Responsibilities

Role	Team Member	Responsibilities
------	-------------	------------------

ML Model Tester & Coordinator	Omar	Chair ML testing and liaise with other staff. Confirm the accuracy of overall models (95-97%) and recall (98-100%). Run model on 195 sample test set and construct confusion matrix Test prediction speed (<160ms) with 100 predictions Imp., feature, documents (confirm hrpctof_max 48%) Integration of API tests with /api/v1/predict/risk endpoint. Prepare end ML test report with all measurements. Current ML leads to demonstration of prototype.
Feature Engineering & Database Tester	Abdallah	Check all 16 feature calculations as mathematically correct. Test code with 20 cases manually calculated. Missing SpO2, extreme HR, elderly patients (test edge cases). Select query database to retrieve test set (195 sessions of activity sessions table). Ensure that database stored sessions possess the appropriate feature values. FT Test aberrancy levels (SpO2 <90, HR >180) Help Omar to analyze false negatives and false positives. Record feature-calculation bugs.
Backend API Support	Bithin & Hamza	Offer running backend API testing of ML

		endpoint. The correctly loaded confirm model files (.pkl) are loaded at startup of a server. Help with debugging of any API errors in ML predictions. Check local API-based predictions of riskscore. Check the generation of safetyalerts with risk score 0.85 and above. support ML endpoint Postman collection.
Frontend Integration Support	Karylle & Ghina	Athlete Test that API riskscore is properly shown in dashboard. Confirm color coding (red/yellow/green) by risk levels. Mobile app predicts risk properly after workout. Test the presence of alerts in dashboard on high-risk detected.
Documentation Reviewer	All Team Members	Check ML test report against completeness. Check all the test cases run and results recorded. Accept ultimate measurements prior to prototype presentation.

5.8 Test Deliverables

Phase	Deliverable	Format	Owner
Feature Calculation Report	Table showing all 16 features with: Feature name, Formula, Test input values, Expected output, Actual output, Pass/Fail status	Excel or Markdown table	Abdallah

Model Accuracy Report	Document including: Test set description (195 sessions), Overall accuracy percentage (95-97%), Confusion matrix (2x2 table), Precision and recall for both classes, F1-scores, Screenshot of classification report	PDF or Word doc	Omar
Confusion Matrix	Visual 2x2 matrix showing: True Positives (correctly identified high-risk), True Negatives (correctly identified safe), False Positives (false alarms), False Negatives (missed dangers - max 2)	Image (PNG) or table	Omar
Recall Validation Report	Analysis of high-risk detection: Total high-risk cases in test set (~81), Number caught (79-81), Number missed (0-2), Recall percentage (98-100%), Details of any false negatives	PDF section	Omar
Speed Benchmark Report	Performance metrics: Average prediction time over 100 predictions, 95th percentile	PDF with graphs	Omar

	latency, Concurrent prediction test (50 simultaneous), Model loading time on startup, Graph showing latency distribution		
Edge Case Testing Results	Test results for: Missing SpO2 (defaults to 98) Extreme HR values (200 BPM) Zero duration (edge case) Elderly patients (age 75+) Very short workouts (<5 min) Pass/Fail for each	Excel or table	Omar
Anomaly Detection Report	Validation of thresholds: HR > 180 correctly flagged SpO2 < 90 correctly flagged Sustained dangerous HR detected False positive rate on normal variations	PDF section	Abdallah
API Integration Evidence	Postman test results: Screenshots of POST /api/v1/predict/risk with valid input Response showing correct risk_score and risk_level Error handling tests (missing fields, invalid types) GET /api/v1/predict/stat	Postman collection export + screenshots	Abdallah

	us response		
Feature Importance Chart	Bar chart or table showing: All 16 features ranked by importance Confirmation that hr_pct_of_max is highest (~48%) Top 10 features listed	Image (bar chart)	Omar & Bithin
Final ML Test Summary	Comprehensive report including: Executive summary (pass/fail, key metrics) All above deliverables compiled Bug list (if any issues found) Recommendations for improvements Sign-off from team lead	PDF (10-15 pages)	Omar

5.9 Entry and Exit Criteria

Entry Criteria (also known as conditions that must be satisfied before testing starts):

Model Readiness:

- Risk model file is accessible and available (679 KB) as risk model.pkl.
- scaler.pkl file available (2 KB)
- feature columns.json with proper order of features.
- Training was done on 778 sessions (80 percent of the data).
- Accuracy in training (original 100, post noise 95.9) recorded.

Data Readiness:

- Test set (20% holdout) of 195 session drawn out of database.
- Test set was assured to have not been observed during training.
- Risk sessions identified (n=81, risk_score>0.85)
- Safe (safe) sessions (~114) were found containing risk_score [?] 0.85.
- Ready an edge case test data (20 sessions with extreme/missing values)

Environment Readiness:

- Python 3.9 with scikit-learn 1.5.0.
- Model files are loaded without errors in Python script.
- Pandas DataFrame can be fed with test data.
- PHP Backend API is up and running.
- Postman assortment designed to test API.
- Connection to the database is functional (to query test data as necessary)

Documentation Readiness:

- Calculation formulae of features recorded.
- Spreadsheet of the test cases ready.
- Precision/recall goals set (95-97%, 98-100%)

Team Readiness:

- Omar and Abdallah are aware of the scope of ML testing.
- Duty of tests allocated properly.
- Channel of communication opened (WhatsApp)

Exit Criteria (conditions that must be met BEFORE testing ends):**Core Metrics Achieved:**

- Total accuracy: 95-97 confirmed on 195 sample test set.
- Treatment recall on cases of high risk: 98% (or more) (caught 79/81)
- Precision: 90-95% (acceptable false alarm rate)
- F1-score: >0.93
- At most, confusion matrix reveals 2 false negative.

Feature Validation Complete:

- Each of 16 features was tested using manual calculations.
- All the formulas of the features proved to be mathematically correct.
- Handled edge cases (extreme values, missing data)
- Standard Scaler ensured normalization.

Performance Requirement Met:

- Average prediction delay < 160ms (on 100 predictions)
- 95th percentile latency < 200ms
- Load time of model can be less than two seconds on server startup.
- Passed concurrent predict test (50 request at a time)

API Integration Validated:

- POST /api/v1/predict/risk gives right riskscore and risklevel.
- Handling of errors is effective (400 which is missing field/fields)
- GET /api/v1/predict/status Verifies loading model.
- Endpoints to anomaly detection operational.
- Members of API responses are in anticipated JSON format.

Robustness Confirmed:

- Missing SpO2 (defaults) 98.
- Extreme HR values are dealt with gracefully (HR=200 does not crash)
- Zero/negative values rejected with an evident error.
- Patients aged 75 and above were accurate predictors.
- Very short exercises (less than 5 min) managed.

Documentation Complete:

- Table of confusion created and evaluated.
- importance chart generated (hrpctofmax is ca. 48)
- All the cases that have been used in performing the test and the results that have been captured.
- List of bugs (identified) made.
- Written final ML test report (10-15 pages).
- Improvement recommendations recorded.

Integration Verified:

- Flow tested: workout data - API - ML - database - dashboard.
- Safetyalerts are made in the database by high-risk sessions.
- Proper risk API scores are displayed in dashboard.
- Mobile application picks the right predictions.

Sign-Off Obtained:

- Omar accepts the accuracy of ML test results.
- Abdallah validates calculations of features.
- Team lead (Omar) certifies that it is ready to present a prototype.
- No bugs left open that are critical.

Acceptance:

- ML model works on or better than the target metrics.
- System in readiness to demonstrate prototype by January 27.
- Belief that model will work well in demo.

5.10 Risks and Mitigation Strategies

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy	Contingency Plan	Owner
ML-R01	Model accuracy drops below 95% during test set validation	Medium	Critical	Verify test set is truly held-out (never used in training) Check feature calculation order matches training Confirm scaler.pkl is from same training run Review any changes to feature engineering code	If accuracy <95%: Re-examine training data for issues, consider retraining with more data or different hyperparameters	Omar
ML-R02	Recall falls below 98% (missing >2 dangerous cases)	Low	Critical	Analyze false negative cases in detail Check if they're all borderline (risk_score ~0.84) Consider lowering threshold from 0.85 to 0.80 Review if medical complexity noise was too aggressive	If recall <98%: Lower classification threshold to 0.80, document why this change was needed, re-test	Omar
ML-R03	Feature calculation differs between	Medium	High	Compare feature_columns.json order with	If mismatch found: Fix API code immediately,	Abdallah

	training and API code			API code Manually calculate features for 5 test cases Run both training script and API on same input, compare Add unit tests for each feature calculation	re-test all predictions, document the bug	
ML-R04	Model files (.pkl) corrupted or incompatible Python version	Low	High	Keep backup copies of risk_model.pkl and scaler.pkl Document exact Python version used (3.9+) Document scikit-learn version (1.5.0) Test model loading immediately in environment	If corrupted: Restore from backup, if no backup: retrain model (takes 2-3 hours)	Omar
ML-R05	Prediction latency exceeds 160ms (too slow for real-time)	Medium	Medium	Profile code to find bottleneck (feature calc or model) Optimize feature calculations (vectorize	If too slow: Accept 200ms as acceptable for prototype, document as future optimization	Omar

				with numpy) Consider loading model once at startup, not per request Test on faster CPU if needed		
ML-R06	API integration breaks (model not accessible from backend)	Medium	High	Test model loading in FastAPI startup event • Verify file paths are correct (MODEL_PATH env variable) Check file permissions on .pkl files Add error logging if model fails to load	If broken: Fix file paths, restart server, add startup health check endpoint	Bithin & Omar
ML-R07	Edge cases cause crashes (missing data, extreme values)	High	Medium	Test NULL/None for each feature Test extreme values (HR=0, HR=300, age=150) Add input validation in API (reject obviously invalid data) Set default values for optional fields	If crashes found: Add try-except blocks, return clear error messages, log the invalid input	Abdallah

				(SpO2=98)		
ML-R08	False positive rate too high (excessive false alarms)	Medium	Medium	Calculate precision on test set (target 90-95%) If precision <90%, consider raising threshold to 0.90 Analyze false positive cases (are they borderline?) Balance recall vs precision tradeoff	If FP rate >10%: Accept it for prototype (better safe than sorry), document for future tuning	Omar
ML-R09	Test data accidentally includes training data (data leakage)	Low	Critical	Verify train-test split was done correctly with random_state=42 Check that test set IDs don't overlap with training IDs Re-split data if necessary and re-run all tests	If leakage detected: Re-create test set properly, re-run ALL tests, document the issue	Omar
ML-R10	Limited time prevents full anomaly detection testing	Medium	Low	Focus on critical thresholds only (HR >180, SpO2 <90) Deprioritize	If time runs out: Document anomaly detection as partially tested, mark	Abdallah

				complex combined anomalies Test with synthetic data instead of finding rare real cases	as future work	
ML-R11	Baseline optimization and confidence scoring not implemented yet	High	Low	Confirm these are stretch goals, not core requirements Test only if time permits after core ML validation Document as "future enhancements" if not tested	If not implemented: Mark endpoints as "coming soon" in API docs, exclude from testing	Omar
ML-R12	Team member (Abdallah) unavailable during critical testing week	Low	Medium	Cross-train: Omar can do feature validation if needed Document test procedures clearly so anyone can run them Have backup plan to split responsibilities	If unavailable: Omar takes over all ML testing, extends timeline by 2-3 days	Omar

Risk Response Priority:

CRITICAL Risks (ML-R01, ML-R02, ML-R09):

- Rapid promotion to team lead.

- Test halted until it was solved.
- Status updates, which are necessary on a daily basis.

HIGH Risks (ML-R03, ML-R04, ML-R06):

- Address within 24-48 hours
- Immediately, mitigation strategies should be put in place.
- Document all actions taken

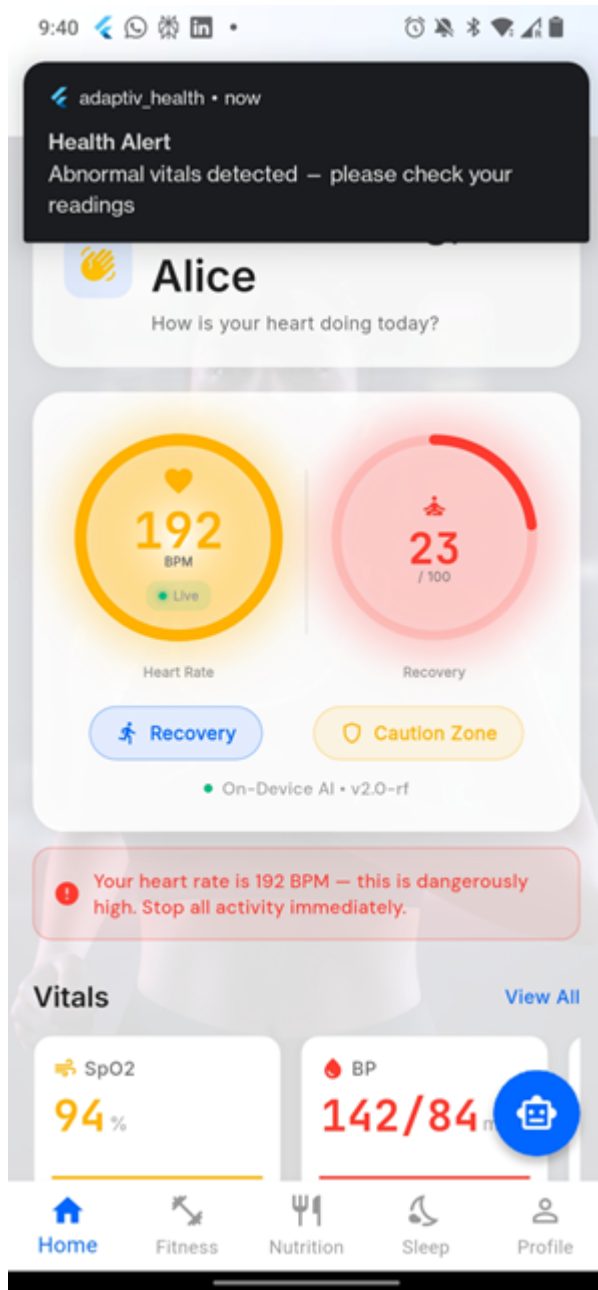
MEDIUM/LOW Risks (all others):

- Observation on typical testing.
- Be proactive in mitigations.
- Record workaround in case of problems.

Screenshots

Flutter App

Home screen showing 192 BPM alert + SpO2 + BP (shows real-time monitoring + alert working)



BLE device pairing screen (shows wearable connectivity)



9:35



← Pair Heart Rate Monitor

Active data source

Simulated (Demo Mode)

DEMO

Scan BLE Devices

Disconnect

Scan complete — took 0s

3 devices found

Health Connect

Samsung, Fitbit, Garmin, Polar, Apple Watch...

Connect

Or connect a BLE ...

Show all BLE devices

Scanning for all nearby BLE devices

Bluetooth icon

10:D7:EE:14:D0:0E

10:D7:EE:14:D0:0E - Microsoft

-52 dBm
Excellent

Last seen: 0s ago

Unknown device

Connect

71:28:3D:B5:AD:8B

71:28:3D:B5:AD:8B - 0x012D

-82 dBm
Weak

Last seen: 0s ago

Unknown device

Connect

Vfit Active

16:3A:7D:97:C7:17 - 0x0056

-85 dBm
Weak

Last seen: 0s ago

Unknown device

Connect

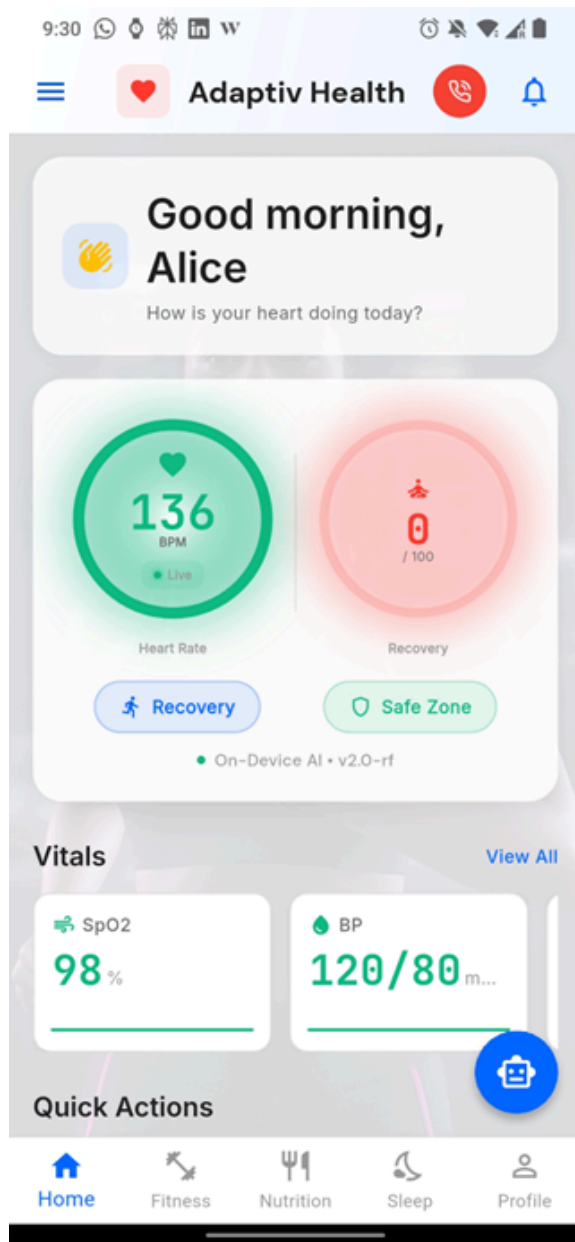
Onboarding screens

The figure displays three sequential screenshots of a mobile application titled "Lifestyle Screening".

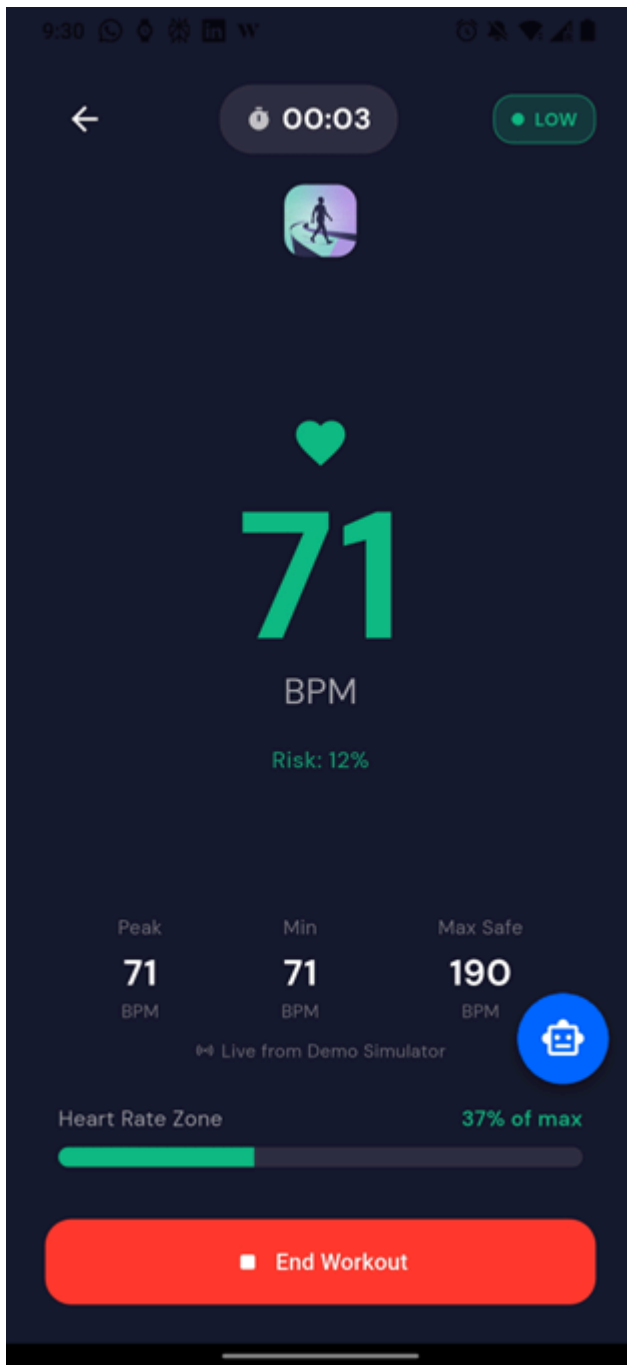
- Screenshot 1 (Left):** The main title is "Lifestyle Screening". Below it is the subtitle "This helps personalize your care recommendations." There are two sections: "Smoking Status" with options "Never" (selected), "Former", and "Current"; and "Alcohol Frequency" with options "Never" (selected), "Occasional", "Moderate", and "Heavy". At the bottom, there is a slider for "Sedentary Hours (per day)" set at 4.0, with "Oh" and "16h" markers.
- Screenshot 2 (Middle):** This screen shows the "Sedentary Hours (per day)" slider again, but now with a large blue "4.0" displayed above it. Below the slider is the "PHQ-2 Mood Check" section, which asks "Over the last 2 weeks, how often have you had little interest or pleasure in doing things?". It has four columns: "Not at all" (selected with a blue box and '0'), "Several days" (1), "More than half the days" (2), and "Nearly every day" (3). A second identical question follows.
- Screenshot 3 (Right):** The title is "Medical Background". The subtitle says "Select any conditions that apply to. This helps tailor safety thresholds." Under "Conditions", several pills represent selected conditions: Hypertension, Heart Failure, Coronary Artery Disease, Atrial Fibrillation, Diabetes, High Cholesterol, Asthma / COPD, Previous Heart Attack, and Previous Stroke. Below this is the "Current Medications (comma-separated)" field containing "e.g. Lisinopril, Metoprolol". At the bottom is the "Allergies (comma-separated)" field, which is currently empty.



Home dashboard "Good morning Alice" + vitals display

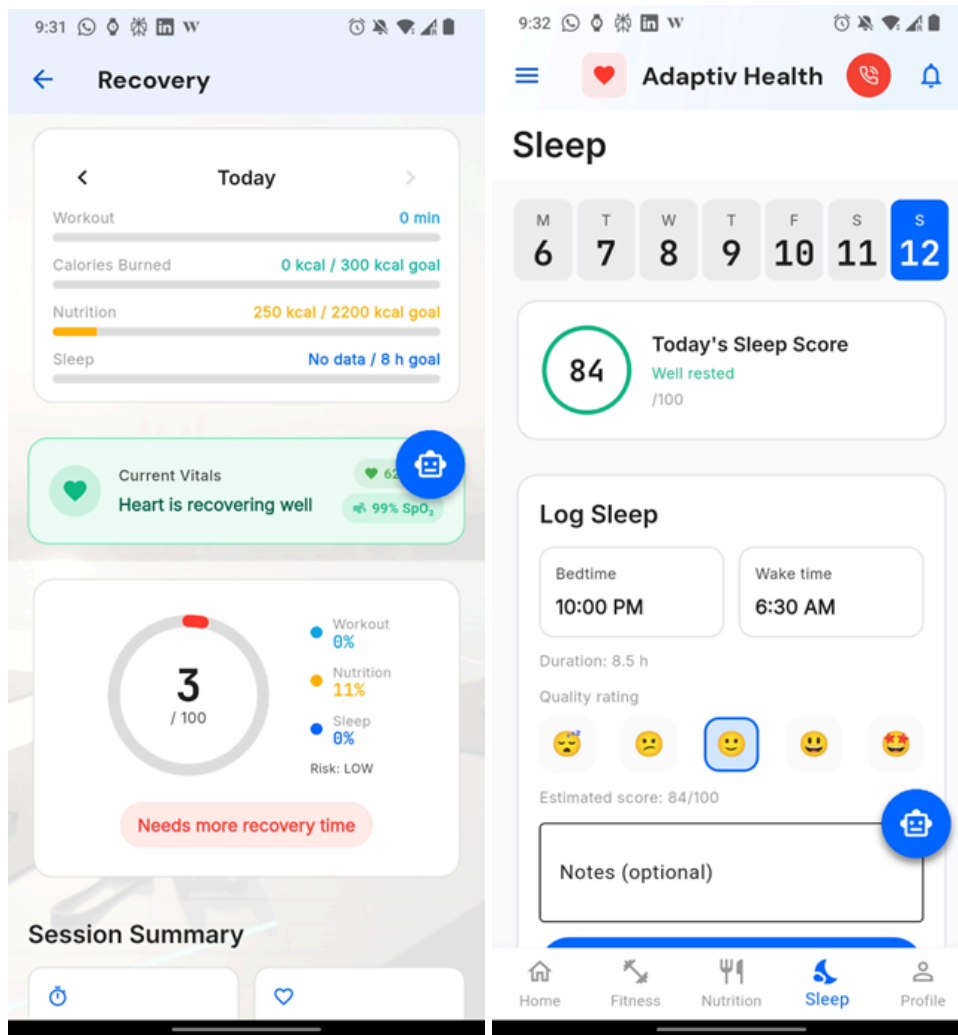


Fitness/Workout screen + active workout session



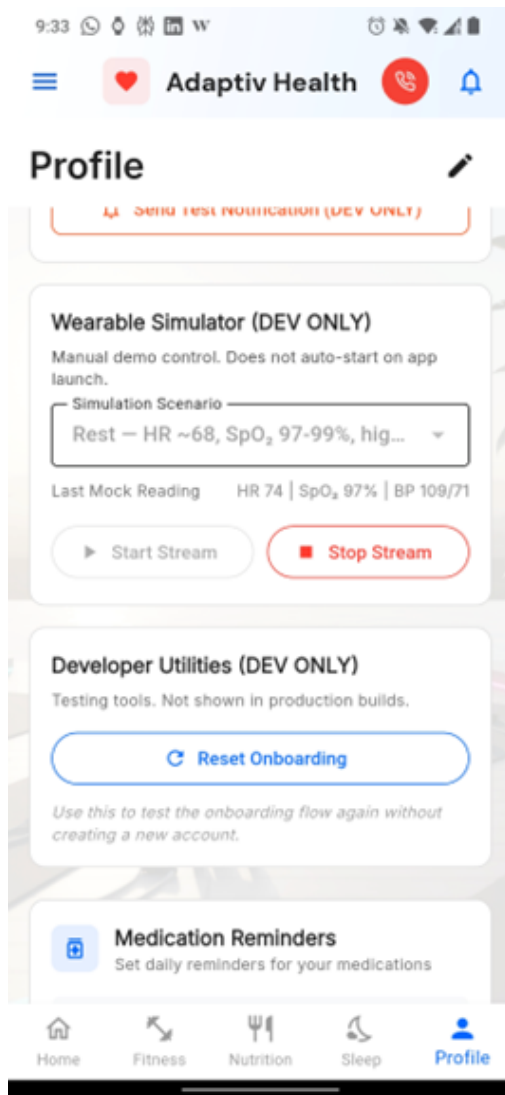


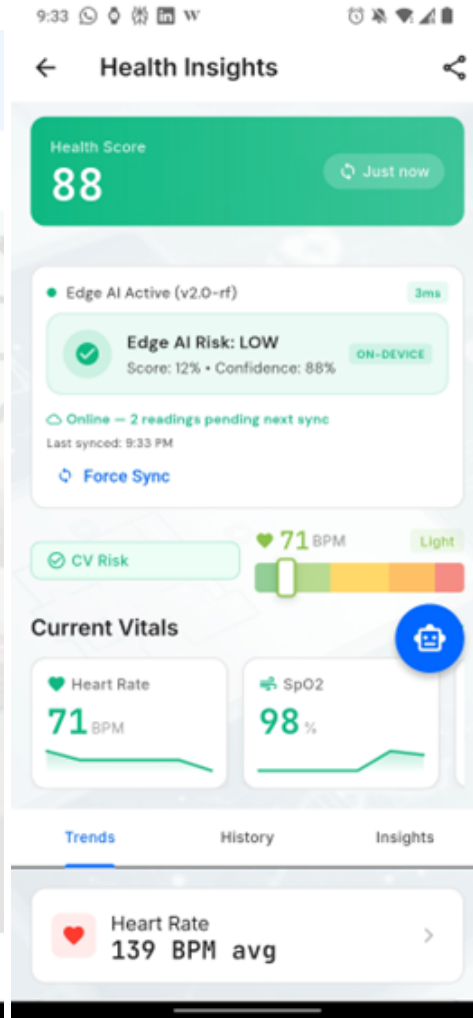
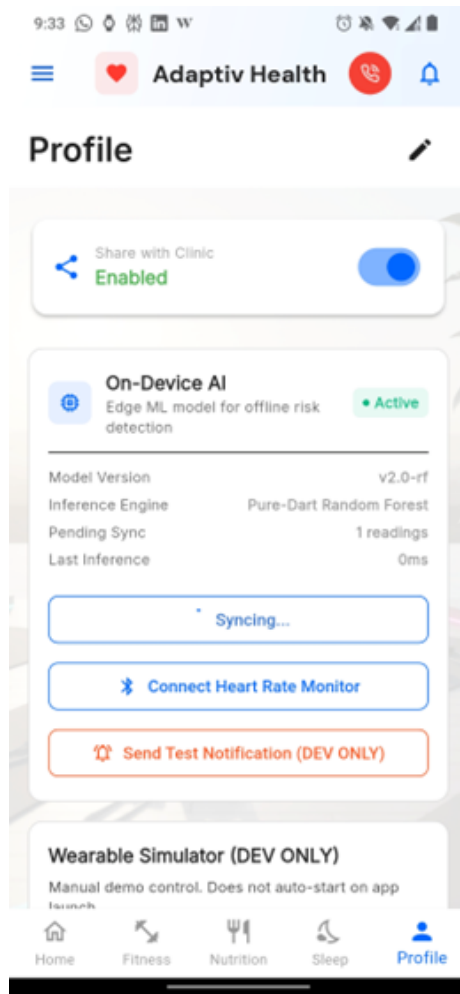
Recovery screen + Sleep screen





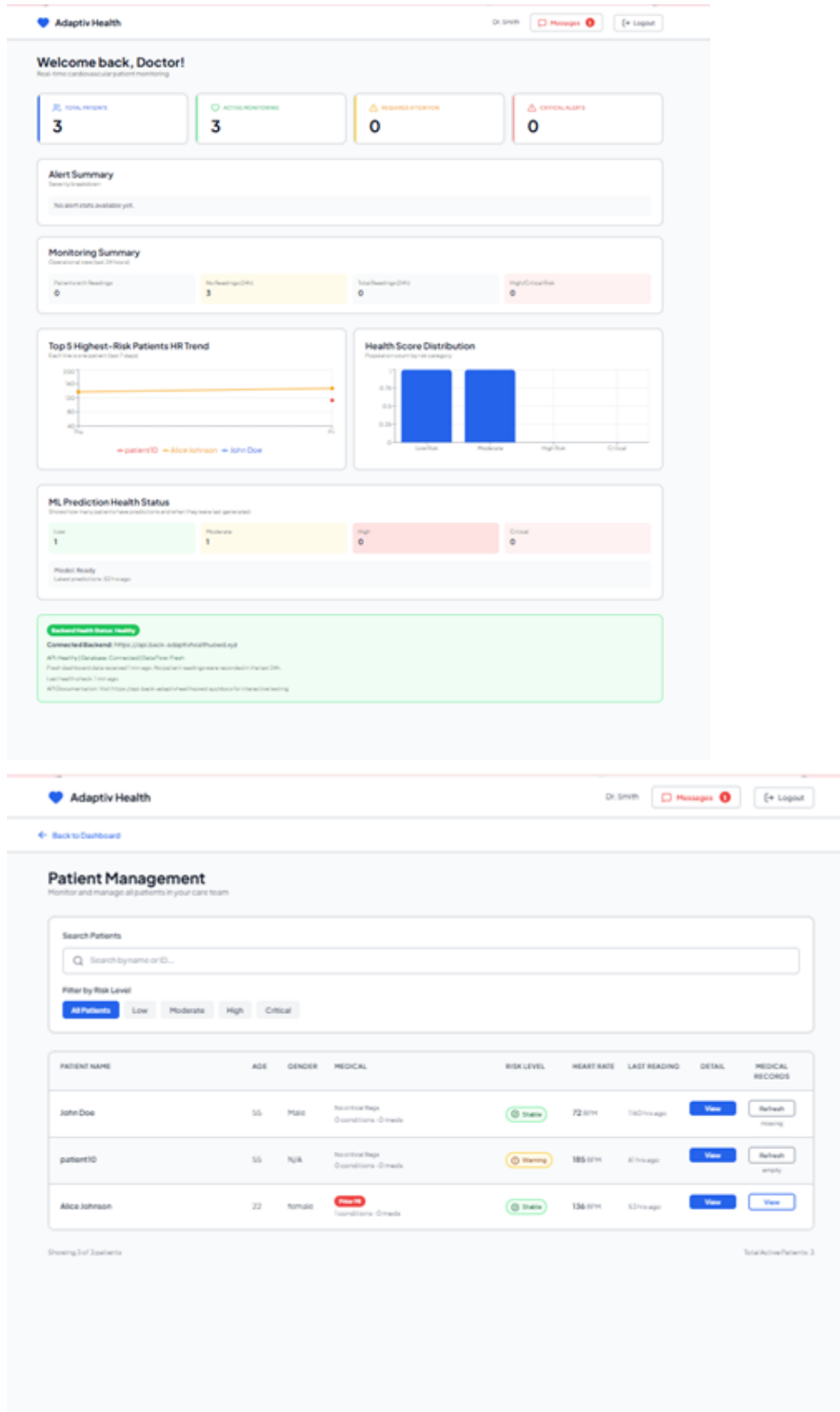
Profile showing On-Device AI active (v2.0-rf, Pure-Dart Random Forest)
+ Health Insights with Edge AI score

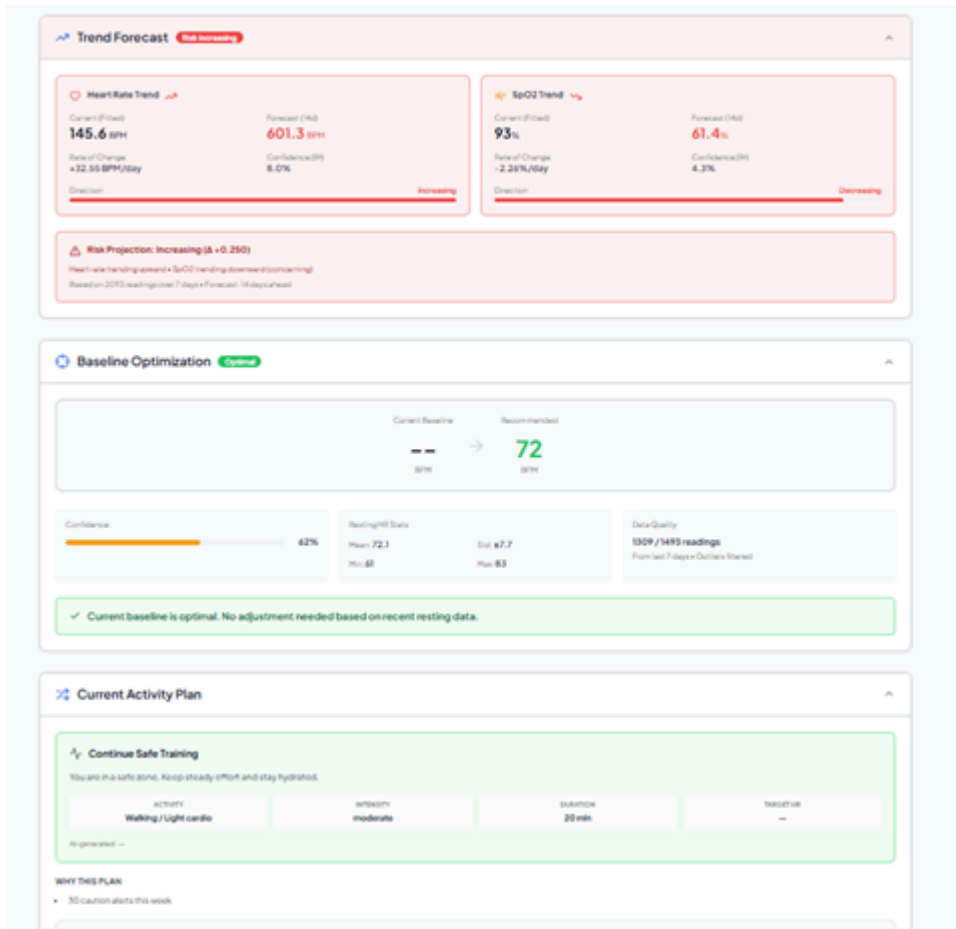




Doctor Dashboard

Overview + patient management table

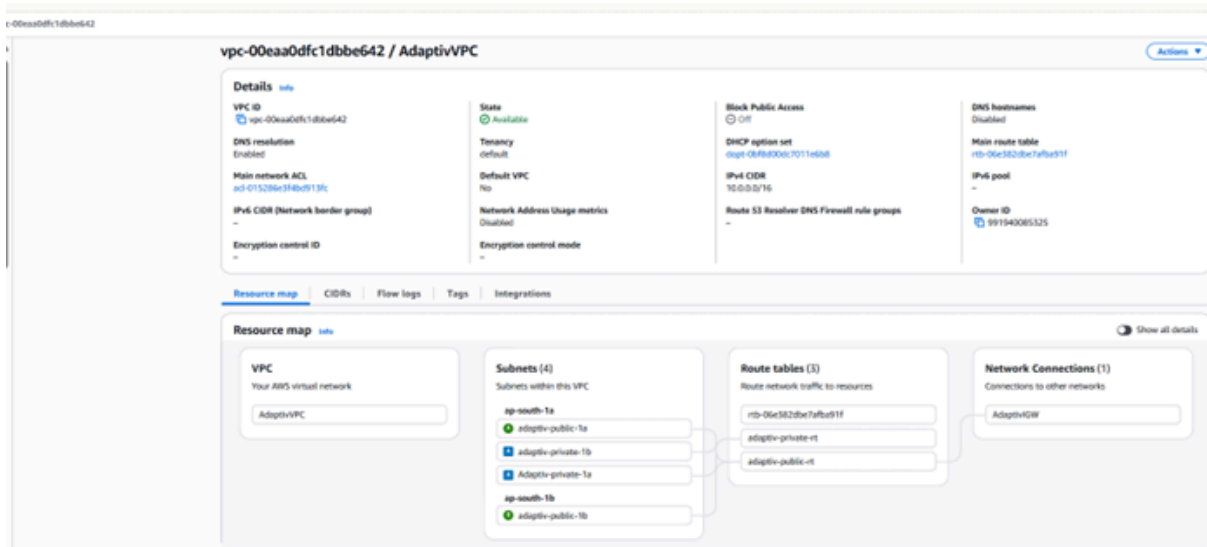






Backend/System

VPC resource map + API auth showing 401 Unauthorized



PostgreSQL terminal showing RBAC roles (PATIENT/CLINICIAN/ADMIN) + Swagger UI

```
ec2-user@ip-10-0-1-110:~  
#_ Amazon Linux 2023  
#####  
#####  
|  
\\#/  
V~' -> https://aws.amazon.com/linux/amazon-linux-2023  
  
Last login: Sun Apr 12 14:33:32 2026 from 37.40.238.120  
[ec2-user@ip-10-0-1-110 ~]$ psql "host=adaptivhealth-db.c34gaqco4qk4.ap-south-1.rds.amazonaws.com \  
port=5432 dbname=postgres user=postgres password=password"  
psql (15.15, server 17.6)  
WARNING: psql major version 15, server major version 17.  
Some psql features might not work.  
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)  
Type "help" for help.  
  
postgres=> -- a) See the enum definition  
SELECT n.nspname AS schema, t.typname AS type, e.enumlabel AS value  
FROM pg_type t  
JOIN pg_enum e ON t.oid = e.enumtypid  
JOIN pg_catalog.pg_namespace n ON n.oid = t.typnamespace  
WHERE t.typname = 'userrole'  
ORDER BY e.enumsortorder;  
  
-- b) See distinct values in the users table (if it exists)  
SELECT DISTINCT role FROM users;  


| schema | type     | value     |
|--------|----------|-----------|
| public | userrole | PATIENT   |
| public | userrole | CLINICIAN |
| public | userrole | ADMIN     |

  
(3 rows)  
  
role  
-----  
CLINICIAN  
ADMIN  
PATIENT  
(3 rows)  
  
postgres=>
```


161



Code	Details
200	Response body<pre>{ "id": 1, "user_id": 1, "heart_rate": 30, "spo2": 100, "blood_pressure": { "systolic": 70, "diastolic": 40 }, "hrv": 0, "source_device": "string", "is_valid": true, "confidence_score": 1, "activity_phase": null, "timestamp": "2026-04-12T15:49:49.414000Z", "created_at": "2026-04-12T15:49:57.327295Z" }</pre> Response headers<pre>access-control-allow-credentials: true access-control-allow-origin: http://localhost:8080 content-length: 278 content-type: application/json date: Sun, 12 Apr 2026 15:49:56 GMT server: uvicorn vary: Origin</pre>
Responses	

Code	Details
200	Response body<pre>{ "risk_score": 0.16, "risk_level": "low", "high_risk": false, "confidence": 0.84, "recommendation": "Safe to continue at current intensity.", "inference_time_ms": 252.75, "model_info": { "name": "RandomForest", "version": "1.0", "accuracy": "96.9%" }, "features_used": { "age": 18, "baseline_hr": 25, "max_safe_hr": 100, "avg_heart_rate": 25, "peak_heart_rate": 40, "min_heart_rate": 20, "avg_spo2": 85, "duration_minutes": 1, "recovery_time_minutes": 1, "hr_pct_of_max": 0.4, "hr_elevation": 0, "hr_range": 20, } }</pre>

Code	Details
200	Response body<pre>{ "recommendation_id": 1, "user_id": 1, "title": "Getting Started: Walking Plan", "suggested_activity": "walking", "intensity_level": "moderate", "duration_minutes": 20, "target_heart_rate_min": null, "target_heart_rate_max": null, "description": "Welcome to AdaptiveHealth! Start with a 20-minute brisk walk at a comfortable pace. This plan will be personalised once your first health assessment is completed.", "warnings": "Consult your physician before starting any exercise programme.", "created_at": "2026-04-12T15:44:24.257868+00:00" }</pre> Download

All References

1. HIPAA Law

Health Insurance Portability and Accountability Act of 1996, Pub. L. No. 104-191, 110 Stat. 1936 (1996).

2. Dubai Health Data Law / DHA

Dubai Health Authority. (2022). *Policy for Health Data Protection and Confidentiality* (HISHD/PP-11). <https://www.dha.gov.ae>

3. AWS HIPAA-Eligible Services

Amazon Web Services. (2025, November 5). *HIPAA compliance - Amazon Web Services (AWS)*.

<https://aws.amazon.com/compliance/hipaa-compliance/>

4. KNNImputer/scikit-learn

Géron, A. (2019). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems* (2nd ed.). O'Reilly.

scikit-learn developers. (2024). *KNNImputer – scikit-learn documentation*.

<https://scikit-learn.org/stable/modules/generated/sklearn.impute.KNNImputer.html>

5. Flutter

Google. (2025, October 31). *Flutter documentation*. <https://docs.flutter.dev>

6. TensorFlow

Abadi, M., et al. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems*. Software available from tensorflow.org.

- Elvas, L.B., Almeida, A. & Ferreira, J.C. (2025) 'The Role of AI in Cardiovascular Event Monitoring and Early Detection: Scoping Literature Review', *JMIR Med Inform*, 13, e64349. Available at: <https://medinform.jmir.org/2025/1/e64349>
- Bhaltadak, V. et al. (2024) 'A Comprehensive Review on Advancements in Wearable Technologies: Revolutionizing Cardiovascular Medicine', *PMC – NCBI*, PMC11212841. Available at: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11212841/>
- Amann, J., Blasimme, A., Vayena, E., Frey, D., & Madai, V. I. (2020). Explainability for artificial intelligence in healthcare: A multidisciplinary perspective. *BMC Medical Informatics and Decision Making*, 20(1), 310. <https://doi.org/10.1186/s12911-020-01332-6>
- HIPAA Journal. (2024). Best practices for HIPAA-compliant cloud storage and transmission. Retrieved from <https://www.hipaajournal.com/>



- Food and Drug Administration (FDA). (2024, December). AI/ML-enabled medical device guidance. U.S. Department of Health and Human Services.
<https://www.fda.gov/>
- AddWeb. (2025, October 23). Python AI libraries 2025. Retrieved from
<https://www.addwebsolution.com>
- AWS Healthcare Blog. (2025, September 21). Agentic AI healthcare. Amazon Web Services.
- European Commission. (2024). Guidelines on artificial intelligence and data protection: GDPR compliance in healthcare applications. Publications Office of the European Union.
- Stanford Medicine. (2024, September 2). Synthetic data in healthcare: Opportunities and risks. Stanford University Press.
- World Health Organization. (2024) *Cardiovascular Diseases (CVDs): Key Facts*. WHO. Available at:
[https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-\(cvds\)](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds))
- Goldstein, B., Saeed, M. & Mark, R.G. (2023) *MIMIC-IV: A Freely Accessible Critical Care Database for Artificial Intelligence and Data Science Research*, PhysioNet. Available at: <https://physionet.org/content/mimiciv/>